# RAPPORT DE STAGE DE PROJET DE FIN D'ÉTUDES

Intitulé du stage

## Conception et Développement d'une Plateforme Web de Gestion du Processus de Recrutement — JOB4YOU

Réalisé par : Haythem Bargaoui

SESAME
School of High-Tech, AI & Management

المدرسة العليا الخاصة للعلوم التطبيقية و إدارة الأعمال
ECOLE SUPÉRIEURE PRIVÉE DES SCIENCES APPLIQUÉES ET DE MANAGEMENT

Entreprise d'accueil

Sopra HR Software Tunisie

sopra hr
SOFTWARE

Encadrant Entreprise
M. Mones Ben Mabrouk

Encadrant SESAME
M. Malek Bouzaiene

**Année Universitaire : 2025 – 2026**

# Dédicace

*Je dédie ce travail à mes parents, dont l'amour inconditionnel, le soutien de chaque instant et les sacrifices ont rendu ce parcours possible. Aucun mot ne saurait pleinement exprimer ma gratitude à leur égard.*

*À mes frères et sœurs, dont les encouragements et la présence, y compris dans les moments les plus difficiles, m'ont accompagné tout au long de ce parcours.*

*À mes amis et collègues, pour leur soutien moral et les bons moments partagés durant cette expérience.*

*À toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail.*

# Remerciements

Au terme de ce projet de fin d'études, j'exprime ma sincère reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué à ce travail.

Mon encadrant entreprise, M. Mones Ben Mabrouk, m'a accompagné avec rigueur et disponibilité tout au long de ce stage ; ses conseils avisés et la confiance qu'il m'a accordée ont compté autant que ses retours techniques, déterminants dans la réussite de ce projet.

Du côté académique, M. Malek Bouzaiene a suivi mon travail avec attention, toujours disponible pour des conseils précieux qui ont orienté aussi bien la rédaction de ce rapport que la démarche méthodologique suivie.

J'ai également pu compter sur un accueil chaleureux de la part de toute l'équipe de Sopra HR Software Tunisie, dont l'esprit d'équipe et le partage d'expertise m'ont permis d'évoluer dans un environnement professionnel stimulant et formateur.

Je n'oublie pas non plus le corps professoral et administratif de SESAME, pour la qualité de la formation reçue tout au long de mon cursus d'ingénieur.

Enfin, je remercie les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail.

# Résumé

Mené chez Sopra HR Software Tunisie, ce Projet de Fin d'Études consistait à concevoir et développer JOB4YOU, une plateforme web destinée à centraliser et automatiser le cycle de recrutement interne de l'entreprise, de la publication des offres d'emploi jusqu'à la décision finale, en passant par le suivi des candidatures, la planification des entretiens et la collecte des feedbacks des managers.

J'ai construit la solution autour d'une architecture trois-tiers découplée : un back-end Java/Spring Boot expose des API REST sécurisées par JWT et gère finement les rôles (Administrateur, RH, Manager, Candidat), un front-end Angular assure une interface moderne et responsive, et PostgreSQL sert de base de données relationnelle. Des agents d'automatisation n8n, couplés à un service de scoring par intelligence artificielle (API de Cohere), prennent en charge l'analyse automatique des CV, en complément d'un système de notifications par e-mail à chaque étape clé du processus. Un assistant conversationnel (chatbot) fondé sur un modèle de langage via l'API OpenRouter permet en outre au candidat de suivre ses candidatures en langage naturel, à partir de ses seules données réelles, le contexte transmis au modèle étant constitué côté serveur par une liste blanche de champs qui exclut les informations internes du dossier.

Ce rapport présente successivement le cadre général du projet, l'analyse et la spécification des besoins, la conception de la solution, sa réalisation, puis un bilan de cette expérience et les perspectives d'évolution de la plateforme.

**Mots-clés : recrutement, plateforme web, Spring Boot, Angular, PostgreSQL, API REST, JWT, intelligence artificielle, automatisation, n8n, chatbot.**

# Abstract

During my End-of-Studies Project at Sopra HR Software Tunisia, I designed and built JOB4YOU, a web platform for managing the company's internal recruitment process end to end — from publishing job offers through application tracking, interview scheduling, manager feedback collection, and the final hiring decision.

The solution rests on a decoupled three-tier architecture: a Java/Spring Boot back-end exposes REST APIs secured by JWT authentication with fine-grained role management (Admin, HR, Manager, Candidate), an Angular front-end delivers a modern, responsive interface, and PostgreSQL handles relational storage. n8n automation agents, paired with an AI-based scoring service built on the Cohere API, handle automatic resume analysis, alongside an email notification system triggered at each key step. A conversational assistant (chatbot), based on a language model accessed via the OpenRouter API, also lets candidates track their applications in natural language, drawing on their own real data through a server-side allowlist of fields that excludes internal case details.

The report walks through the general project framework, requirements analysis and specification, the solution's design and implementation, before closing with an assessment of the experience and the platform's future development perspectives.

**Keywords: recruitment, web platform, Spring Boot, Angular, PostgreSQL, REST API, JWT, artificial intelligence, automation, n8n, chatbot.**

# Table des matières

Table des matières ..... 6
Liste des figures ..... 9
Liste des tableaux ..... 11
Introduction Générale ..... 12
Chapitre 1 : Cadre Général du Projet ..... 13
1.1 Introduction ..... 13
1.2 Présentation de l'organisme d'accueil ..... 13
1.2.1 Le groupe Sopra Steria ..... 13
1.2.2 Sopra HR Software ..... 13
1.3 Étude de l'existant ..... 13
1.4 Critique de l'existant et problématique ..... 14
1.5 Objectifs du projet ..... 14
1.6 Présentation de la solution proposée : JOB4YOU ..... 15
1.7 Méthodologie de travail adoptée ..... 16
1.8 Planification du projet ..... 16
1.9 Conclusion ..... 18
Chapitre 2 : Analyse et Spécification des Besoins ..... 19
2.1 Introduction ..... 19
2.2 Étude fonctionnelle ..... 19
2.2.1 Identification des acteurs du système ..... 19
2.2.2 Identification des besoins fonctionnels ..... 20
2.2.3 Identification des besoins non fonctionnels ..... 21
2.2.4 Diagramme d'activité — Vue d'ensemble du processus de recrutement ..... 22
2.2.5 Diagrammes de cas d'utilisation ..... 24
2.2.6 Description textuelle de quelques cas d'utilisation ..... 27
2.2.7 Diagramme de séquence système (boîte noire) ..... 29
2.3 Étude technique ..... 29
2.3.1 Architecture physique cible ..... 29
2.3.2 Côté client — Angular ..... 30
2.3.3 Côté serveur — Spring Boot ..... 30

2.3.4 La sécurité — Spring Security et JWT ..... 31
2.4 Conclusion ..... 31
Chapitre 3 : Conception ..... 32
3.1 Introduction ..... 32
3.2 Diagramme des paquetages ..... 32
3.3 Diagrammes de classes ..... 33
3.4 Séquence technique — Soumission asynchrone et scoring IA ..... 36
3.5 Diagrammes de séquence objet ..... 36
3.5.1 Planifier un entretien ..... 37
3.5.2 Décision finale de recrutement ..... 38
3.5.3 Analyse IA du CV (extraction, scoring via n8n et Cohere) ..... 39
3.5.4 Passage en évaluation technique — routage et notification des managers ..... 40
3.6 Diagramme d'états — Cycle de vie d'une candidature ..... 41
3.7 Conception de la base de données ..... 43
3.8 Diagramme de composants ..... 45
3.9 Diagramme de déploiement ..... 46
3.10 Conclusion ..... 46
Chapitre 4 : Réalisation ..... 47
4.1 Introduction ..... 47
4.2 Environnement de travail ..... 47
4.2.1 Environnement matériel ..... 47
4.2.2 Environnement logiciel ..... 47
4.2.3 Gestion de version ..... 48
4.2.4 Sécurité applicative ..... 48
4.3 Interfaces graphiques de l'application ..... 48
4.4 Tests ..... 56
4.4.1 Tests unitaires (JUnit 5 / Mockito) ..... 56
4.4.2 Tests fonctionnels manuels ..... 56
4.4.3 Tests des API avec Postman et Swagger ..... 58
4.4.4 Vérification de bout en bout ..... 58
4.4.5 Évaluation du matching sur un jeu de référence annoté (Gold Set) ..... 58

4.4.6 Validation humaine du score et explicabilité ..... 60
4.5 Difficultés rencontrées et solutions apportées ..... 60
4.5.1 Notification incomplète des managers ..... 60
4.5.2 Distinguer un avis manager d'une décision finale RH ..... 61
4.5.3 Cohérence des transitions de statut ..... 61
4.5.4 Filtrage incomplet des routes de lecture pour les managers ..... 61
4.5.5 Solliciter le manager au bon moment du processus ..... 61
4.5.6 Pertinence du matching Job idéal ..... 62
4.5.7 Traitement distinct des candidatures retirées ..... 62
4.5.8 Entretien téléphonique RH obligatoire avant acceptation ..... 62
4.5.9 Découplage entre planification d'entretien et statut du candidat ..... 62
4.5.10 Fiabiliser l'intégration Google Meet et le format des webhooks n8n ..... 63
4.5.11 Cloisonnement des données dans le chatbot candidat ..... 63
4.6 Protection des données et éthique de l'IA ..... 63
4.7 Bilan des fonctionnalités réalisées ..... 65
4.8 Conclusion ..... 66
Conclusion Générale et Perspectives ..... 67
Annexe A — Captures d'écran complémentaires ..... 69
Bibliographie ..... 76
Glossaire ..... 78

# Liste des figures

Figure 1.1 : Diagramme de Gantt — Planification mensuelle des phases du projet .....17

Figure 2.1 : Diagramme d'activité — Vue d'ensemble du processus de recrutement .....23

Figure 2.2 : Diagramme de cas d'utilisation global .....24

Figure 2.3 : Diagramme de cas d'utilisation — Portail Candidat & Visiteur.....25

Figure 2.4 : Diagramme de cas d'utilisation — Processus de Recrutement & Évaluation.....26

Figure 2.5 : Diagramme de cas d'utilisation — Administration & Paramétrage.....27

Figure 2.6 : Diagramme de séquence système — Soumettre une candidature (boîte noire) .....29

Figure 3.1 : Diagramme des paquetages.....32

Figure 3.2 : Diagramme de classes — vue globale du domaine métier .....33

Figure 3.2a : Diagramme de classes — modèle recrutement .....34

Figure 3.2b : Diagramme de classes — utilisateurs et sécurité .....35

Figure 3.3 : Diagramme de séquence technique — Soumission asynchrone et scoring IA .....36

Figure 3.4 : Diagramme de séquence objet — Planifier un entretien .....37

Figure 3.5 : Diagramme de séquence objet — Décision finale de recrutement.....38

Figure 3.6 : Diagramme de séquence objet — Analyse IA du CV.....39

Figure 3.7 : Diagramme de séquence objet — Routage et notification des managers .....40

Figure 3.8 : Diagramme d'états du cycle de vie d'une candidature.....41

Figure 3.9 : Schéma relationnel de la base de données PostgreSQL .....44

Figure 3.10 : Diagramme de composants de la plateforme JOB4YOU.....45

Figure 3.11 : Diagramme de déploiement de la plateforme JOB4YOU.....46

Figure 4.1 : Page d'accueil publique — recherche d'offres d'emploi .....48

Figure 4.2 : Gestion des offres d'emploi (espace RH) .....49

Figure 4.3 : Gestion des candidats — vue liste avec score IA .....49

Figure 4.4 : Tableau Kanban du pipeline de recrutement.....50

Figure 4.5 : Gestion des utilisateurs et des rôles (espace Administrateur) .....51

Figure 4.6 : Paramétrage du scoring IA par famille de métier .....51

Figure 4.7 : Assistant conversationnel candidat (chatbot).....52

Figure 4.8 : Fonctionnalité « Job idéal » — suggestions d'offres selon le CV .....53

Figure 4.9 : Analyse détaillée d'une candidature par intelligence artificielle.....54

Figure 4.10 : Espace Manager — page « Mon calendrier », entretiens et rappels.....55

Figure A.1 : Page de connexion ..... 69
Figure A.2 : Modification d'un dossier candidat ..... 70
Figure A.3 : Gestion des entretiens ..... 71
Figure A.4 : Gestion des feedbacks ..... 71
Figure A.5 : Journal d'audit ..... 72
Figure A.6 : Soumission d'une candidature avec dépôt de CV ..... 72
Figure A.7 : Espace Manager — candidatures relevant du périmètre du manager ..... 73
Figure A.8 : Console n8n — les trois flux d'automatisation publiés et leurs statistiques ..... 74
Figure A.9 : Flux n8n de l'Agent 1 — webhook de candidature, appel du scoring Cohere puis e-mail au candidat ..... 74
Figure A.10 : E-mail automatique de confirmation de candidature reçu par le candidat ..... 75

# Liste des tableaux

Tableau 1.1 : Macro-planification des phases du projet ..... 16

Tableau 1.2 : Backlog reconstitué des user stories par phase ..... 17

Tableau 2.1 : Matrice des responsabilités par rôle ..... 20

Tableau 2.2 : Description textuelle de deux cas d'utilisation représentatifs ..... 28

Tableau 2.3 : Rôle des agents d'automatisation n8n ..... 30

Tableau 4.1 : Synthèse des tests fonctionnels réalisés ..... 57

Tableau 4.2 : Matrice de confusion du Gold Set selon le seuil de décision ..... 59

Tableau 4.3 : Bilan des fonctionnalités réalisées ..... 65

# Introduction Générale

La fonction Ressources Humaines prend une place de plus en plus stratégique dans les entreprises, portée par la digitalisation accélérée des processus métiers. Le recrutement, première étape du cycle de vie du collaborateur, conditionne directement la qualité du capital humain d'une organisation — et pourtant, dans beaucoup de structures, ce processus reste fragmenté entre des outils hétérogènes : messagerie électronique, tableurs, formulaires papier. Cette dispersion nuit à la traçabilité des décisions, à la collaboration entre intervenants, et à l'expérience vécue par les candidats.

C'est dans ce contexte que Sopra HR Software Tunisie m'a confié le projet JOB4YOU, dans le cadre de mon Projet de Fin d'Études : concevoir et développer une plateforme web centralisée, sécurisée et intuitive, couvrant l'intégralité du processus de recrutement interne, de la publication d'une offre jusqu'à la décision finale d'embauche, en passant par la gestion des candidatures, l'organisation des entretiens et la collecte des retours des managers.

Ce rapport retrace la démarche suivie tout au long du projet, en quatre chapitres. Le premier pose le cadre général — organisme d'accueil, étude de l'existant, problématique et méthodologie adoptée. Le deuxième analyse et spécifie les besoins fonctionnels et non fonctionnels de la plateforme. Le troisième détaille la conception de la solution, sur le plan architectural comme sur celui des modèles de données. Le quatrième décrit la réalisation technique : environnement de travail et principales fonctionnalités développées. Le rapport se conclut par un bilan du travail effectué et les perspectives d'évolution envisagées pour la plateforme.

# Chapitre 1 : Cadre Général du Projet

J'ai rejoint Sopra HR Software Tunisie avec pour mission de digitaliser un processus de recrutement resté, jusque-là, largement manuel. Ce chapitre revient sur le point de départ de ce travail : l'entreprise qui m'a accueilli, les lacunes que j'ai concrètement constatées dans l'existant, la problématique qui en découle, et la manière dont j'ai organisé mon travail sur toute la période de développement de ce Projet de Fin d'Études.

## 1.1 Introduction

Je situe d'abord le cadre organisationnel du projet JOB4YOU, avant de détailler la démarche qui m'a mené à la solution finalement retenue.

## 1.2 Présentation de l'organisme d'accueil

### *1.2.1 Le groupe Sopra Steria*

Sopra Steria est un groupe européen de conseil, de services numériques et d'édition de logiciels, reconnu pour son expertise sectorielle et son approche de bout en bout sur les projets de transformation numérique. Né du rapprochement entre Sopra Group et Steria en 2014 [1], il rassemble plusieurs dizaines de milliers de collaborateurs répartis dans de nombreux pays, et intervient auprès de grands comptes et d'administrations publiques dans des secteurs variés — finance, télécommunications, énergie, secteur public, ou encore ressources humaines.

### *1.2.2 Sopra HR Software*

Sopra HR Software est la filiale du groupe spécialisée dans l'édition de solutions logicielles pour la gestion des ressources humaines : paie, gestion administrative du personnel, gestion des talents et gestion du recrutement. En Tunisie, l'entité constitue un centre de développement et de compétences qui contribue à la conception, au développement et à la maintenance des solutions du groupe, tout en gérant en interne ses propres processus, dont le recrutement.

C'est précisément pour répondre à ce besoin interne de digitalisation que le projet JOB4YOU m'a été proposé.

## 1.3 Étude de l'existant

Avant la mise en place du projet, le processus de recrutement au sein de Sopra HR Software reposait sur une combinaison d'outils disparates et de tâches manuelles :

- La publication des offres d'emploi et leur diffusion s'effectuaient de manière peu centralisée, sans visibilité unifiée sur l'ensemble des postes ouverts.
- La réception des candidatures et des CV se faisait principalement par messagerie électronique, rendant leur classement, leur archivage et leur consultation fastidieux.
- Le suivi de l'avancement d'une candidature (CV reçu, examiné, entretien planifié, décision) n'était pas formalisé dans un outil unique, ce qui entraînait une perte d'information entre les différents intervenants (RH, managers).

- La planification des entretiens et la collecte des feedbacks des managers se faisaient de manière informelle, sans traçabilité ni historique structuré.
- Aucune notification automatique n'informait les candidats de l'état d'avancement de leur candidature, ce qui pouvait nuire à leur expérience et à l'image de l'entreprise.
- L'évaluation initiale des candidatures reposait entièrement sur l'analyse manuelle des CV par les équipes RH, un processus chronophage à mesure que le volume de candidatures augmente.

Il existe sur le marché des logiciels de suivi des candidatures (ATS, Applicant Tracking System) tels que Workday Recruiting, SAP SuccessFactors Recruiting, Greenhouse ou les modules de recrutement des grands SIRH. Ces outils couvrent la diffusion d'offres, le suivi des candidatures et la planification d'entretiens, mais ils sont pensés pour de grands groupes, coûteux à licencier et à paramétrer, et rarement adaptés au périmètre et aux contraintes d'une entité locale. JOB4YOU se positionne comme une alternative interne, sur mesure et maîtrisée : il reprend les fonctions essentielles d'un ATS (pipeline de candidatures, entretiens, feedbacks, décision finale) tout en y ajoutant un scoring IA des CV paramétrable par famille de métier, un matching inverse « Job idéal » et un assistant conversationnel candidat, le tout intégré aux processus et aux rôles propres à Sopra HR Software Tunisie.

## 1.4 Critique de l'existant et problématique

Cette analyse de l'existant fait ressortir plusieurs limites majeures. Sans outil centralisé, l'information se disperse entre plusieurs canaux — e-mails, fichiers, échanges oraux — ce qui complique la collaboration entre responsables RH et managers impliqués dans la décision de recrutement. Le manque de traçabilité des décisions et des statuts rend le suivi de chaque candidature difficile et allonge les délais de traitement. L'absence d'automatisation, enfin, que ce soit dans la présélection des candidats ou dans la communication avec eux, freine l'efficacité et la scalabilité du processus, surtout à mesure que le volume de candidatures grandit.

De ces constats découle la problématique suivante : comment centraliser, sécuriser et automatiser l'ensemble du processus de recrutement — de la publication d'une offre jusqu'à la décision finale — tout en offrant une visibilité en temps réel à tous les acteurs concernés (administrateurs, RH, managers et candidats) ?

## 1.5 Objectifs du projet

Pour répondre à cette problématique, les objectifs suivants ont été fixés pour ce Projet de Fin d'Études :

- Concevoir et développer une plateforme web unique regroupant l'ensemble des étapes du processus de recrutement.
- Mettre en place une gestion des rôles et des accès (Administrateur, RH, Manager, Candidat) garantissant que chaque acteur dispose des fonctionnalités et de la visibilité correspondant à son périmètre.

- Sécuriser les échanges et les accès à la plateforme via un mécanisme d'authentification robuste (JSON Web Token).
- Automatiser la pré-analyse des candidatures grâce à un système de scoring assisté par intelligence artificielle, afin d'accélérer le tri initial des CV tout en laissant la décision finale aux équipes RH et managers.
- Intégrer un système de notifications automatiques par e-mail informant les candidats et les parties prenantes internes à chaque étape clé du processus.
- Fournir aux équipes RH des outils de pilotage (tableau Kanban, tableau de bord, indicateurs) permettant un suivi visuel et en temps réel de l'avancement des recrutements.

## 1.6 Présentation de la solution proposée : JOB4YOU

La solution proposée, baptisée JOB4YOU, est une plateforme web de gestion du processus de recrutement structurée autour de cinq profils d'utilisateurs :

- Le Visiteur, utilisateur non authentifié, qui consulte et recherche les offres d'emploi publiées et obtient des suggestions « Job idéal » en déposant son CV.
- Le Candidat, utilisateur authentifié, qui soumet une candidature accompagnée de son CV, suit l'état d'avancement de son dossier et peut échanger avec l'assistant conversationnel.
- Le Responsable RH, qui publie et gère les offres d'emploi, examine les candidatures reçues, valide les dossiers, planifie les entretiens et contre-signe la décision finale — embauche (avec date de présentation des documents) ou refus motivé —, seule étape qui déclenche l'e-mail de réponse au candidat.
- Le Manager, qui intervient à partir de l'entrée du dossier en évaluation technique (statut TECHNICAL_TEST), mène ou participe aux entretiens, fournit un feedback structuré et émet un avis — favorable (ACCEPTED) ou refus proposé (MANAGER_REJECTED) — sur les dossiers de son périmètre ; cet avis est tracé dans le journal d'audit et transmis au RH, qui le contre-signe.
- L'Administrateur, qui supervise la plateforme dans son ensemble, gère les comptes utilisateurs, les rôles et les paramètres globaux du système, notamment ceux liés au scoring par intelligence artificielle.

Techniquement, la plateforme s'appuie sur une architecture découplée : un back-end Spring Boot expose des API REST sécurisées, un front-end Angular porte l'interface utilisateur, PostgreSQL sert de base de données relationnelle, et des agents d'automatisation n8n prennent en charge l'analyse des

CV par intelligence artificielle (via l'API de Cohere) ainsi que l'envoi des notifications aux différents acteurs. Le chapitre consacré à la conception détaille cette architecture.

## 1.7 Méthodologie de travail adoptée

Les besoins évoluant au fil du stage, et les retours réguliers de mon encadrant entreprise étant essentiels, j'ai adopté une démarche itérative et incrémentale inspirée des principes Agile/Scrum [2]. Le développement s'est organisé en cycles courts (sprints de 2 à 3 semaines), chacun débutant par la sélection et le dimensionnement des user stories prioritaires du Product Backlog, puis enchaînant analyse, conception, implémentation et tests unitaires et fonctionnels. Chaque sprint se clôturait par une revue d'itération (Sprint Review) et une démonstration à mon encadrant entreprise, suivies d'une rétrospective visant à ajuster priorités et spécifications. S'agissant d'un projet mené seul, j'ai endossé les trois rôles Scrum : Product Owner (priorisation du backlog avec l'encadrant), Scrum Master (respect des cérémonies et du rythme des itérations) et équipe de développement (conception, code et tests). Git/GitHub a servi tout du long à assurer la traçabilité du code, faciliter les retours en arrière et documenter l'évolution du projet à travers l'historique des commits. Le rythme de ces itérations et le contenu du backlog associé sont retracés en section 1.8 (figure 1.1 et tableau 1.2).

## 1.8 Planification du projet

Le développement du projet a été réparti en grandes phases sur toute la période de réalisation de ce Projet de Fin d'Études. Afin de concilier la souplesse de la démarche agile en sprints avec les jalons de suivi académiques et industriels, la macro-planification de ces phases est résumée dans le tableau suivant.

*Tableau 1.1 : Macro-planification des phases du projet*

| **Phase** | **Repère mensuel** | **Activités principales** |
|---|---|---|
| Prise en main et cadrage | Février 2026 | Intégration au sein de l'équipe, étude de l'existant, analyse du besoin, rédaction du cahier des charges |
| Analyse et conception | Mars 2026 | Spécification des besoins, modélisation UML [3] (cas d'utilisation, séquence, classes), conception de la base de données |
| Développement — Socle applicatif | Avril – Mai 2026 | Mise en place de l'architecture back-end/front-end, authentification JWT, gestion des rôles, gestion des offres et des candidatures |
| Développement — Fonctionnalités avancées | Juin – Juillet 2026 | Gestion des entretiens et des feedbacks, scoring IA, agents n8n, notifications e-mail, tableau Kanban |

| Phase | Repère mensuel | Activités principales |
|---|---|---|
| Tests, corrections et finalisation | Août 2026 | Tests fonctionnels, corrections de bugs, améliorations UX et sécurité, rédaction du rapport |

La figure 1.1 reprend cette macro-planification sur un calendrier mensuel, d'après les repères mensuels du tableau 1.1.

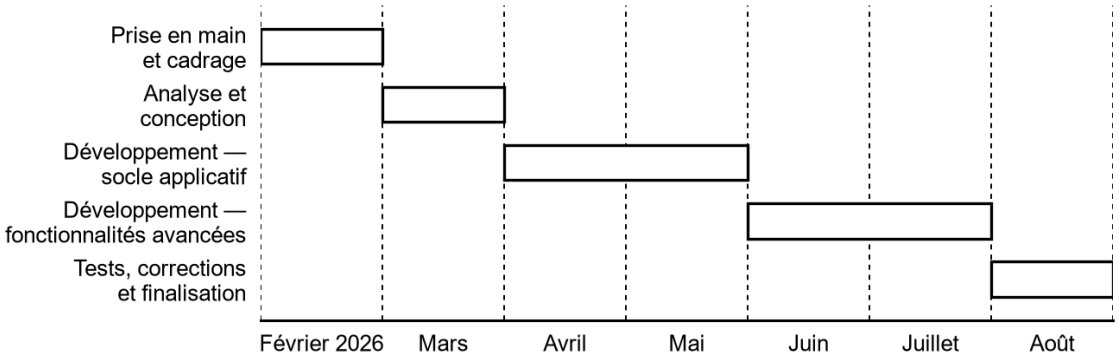



*Figure 1.1 : Diagramme de Gantt — Planification mensuelle des phases du projet*

Le tableau 1.2 reconstitue les user stories des principales fonctionnalités livrées, par acteur et par phase du tableau 1.1.

*Tableau 1.2 : Backlog reconstitué des user stories par phase*

| User story | Acteur bénéficiaire | Phase de réalisation |
|---|---|---|
| Créer un compte et postuler à une offre (dépôt du CV) | Candidat | Développement — socle applicatif |
| Suivre l'avancement de sa candidature à chaque étape | Candidat | Développement — socle applicatif |
| Publier et gérer les offres d'emploi du périmètre RH | Responsable RH | Développement — socle applicatif |
| Faire présélectionner les CV par un score IA paramétrable | Responsable RH | Développement — fonctionnalités avancées |
| Planifier les entretiens, émettre un avis et le contre-signer | RH et Manager | Développement — fonctionnalités avancées |
| Consulter le tableau Kanban et le journal d'audit | Responsable RH | Développement — fonctionnalités avancées |
| Disposer d'un calendrier d'entretiens avec rappels | Manager | Développement — fonctionnalités avancées |
| Bénéficier des suggestions « Job idéal » et du chatbot | Candidat | Développement — fonctionnalités avancées |
| Évaluer, corriger et expliquer le scoring IA | Responsable RH | Tests, corrections et finalisation |

*Backlog reconstitué a posteriori à partir de l'historique Git, non tenu en temps réel durant le projet.*

## 1.9 Conclusion

Les limites du processus existant chez Sopra HR Software sont maintenant posées, tout comme la méthodologie suivie pendant le stage. Reste à entrer dans le vif du sujet : quels besoins précis JOB4YOU devait-il couvrir ? C'est l'objet du chapitre suivant.

# Chapitre 2 : Analyse et Spécification des Besoins

Le contexte posé, restait à traduire la problématique en besoins concrets : qui utilise la plateforme, pour faire quoi, et avec quelles contraintes ? C'est ce que je détaille dans ce chapitre, acteur par acteur, avant de justifier les choix techniques préliminaires qui ont guidé l'architecture retenue.

## 2.1 Introduction

Recueillir les besoins n'a rien eu d'un exercice ponctuel : cela s'est étalé sur les premières semaines du stage, au fil des échanges avec mon encadrant et de l'observation directe du processus RH. Ce qui suit est le résultat de cette phase, et sert de base directe à la conception présentée au chapitre suivant.

## 2.2 Étude fonctionnelle

### *2.2.1 Identification des acteurs du système*

Cinq acteurs interviennent sur la plateforme JOB4YOU, chacun avec ses propres privilèges et fonctionnalités :

- Le Visiteur : utilisateur anonyme, non authentifié, qui consulte les offres d'emploi publiées et peut déposer son CV dans la fonctionnalité « Job idéal » pour obtenir des suggestions ; il doit créer un compte et s'authentifier pour soumettre une candidature.
- Le Candidat : utilisateur authentifié disposant d'un compte (rôle ROLE_USER), qui soumet ses candidatures, en suit l'avancement, échange avec l'assistant conversationnel et peut retirer un dossier tant qu'il n'est pas clos.
- Le Responsable RH : acteur central du processus, il publie les offres, examine et valide les candidatures, planifie les entretiens et contre-signe la décision finale (embauche ou refus), seul moment où le candidat est notifié.
- Le Manager : intervient à partir de l'entrée du dossier en évaluation technique (statut TECHNICAL_TEST) ; il consulte les candidatures relevant de son périmètre, mène ou participe aux entretiens, fournit un feedback structuré et émet un avis (favorable ou refus proposé) sur les dossiers de son périmètre, qui reste à contre-signer par le RH.
- L'Administrateur : acteur transverse chargé de la gestion des comptes utilisateurs, des rôles, du paramétrage du scoring IA par famille de métier et de la supervision globale de la plateforme (journal d'audit).

Le tableau ci-dessous précise, action par action, les rôles habilités — une répartition qui correspond exactement aux règles d'autorisation implémentées côté back-end.

*Tableau 2.1 : Matrice des responsabilités par rôle*

| Action | RH | Manager | Administrateur | Candidat |
|---|---|---|---|---|
| Publier / gérer une offre d'emploi | Oui | Non | Oui | Non |
| Valider un CV (passage CV_REVIEWED) | Oui | Non | Oui | Non |
| Planifier un entretien | Oui | Oui (son périmètre) | Oui | Non |
| Saisir un feedback | Oui | Oui | Oui | Non |
| Valider / transmettre un feedback au candidat | Oui | Non | Oui | Non |
| Avis sur la candidature (favorable / refus proposé) | Oui | Oui (son périmètre) | Oui | Non |
| Décision finale contre-signée (HIRED / REJECTED) | Oui | Non | Oui | Non |
| Gérer les utilisateurs et les rôles | Non | Non | Oui | Non |
| Paramétrer le scoring IA | Non | Non | Oui | Non |
| Consulter le journal d'audit | Oui | Non | Oui | Non |

J'ai volontairement donné au RH et à l'Administrateur la main sur l'ensemble du cycle : ils peuvent enregistrer un avis comme le ferait le manager — pour corriger ou débloquer un dossier en son absence — et sont seuls à pouvoir contre-signer la décision finale. Un compte purement manager ne peut écrire ni l'embauche (HIRED) ni un statut du pipeline : la plateforme le refuse par une erreur 403. Qui a réellement pris chaque décision reste visible dans le journal d'audit.

### *2.2.2 Identification des besoins fonctionnels*

Les échanges avec mon encadrant entreprise et l'étude de l'existant ont fait émerger les besoins fonctionnels suivants :

- Authentification et gestion des sessions sécurisée par jeton JWT, avec gestion différenciée des accès selon le rôle de l'utilisateur.

- Gestion des offres d'emploi : création, modification, publication, clôture et expiration automatique des offres dont la date limite est dépassée.
- Gestion des candidatures : soumission en ligne avec dépôt de CV, consultation, filtrage et suivi de l'état d'avancement de chaque dossier.
- Analyse automatique des candidatures par intelligence artificielle : extraction du contenu du CV et calcul d'un score de pertinence (technique, communication, adéquation séniorité) avec possibilité de correction manuelle par le RH.
- Gestion des entretiens : planification, changement de statut (planifié, en cours, terminé, annulé, reprogrammé), envoi d'invitations et de rappels automatiques.
- Gestion des feedbacks : saisie d'un retour structuré par le manager après entretien, validation par le RH et notification au candidat.
- Décision à deux niveaux : le manager émet un avis (favorable ou refus proposé) que le RH ou l'Administrateur contre-signe pour produire la décision finale — embauche avec date de présentation des documents, ou refus motivé —, avec garde-fou empêchant toute décision sur un dossier non validé par le RH et toute validation finale par un compte purement manager.
- Notifications automatiques par e-mail à chaque étape clé du processus (confirmation de candidature, invitation à un entretien, décision finale) ; l'e-mail de décision n'est adressé au candidat qu'après le contreseing du RH, l'avis du manager notifiant le RH.
- Tableau de pilotage visuel de type Kanban permettant de visualiser en un coup d'œil la répartition des candidatures par statut.
- Gestion des utilisateurs, des rôles et paramétrage des profils de pondération du score IA par famille de métier.
- Matching inverse (« Job idéal ») : un visiteur peut déposer son CV sans postuler à une offre précise ; le système le compare aux offres actives et lui suggère les plus pertinentes avec un score de compatibilité.

### *2.2.3 Identification des besoins non fonctionnels*

Au-delà des fonctionnalités métier, la plateforme doit aussi répondre aux exigences non fonctionnelles suivantes :

- Sécurité : authentification robuste par jeton JWT signé d'une durée de vie de 24 heures, hachage des mots de passe avec BCrypt, contrôle d'accès basé sur les rôles (RBAC) appliqué à la fois côté front-end et côté back-end, jeton de réinitialisation de mot de passe valable 15 minutes et limitation du nombre de tentatives de connexion.

- Performance : temps de réponse cible inférieur à 500 ms pour les opérations de lecture courantes et inférieur à 2 s pour les opérations d'écriture, chargement asynchrone des traitements longs (scoring IA, envoi d'e-mails) afin de ne pas bloquer l'utilisateur, et taille maximale des CV déposés fixée à 10 Mo côté serveur (formats PDF, DOC, DOCX), le formulaire de candidature refusant déjà les fichiers de plus de 5 Mo. Cette différence de plafond est volontaire : la validation du formulaire est un confort offert à l'utilisateur, la limite serveur est la véritable barrière de sécurité, seule opposée à un client qui contournerait l'interface.
- Ergonomie et responsive design : interface claire, intuitive et utilisable aussi bien sur poste de travail que sur tablette.
- Fiabilité : garde-fous métier empêchant les transitions de statut incohérentes (ex. acceptation avant l'entretien téléphonique RH, validation finale sans avis manager en attente, embauche sans date de présentation des documents, entretien planifié sur un dossier clos).
- Maintenabilité et évolutivité : architecture modulaire en couches, séparation claire entre front-end, back-end et automatisation, facilitant l'ajout de nouvelles fonctionnalités.
- Traçabilité : journalisation des actions sensibles (changements de statut, corrections de score) dans un journal d'audit.

### *2.2.4 Diagramme d'activité — Vue d'ensemble du processus de recrutement*

Avant de détailler les fonctionnalités attendues, ce diagramme d'activité pose le flux métier global : il décrit l'enchaînement des activités du processus de recrutement, de la publication de l'offre jusqu'à la décision finale, ainsi que les acteurs qui les portent, indépendamment de toute considération technique.

Le diagramme d'activité de la figure 2.1 (page suivante) rend explicite un point que le texte seul rendait moins visible : l'entretien téléphonique RH est un passage obligé avant toute évaluation technique (section 4.5.8), et le routage vers le ou les managers concernés (ManagerRoutingService, section 3.5.4) n'intervient qu'à ce moment-là, pas dès la validation du CV — conformément à la correction décrite en section 4.5.5. Le rejet automatique par l'IA n'y constitue pas une fin de parcours distincte : il rejoint, comme la contre-signature du RH, l'étape commune de notification et de journalisation, et reste réversible par un override RH justifié (transition pointillée de la figure 3.8).

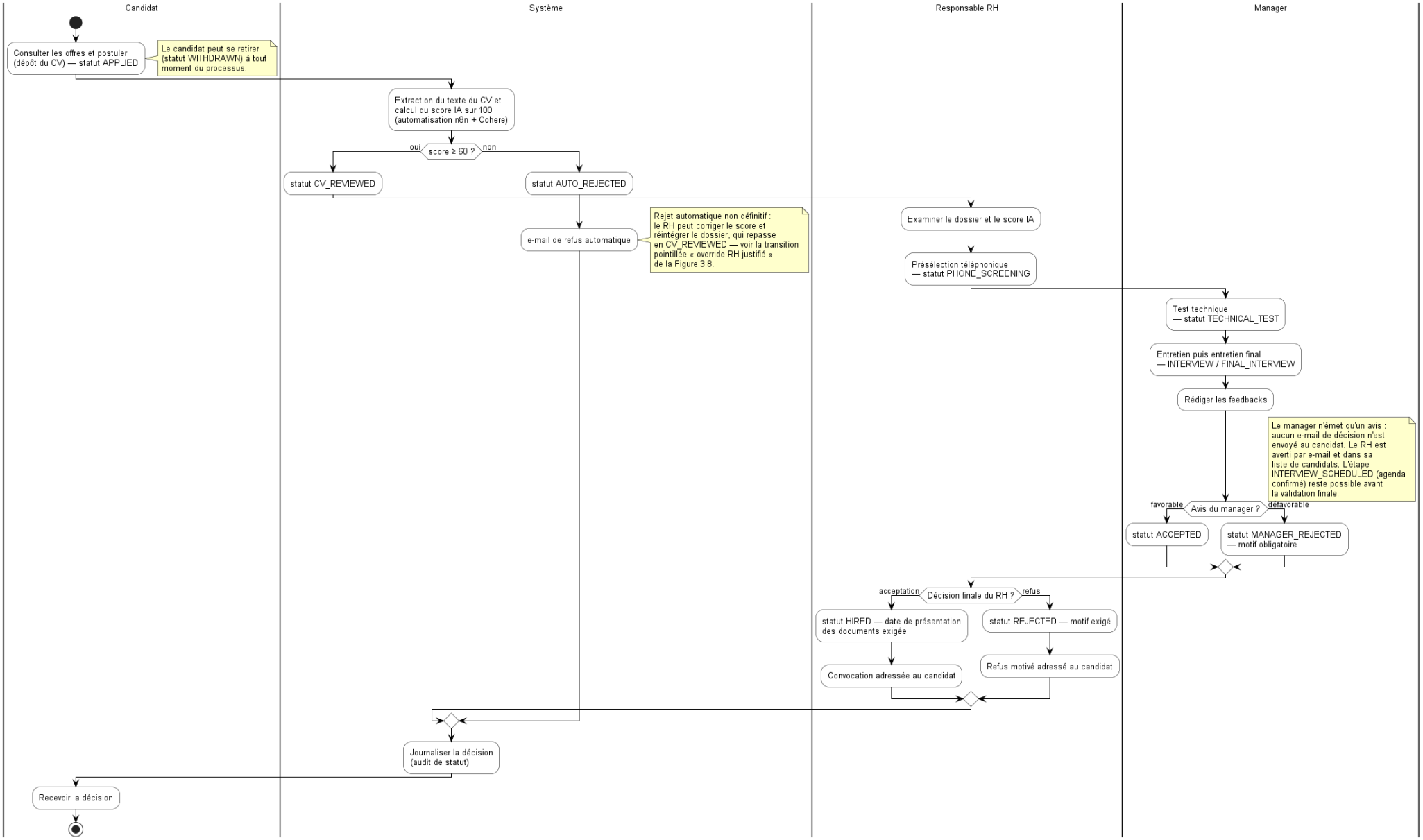



*Figure 2.1 : Diagramme d'activité — Vue d'ensemble du processus de recrutement*

### 2.2.5 Diagrammes de cas d'utilisation

Afin d'éviter l'effet « toile d'araignée », le périmètre fonctionnel est d'abord présenté en vue macro ci-dessous (figure 2.2), puis décliné en trois sous-diagrammes par module (figures 2.3 à 2.5). L'authentification par jeton JWT, bien qu'obligatoire pour les espaces sécurisés, est représentée par un cas commun « S'authentifier » (la relation « include » n'est illustrée que sur quelques cas représentatifs) ; les relations d'inclusion pointent du cas principal vers le sous-cas requis, conformément à la norme UML [3], et le Candidat généralise le Visiteur.

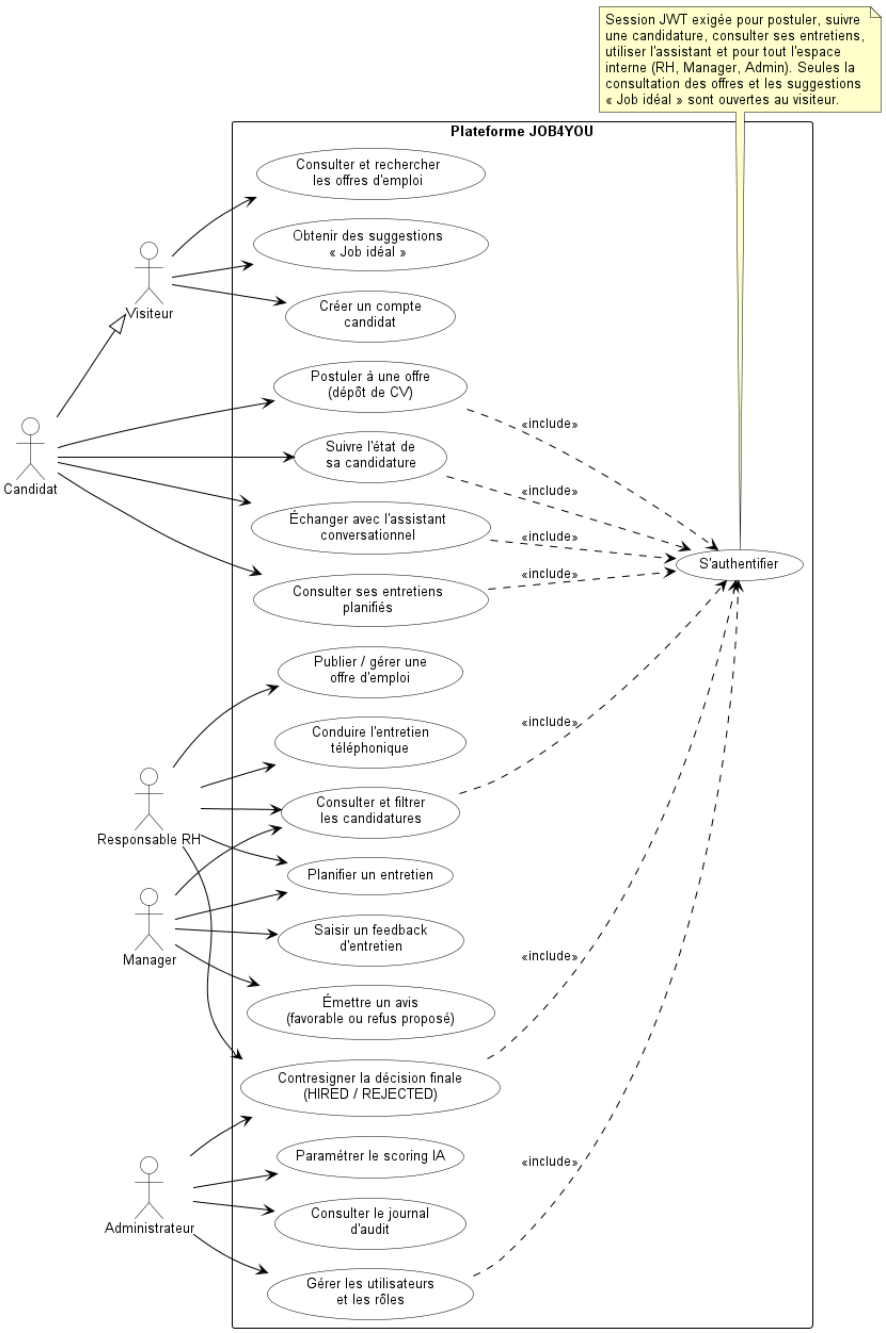



*Figure 2.2 : Diagramme de cas d'utilisation global*

*Seuls 7 cas d'usage représentatifs sont reliés à « S'authentifier » pour la lisibilité ; l'authentification s'applique à tout l'espace interne.*

L’authentification constitue une fonctionnalité transversale de la plateforme : le cas d’utilisation « S’authentifier » est commun au Candidat, au Responsable RH, au Manager et à l’Administrateur, qui doivent s’identifier avant d’accéder à leur espace sécurisé. Pour ne pas surcharger la figure 2.2, la relation « include » n’y est représentée que pour sept cas représentatifs (postuler à une offre, suivre l’état de sa candidature, consulter ses entretiens planifiés, échanger avec l’assistant conversationnel, consulter et filtrer les candidatures, contre-signer la décision finale, gérer les utilisateurs et les rôles) ; elle s’applique à toutes les fonctionnalités des espaces sécurisés. Seules la consultation et la recherche des offres ainsi que les suggestions « Job idéal » demeurent accessibles au visiteur non authentifié, l’assistant conversationnel restant réservé au candidat authentifié. Les autorisations sont ensuite contrôlées côté serveur selon le rôle et le périmètre métier de l’utilisateur. Les diagrammes ne représentent que les cas principaux : la matrice du tableau 2.1 fait foi pour le détail des droits (par exemple, le RH peut aussi saisir un feedback et consulter le journal d’audit).

Le premier zoom couvre le portail public et candidat : consultation des offres, matching « Job idéal », dépôt de candidature et suivi en libre-service (figure 2.3).

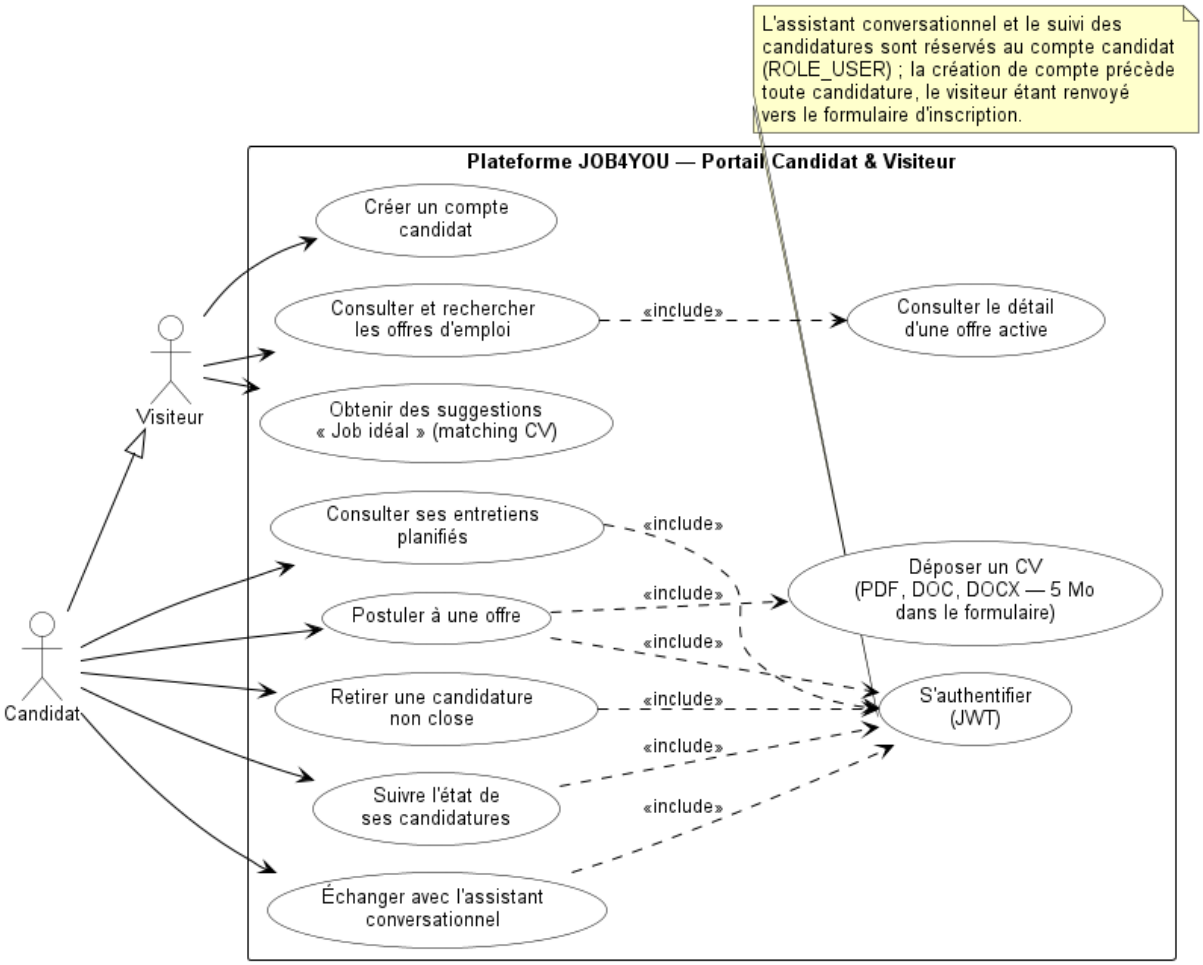



*Figure 2.3 : Diagramme de cas d’utilisation — Portail Candidat & Visiteur*

Le deuxième zoom décrit le cœur du processus côté RH et manager : publication des offres, filtrage, entretiens, feedbacks, avis du manager puis contre-signature RH de la décision finale (figure 2.4). L’envoi des notifications par e-mail y est modélisé comme un cas inclus, déclenché par la planification ou par la décision, et non comme un cas autonome.

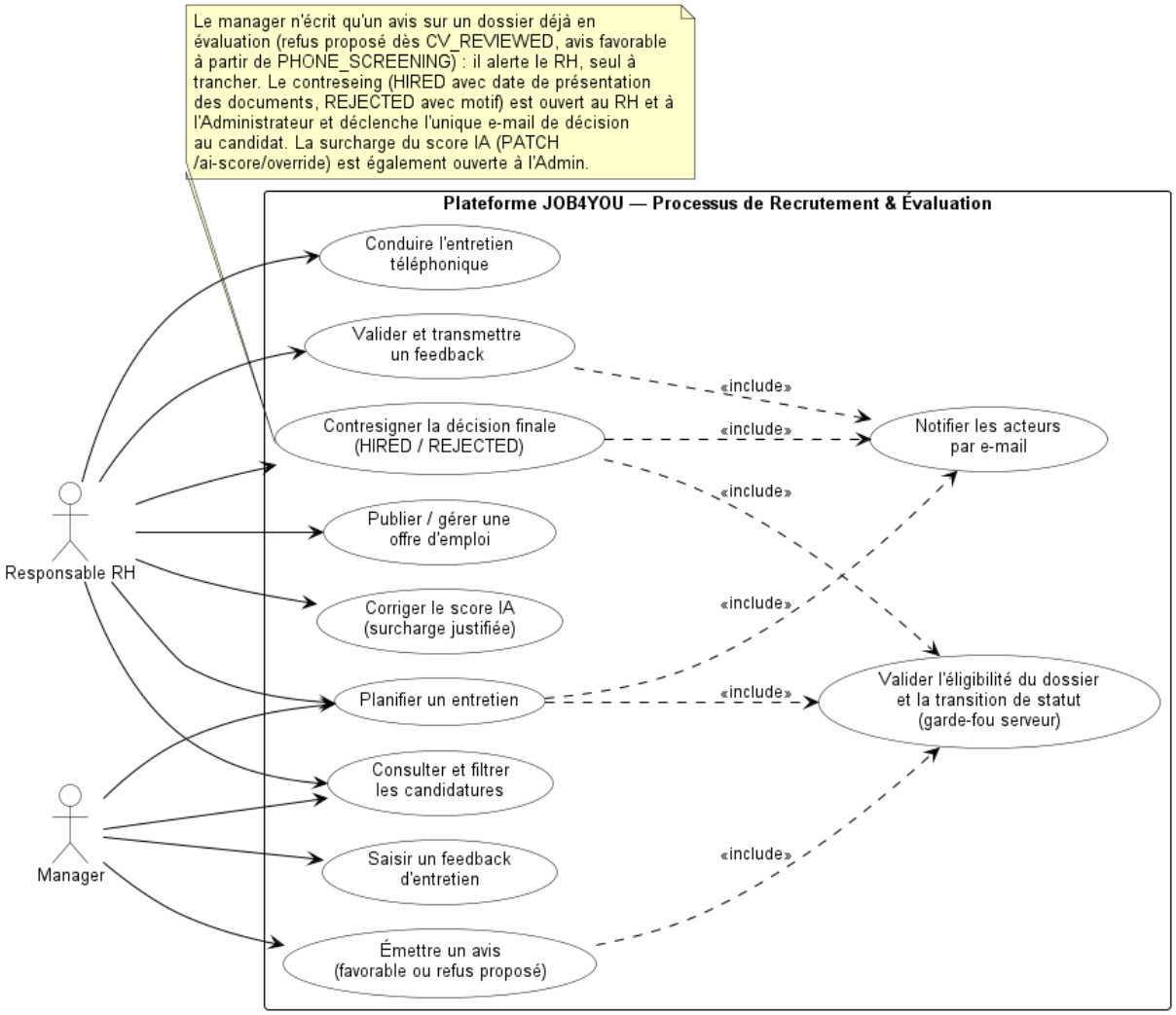



*Figure 2.4 : Diagramme de cas d'utilisation — Processus de Recrutement & Évaluation*

Le troisième zoom isole le périmètre d'administration : comptes et rôles, référentiel des départements et des familles de métier, rattachements correspondants, pondérations du scoring IA et journal d'audit (figure 2.5).

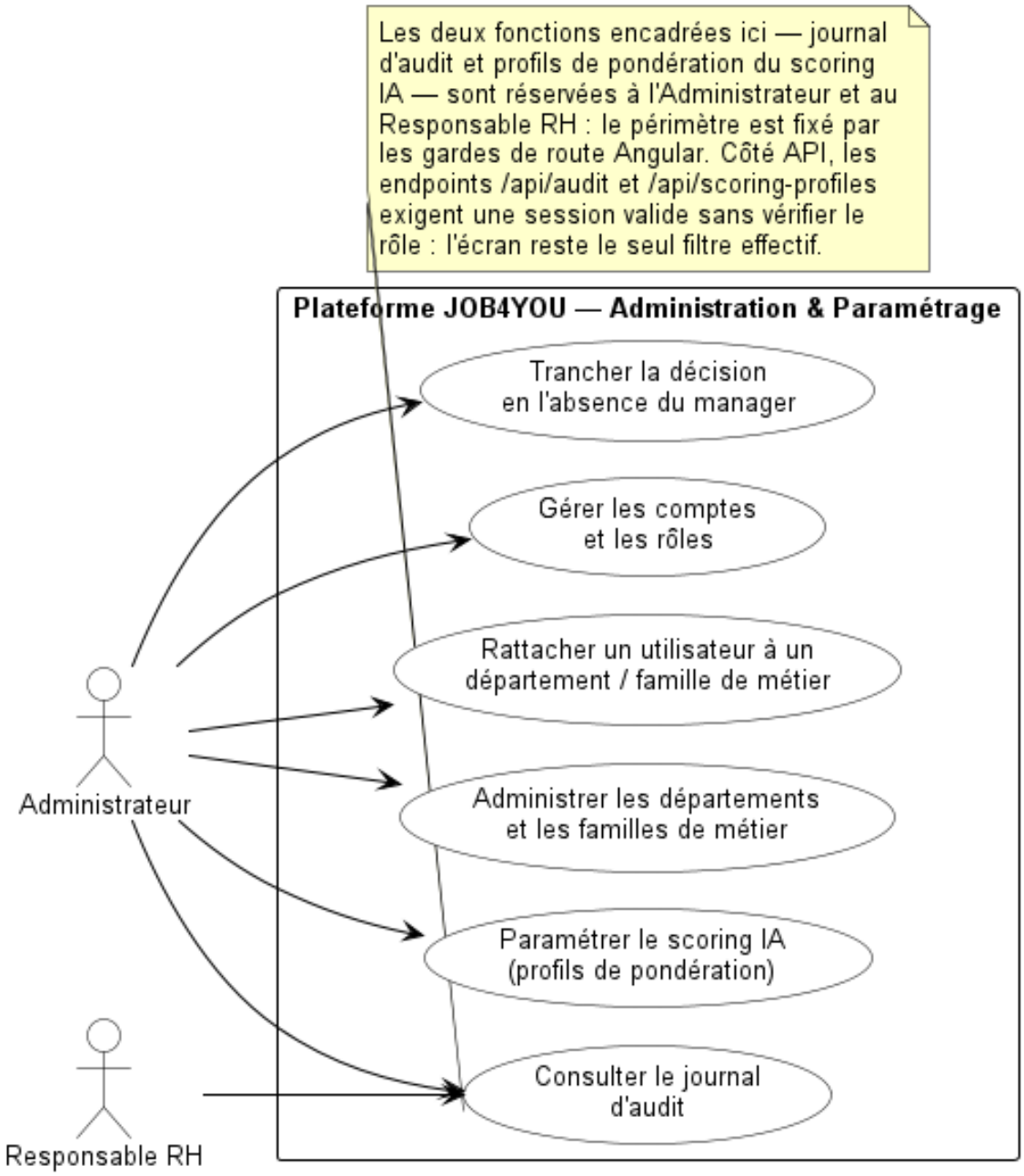



*Figure 2.5 : Diagramme de cas d'utilisation — Administration & Paramétrage*

### *2.2.6 Description textuelle de quelques cas d'utilisation*

Le tableau qui suit détaille le scénario nominal de deux cas d'utilisation représentatifs du processus métier.

*Tableau 2.2 : Description textuelle de deux cas d'utilisation représentatifs*

| **Élément** | **Cas d'utilisation « Postuler à une offre »** | **Cas d'utilisation « Avis manager, puis décision finale »** |
|---|---|---|
| Acteur principal | Candidat | Manager (avis), puis Responsable RH (contreseing) |
| Pré-condition | L'offre est publiée (statut ACTIVE) | Le dossier est en cours d'évaluation : un refus proposé (MANAGER_REJECTED) est admis dès la présélection du CV (statut CV_REVIEWED ou ultérieur) ; un avis favorable (ACCEPTED) exige d'être passé par l'entretien téléphonique RH (PHONE_SCREENING, TECHNICAL_TEST, INTERVIEW ou FINAL_INTERVIEW). |
| Scénario nominal | Le candidat consulte l'offre, remplit le formulaire de candidature et dépose son CV. Le système enregistre la candidature, envoie un e-mail de confirmation et déclenche l'analyse IA du CV. | Le manager consulte le dossier du candidat, ses scores et ses éventuels feedbacks, puis émet un avis — favorable, ou refus proposé accompagné d'un motif. Le RH, averti par e-mail et dans sa liste de candidats, contre-signe cet avis : embauche (HIRED) avec date de présentation des documents, ou refus final (REJECTED) motivé. |
| Post-condition | La candidature est créée avec le statut APPLIED, puis mise à jour automatiquement selon le score IA. | L'avis est enregistré (ACCEPTED ou MANAGER_REJECTED) et notifié au RH ; la décision finale (HIRED ou REJECTED) n'est acquise qu'après son contreseing, seul moment où le candidat reçoit un e-mail. |
| Exception | Candidature déjà déposée pour cette offre : le système rejette la demande. | Dossier à un statut qui n'autorise pas la décision demandée — par exemple un avis favorable avant l'entretien téléphonique RH — ou refus manager sans motif : le système retourne une erreur 400. Un compte purement manager qui appelle le point d'entrée de validation finale reçoit une erreur 403. |

### 2.2.7 Diagramme de séquence système (boîte noire)

Ce diagramme adopte une approche « boîte noire » stricte : il modélise uniquement les échanges entre l'acteur Candidat et le système JOB4YOU considéré comme un tout. Il met en évidence la requête de soumission du formulaire et le retour synchrone de confirmation (HTTP 201 Created), masquant à ce stade l'infrastructure interne et les traitements asynchrones, qui relèvent de la conception détaillée (section 3.4).

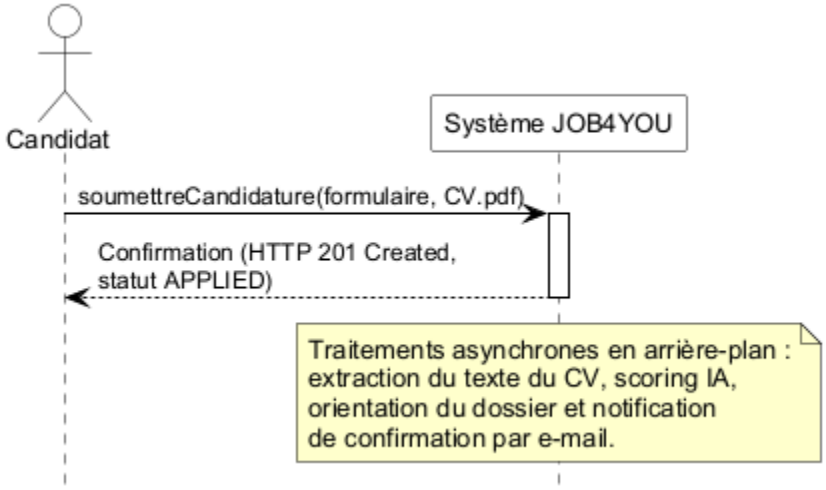



*Figure 2.6 : Diagramme de séquence système — Soumettre une candidature (boîte noire)*

## 2.3 Étude technique

Les choix techniques répondent aux besoins identifiés. Spring Boot [4], [5] a été retenu pour la robustesse de son écosystème (Spring Security, Spring Data JPA) et la rapidité de mise en place d'API REST sécurisées. Angular [6], [7] impose une structure (composants, services, routage, gardes de route) adaptée à une application multi-rôles maintenue dans la durée. PostgreSQL [8] offre un modèle relationnel solide et des garanties transactionnelles pour des données fortement liées (offres, candidatures, entretiens, feedbacks). Enfin, n8n [9] permet de construire et de faire évoluer les workflows d'automatisation (planification, notifications, déclenchement du scoring) sans alourdir le code du back-end, tout en isolant les dépendances vis-à-vis des services externes.

### 2.3.1 Architecture physique cible

La plateforme cible une architecture distribuée par couches : un front-end Angular servi au navigateur, un back-end Spring Boot exposant des API REST sécurisées, une base PostgreSQL (SGBD) et une couche d'automatisation n8n chargée de la planification des entretiens (Google Calendar [10]) et des notifications (SMTP) ; le back-end appelle directement l'API de Cohere [11] (scoring IA) et l'API OpenRouter [12] (assistant conversationnel). Cette organisation physique est détaillée au chapitre 3 par les diagrammes de composants (figure 3.10) et de déploiement (figure 3.11) ; le tableau ci-dessous précise le rôle de chaque agent d'automatisation.

*Tableau 2.3 : Rôle des agents d'automatisation n8n*

| Agent | Déclencheur | Rôle et traitement |
|---|---|---|
| Agent 1 — « CV Parser + Score IA + Email Candidat » (scoring IA) | Webhook déclenché à la création d'une candidature (statut APPLIED) | Extrait le texte du CV puis rappelle le back-end (POST /api/candidates/{id}/ai-score/recompute). C'est le back-end, via son client CohereClient, qui interroge l'API de Cohere, calcule le score et le persiste ; le statut bascule ensuite vers CV_REVIEWED ou AUTO_REJECTED selon le seuil. |
| Agent 2 — « Planification entretien + Email participants » (Google Meet et convocations) | Webhook déclenché à la planification d'un entretien | Crée réellement l'événement Google Calendar, génère le lien Google Meet correspondant, ajoute les participants et envoie les invitations et rappels au candidat et à l'intervieweur. |
| Agent 3 — « RH vers Manager (dossier validé) » (routage des managers et décisions automatiques) | Webhooks déclenchés à l'entrée en évaluation technique (TECHNICAL_TEST), au rejet automatique et au retrait d'une candidature | Résout les managers destinataires (ManagerRoutingService) et envoie les notifications correspondantes aux managers concernés, sans dupliquer l'e-mail candidat ; les e-mails de décision — avis transmis au RH, convocation ou refus final adressé au candidat — sont émis par le back-end via NotificationService. |

### *2.3.2 Côté client — Angular*

J'ai développé le front-end sous forme de SPA (Single Page Application) avec Angular, en architecture par composants autonomes (« standalone components »), organisée en pages publiques (accueil, offres d'emploi, candidature en ligne), pages d'administration (candidats, entretiens, feedbacks, offres, utilisateurs, tableau Kanban, tableau de bord) et services partagés qui assurent la communication avec l'API REST du back-end. Bootstrap prend en charge la mise en forme des interfaces.

### *2.3.3 Côté serveur — Spring Boot*

Côté serveur, j'ai développé le back-end en Java avec Spring Boot 3, selon une architecture en couches classique : contrôleurs REST, services métier, repositories Spring Data JPA et entités JPA (mapping objet-relationnel, ORM, assuré par Hibernate) mappées sur la base PostgreSQL. Cette organisation garde les responsabilités bien séparées et facilite les évolutions futures.

### *2.3.4 La sécurité — Spring Security et JWT*

L'accès à la plateforme repose sur Spring Security [13], couplé à une authentification par jeton JWT (JSON Web Token). À la connexion, le serveur génère un jeton signé portant l'identité et les rôles de l'utilisateur ; ce jeton est ensuite transmis dans l'en-tête de chaque requête, ce qui authentifie l'utilisateur sans nécessiter d'état de session côté serveur. Chaque point d'entrée de l'API est protégé via l'annotation @PreAuthorize, garantissant que seuls les rôles autorisés (Administrateur, RH, Manager) accèdent à une ressource donnée.

## 2.4 Conclusion

Acteurs, besoins et architecture technique étant désormais clairement identifiés, une question s'est imposée : comment traduire tout cela en un modèle concret, capable de guider le développement ? C'est ce que j'aborde dans le chapitre suivant, consacré à la conception.

# Chapitre 3 : Conception

Passer d'une liste de besoins à un système cohérent implique des choix : comment organiser le code, comment modéliser les données, comment éviter qu'un candidat se retrouve dans un état incohérent. Ce chapitre documente les principales décisions de conception derrière JOB4YOU — paquetages, diagramme de classes, scénarios d'interaction et machine à états du cycle de vie d'une candidature.

## 3.1 Introduction

Chaque modèle présenté ici sert directement l'implémentation décrite au chapitre 4 : chaque diagramme correspond à une décision de conception effectivement prise pendant le développement.

## 3.2 Diagramme des paquetages

La figure suivante illustre l'organisation du back-end en paquetages (configuration, sécurité, contrôleurs, services, repositories, entités). L'organisation du front-end et les interactions avec les services externes sont décrites ensuite, puis détaillées aux sections 3.8 et 3.9.

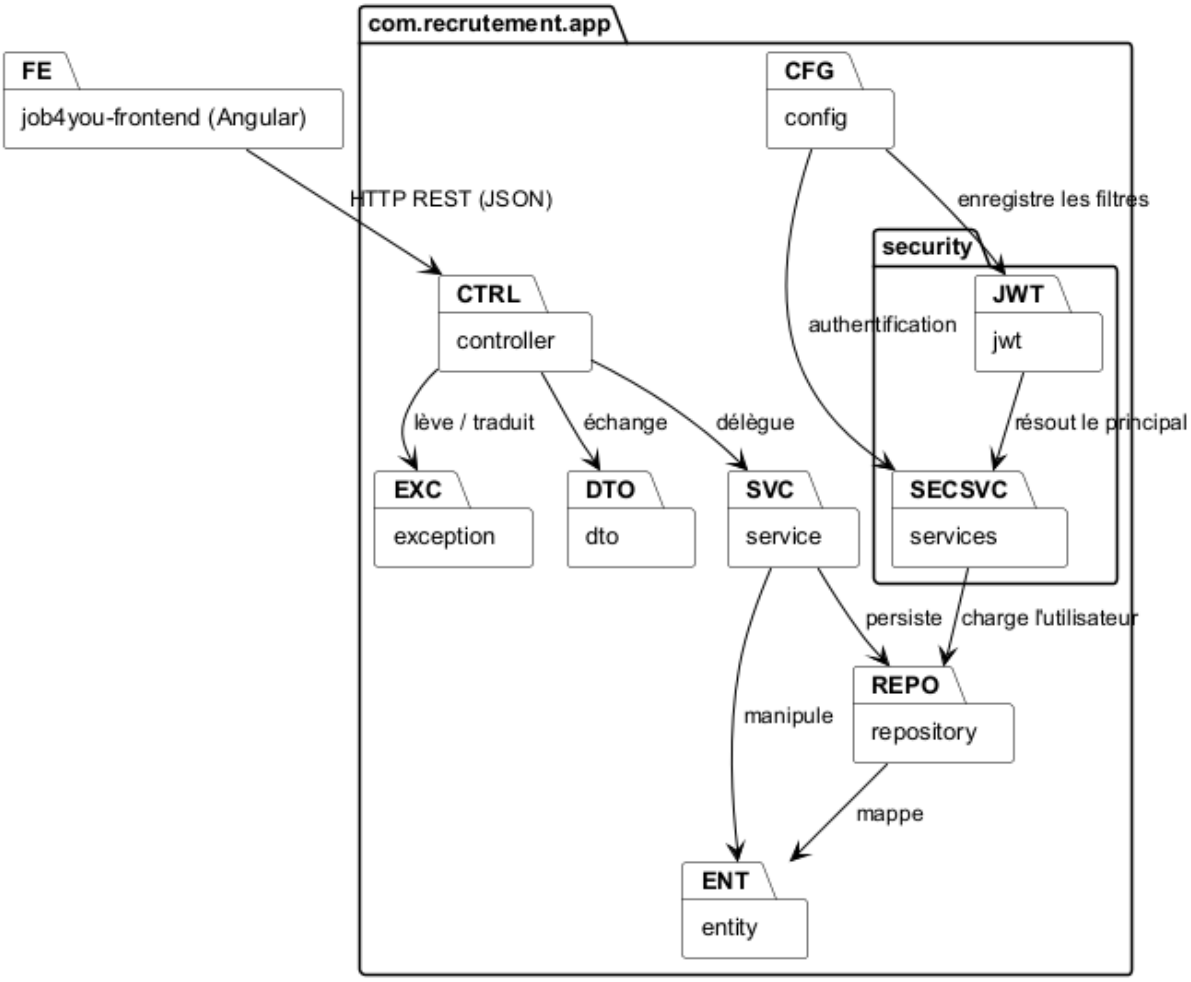



*Figure 3.1 : Diagramme des paquetages*

Côté back-end, j'ai suivi un découpage classique en couches : la couche controller expose les API REST, la couche service porte la logique métier (règles de gestion, garde-fous, orchestration des appels vers n8n), la couche repository assure la persistance via Spring Data JPA, et les entités JPA

modélisent le domaine métier. Côté front-end, chaque page consomme des services Angular dédiés à une ressource métier (candidats, entretiens, feedbacks, offres), qui communiquent eux-mêmes avec le back-end via des appels HTTP authentifiés.

## 3.3 Diagrammes de classes

La figure 3.2 ci-dessous donne une vue d'ensemble du modèle de données : les neuf entités métier principales de la plateforme, leurs attributs caractéristiques et les associations réellement mappées par JPA. Les deux figures suivantes en proposent des zooms thématiques — le recrutement (figure 3.2a), puis les utilisateurs, la sécurité et l'audit (figure 3.2b) — afin d'en conserver une lecture confortable.

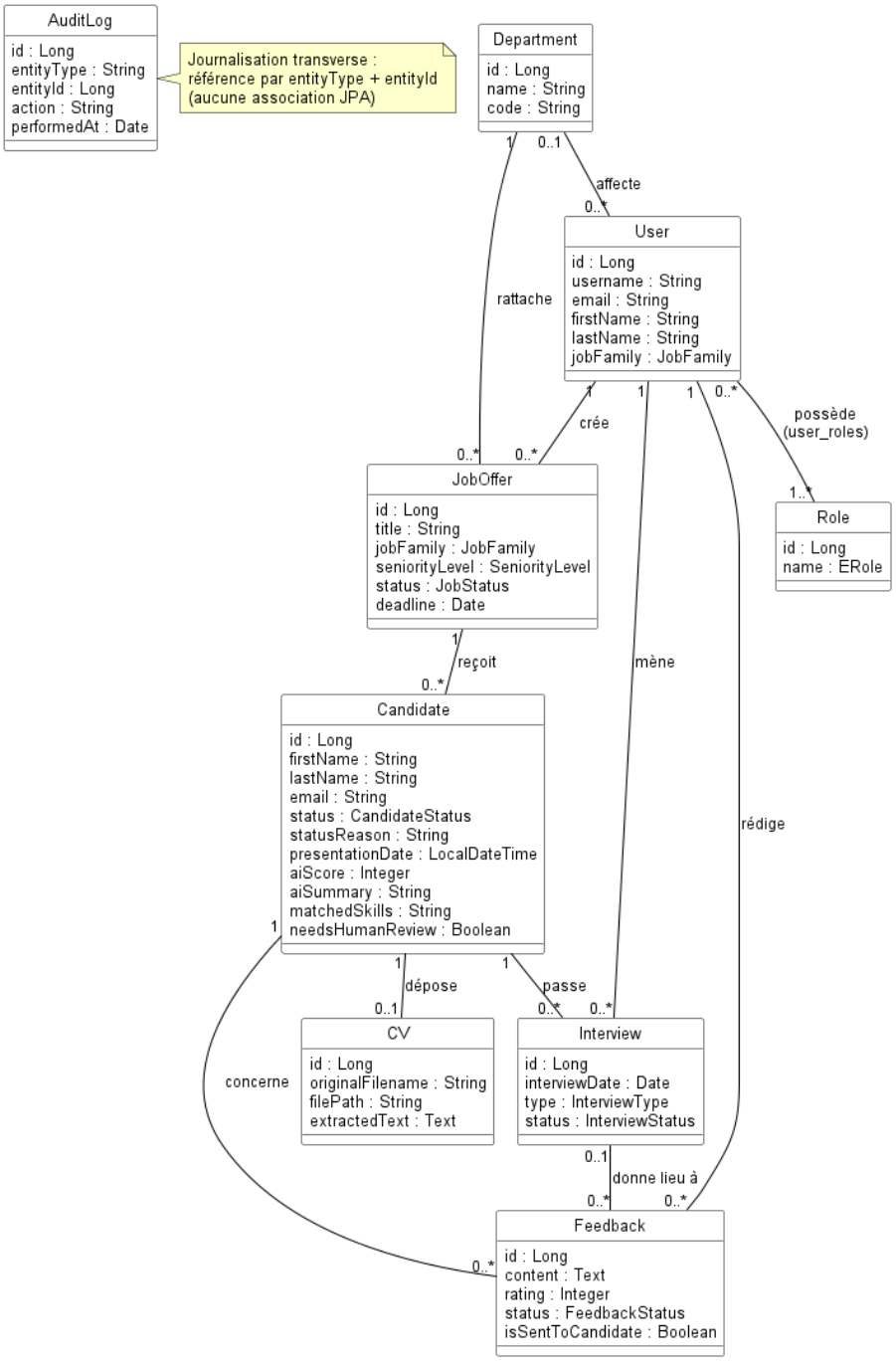



*Figure 3.2 : Diagramme de classes — vue globale du domaine métier*

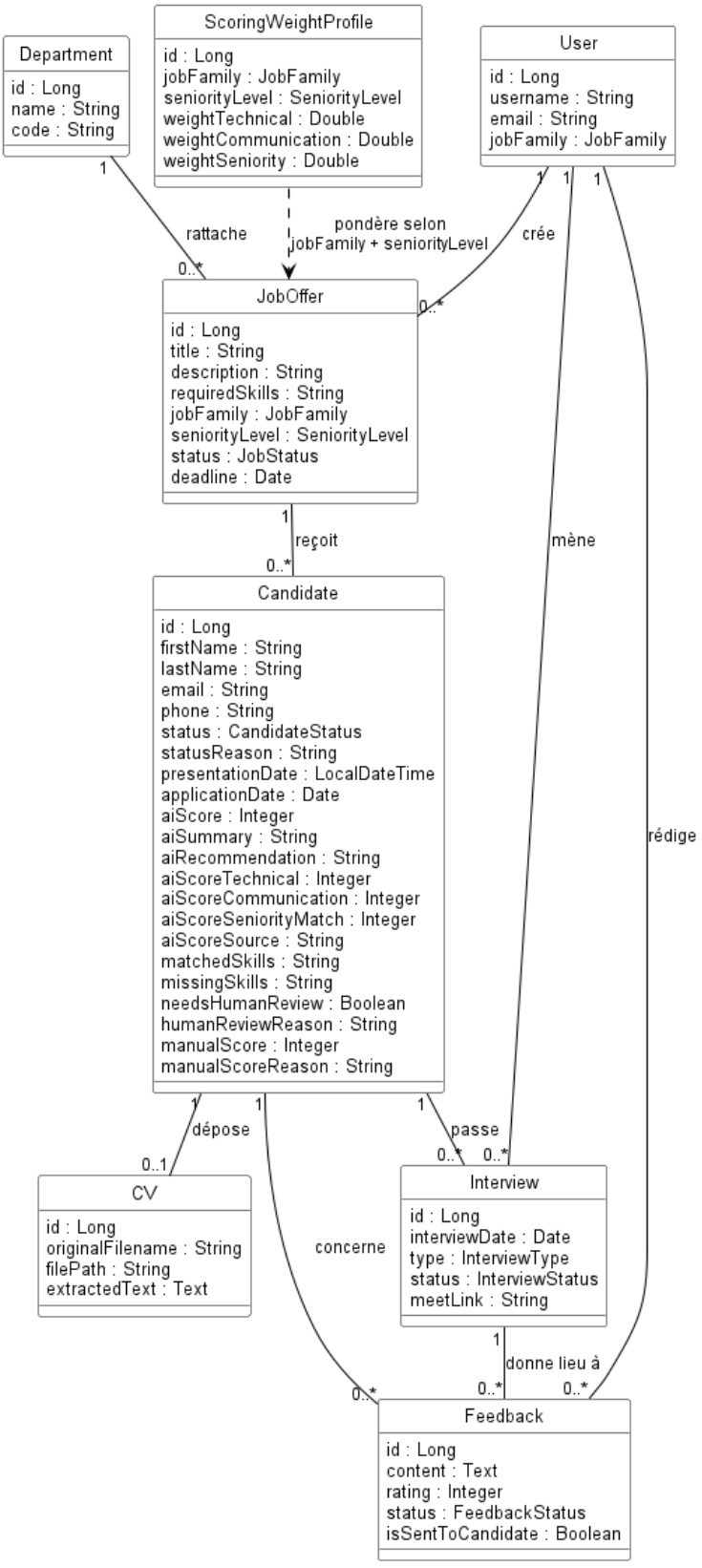



*Figure 3.2a : Diagramme de classes — modèle recrutement*

Ce modèle fait ressortir plusieurs relations structurantes : un candidat dépose au plus un CV et peut passer par plusieurs entretiens, chacun pouvant donner lieu à un ou plusieurs feedbacks rédigés par un utilisateur (manager ou RH). Une offre d'emploi, rattachée à un département, reçoit plusieurs candidatures. Quant à la classe User, elle centralise l'identité des utilisateurs disposant d'un compte (RH, managers, administrateurs, ainsi que les candidats authentifiés, avec le rôle ROLE_USER) et porte, via la classe Role, un système de rôles multiples pour une gestion fine des permissions. Les associations de User avec les offres (création), les entretiens (intervieweur) et les feedbacks (auteur) sont matérialisées dans le schéma relationnel (figure 3.9) ; le rattachement d'un compte à ses candidatures s'appuie sur l'adresse e-mail du compte, pré-remplie et non modifiable lors du dépôt d'une candidature.

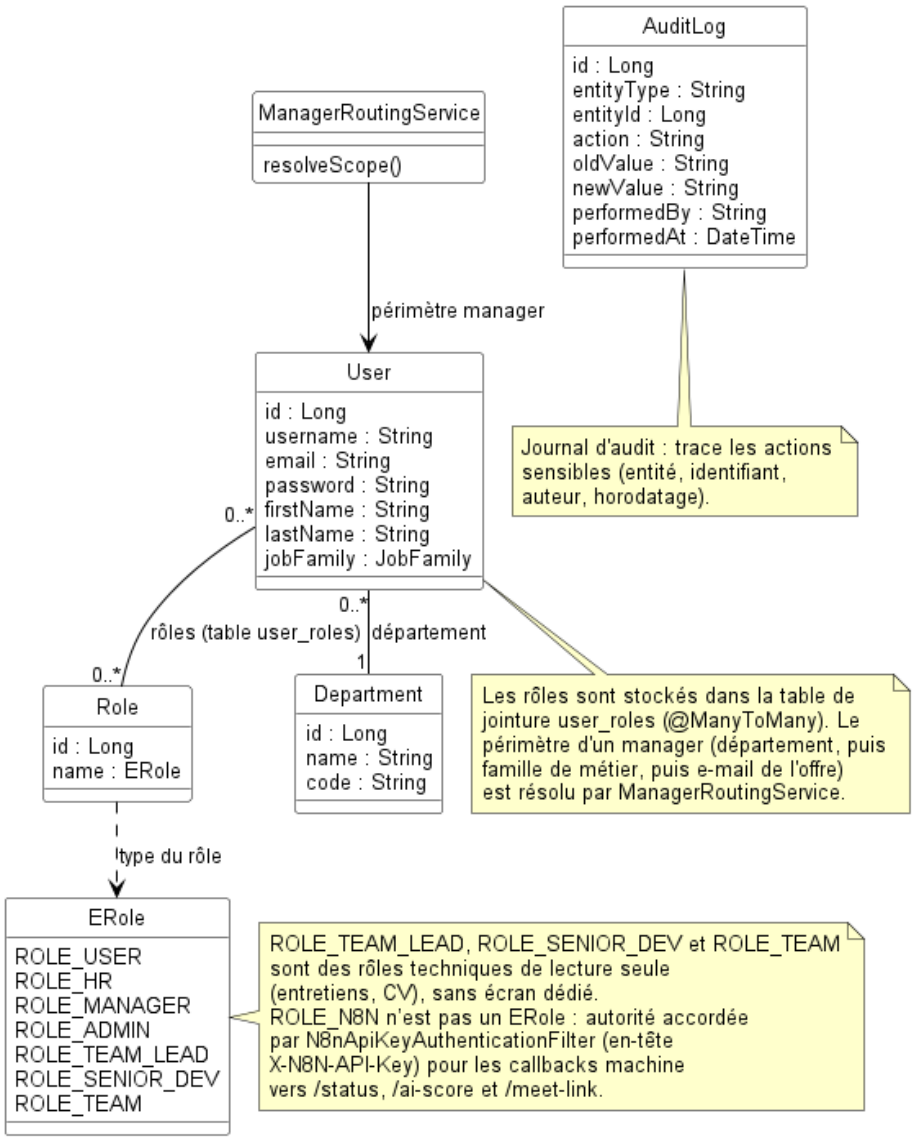



*Figure 3.2b : Diagramme de classes — utilisateurs et sécurité*

La vue sécurité isole les entités d'authentification et d'audit : chaque utilisateur porte un ou plusieurs rôles (ERole) et un rattachement à un département, ManagerRoutingService résout le périmètre de lecture d'un manager, et AuditLog journalise les actions sensibles.

## 3.4 Séquence technique — Soumission asynchrone et scoring IA

Lors de la soumission d'une candidature, l'application orchestre plusieurs composants internes et externes de manière asynchrone afin de ne pas bloquer l'utilisateur. Comme l'illustre la figure 3.3, le front-end Angular transmet la charge utile au back-end Spring Boot, qui persiste immédiatement la candidature (statut APPLIED) ; un mécanisme de file d'attente (outbox) notifie ensuite le serveur d'automatisation n8n, chargé de l'extraction du texte, avant qu'un callback sécurisé ne permette au back-end de solliciter l'API Cohere et d'enregistrer le score de pertinence.

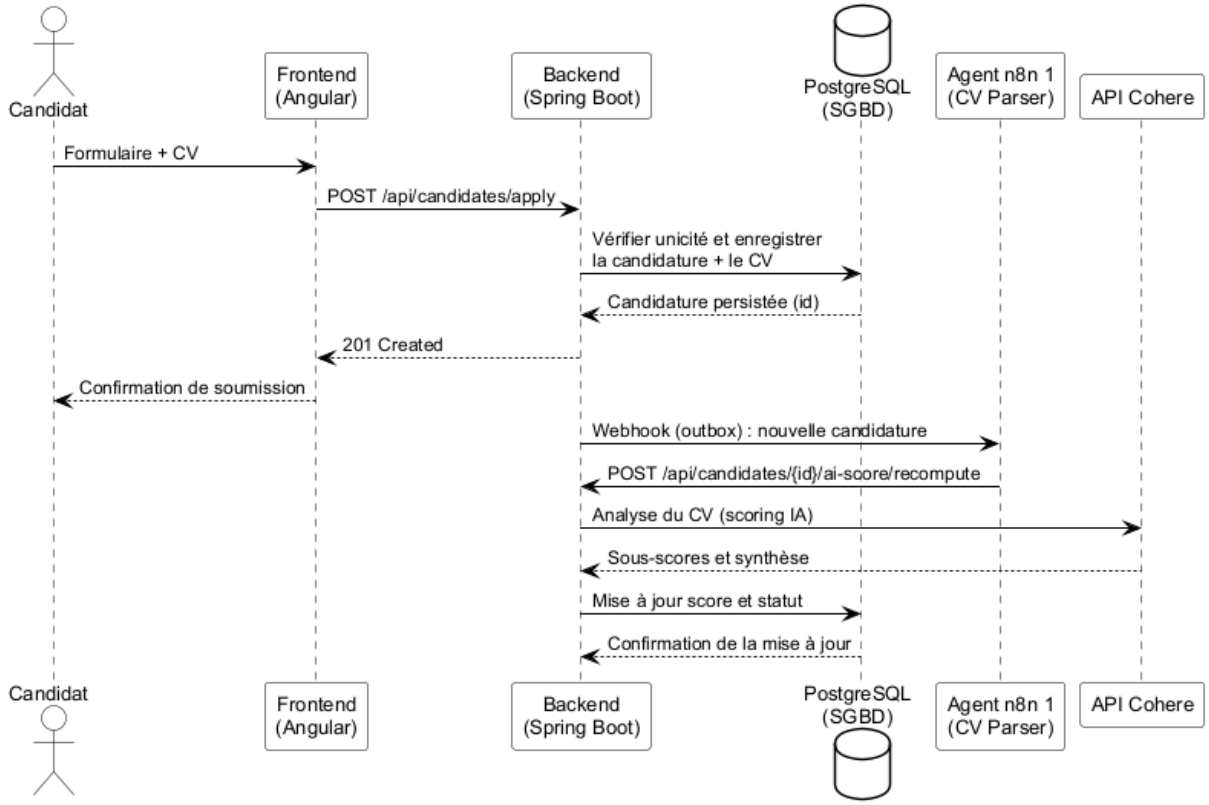



*Figure 3.3 : Diagramme de séquence technique — Soumission asynchrone et scoring IA*

## 3.5 Diagrammes de séquence objet

Les quatre diagrammes qui suivent détaillent, au niveau objet, quatre étapes clés du processus de recrutement : la planification d'un entretien, la décision finale de recrutement, l'analyse IA du CV et le routage des managers.

### 3.5.1 Planifier un entretien

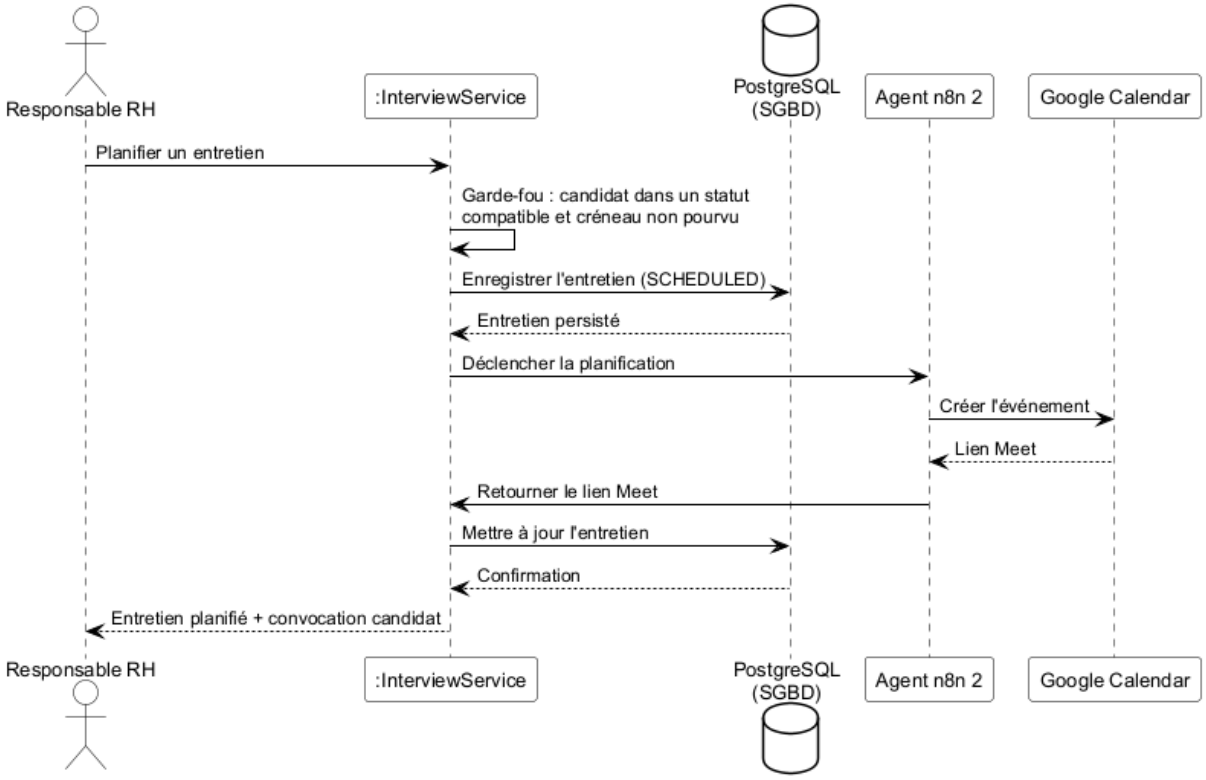



*Figure 3.4 : Diagramme de séquence objet — Planifier un entretien*

Ce diagramme met en avant un garde-fou métier important : avant toute planification, le service s'assure que le dossier du candidat n'est pas dans un état clos négativement (rejeté ou retiré), pour éviter de planifier un entretien sur une candidature déjà écartée.

### 3.5.2 Décision finale de recrutement

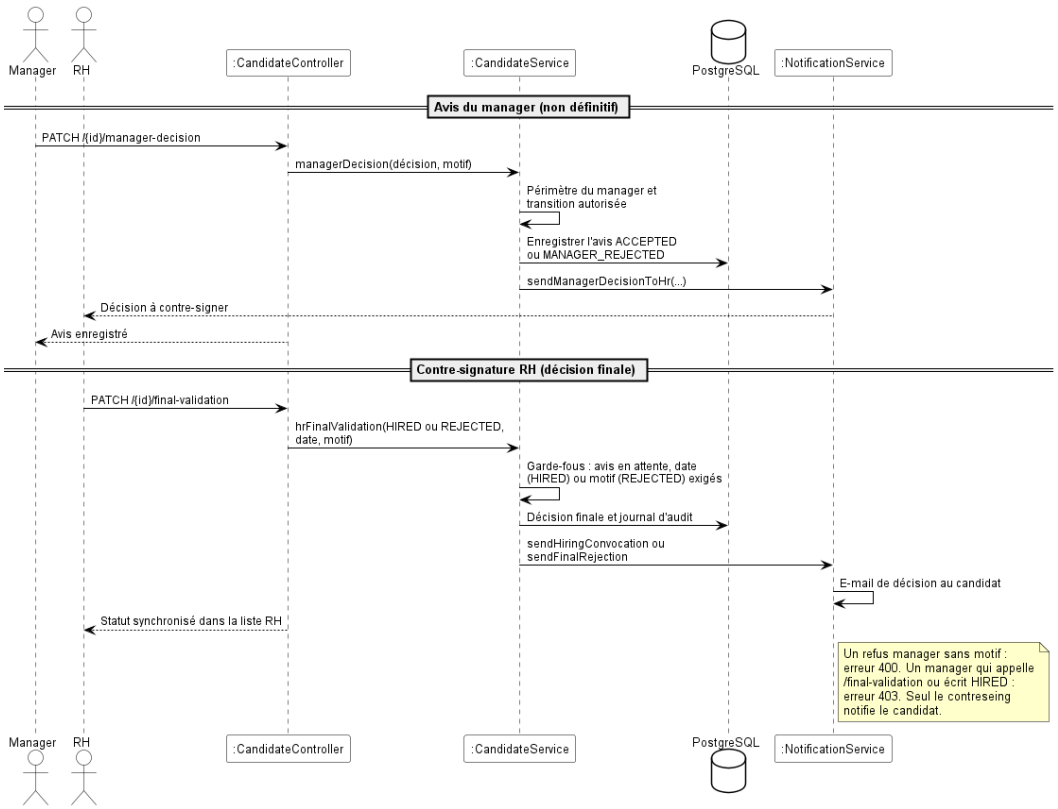



*Figure 3.5 : Diagramme de séquence objet — Décision finale de recrutement*

Ce second diagramme illustre le garde-fou central du processus, dont la décision est désormais répartie sur deux niveaux. Le manager consulte le dossier et émet un avis : un refus proposé (MANAGER_REJECTED) est admis dès que le dossier a été validé par le RH (statut CV_REVIEWED ou ultérieur), à condition d'être motivé, tandis qu'un avis favorable (ACCEPTED) exige d'être passé par l'entretien téléphonique RH (PHONE_SCREENING) — l'entrée en évaluation technique n'étant pas un prérequis. L'avis est journalisé et notifie le RH par e-mail et dans sa liste de candidats, sans aucun envoi au candidat. Le RH ou l'Administrateur contre-signe ensuite via PATCH /api/candidates/{id}/final-validation : HIRED exige la date de présentation des documents et déclenche la convocation, REJECTED exige un motif et déclenche le refus motivé. Un compte purement manager qui appelle ce point d'entrée, écrit HIRED ou un statut du pipeline reçoit une erreur 403. L'ensemble empêche qu'une candidature soit acceptée sans ce premier filtre humain (section 4.5.8), et qu'un manager clôture seul le processus.

### *3.5.3 Analyse IA du CV (extraction, scoring via n8n et Cohere)*

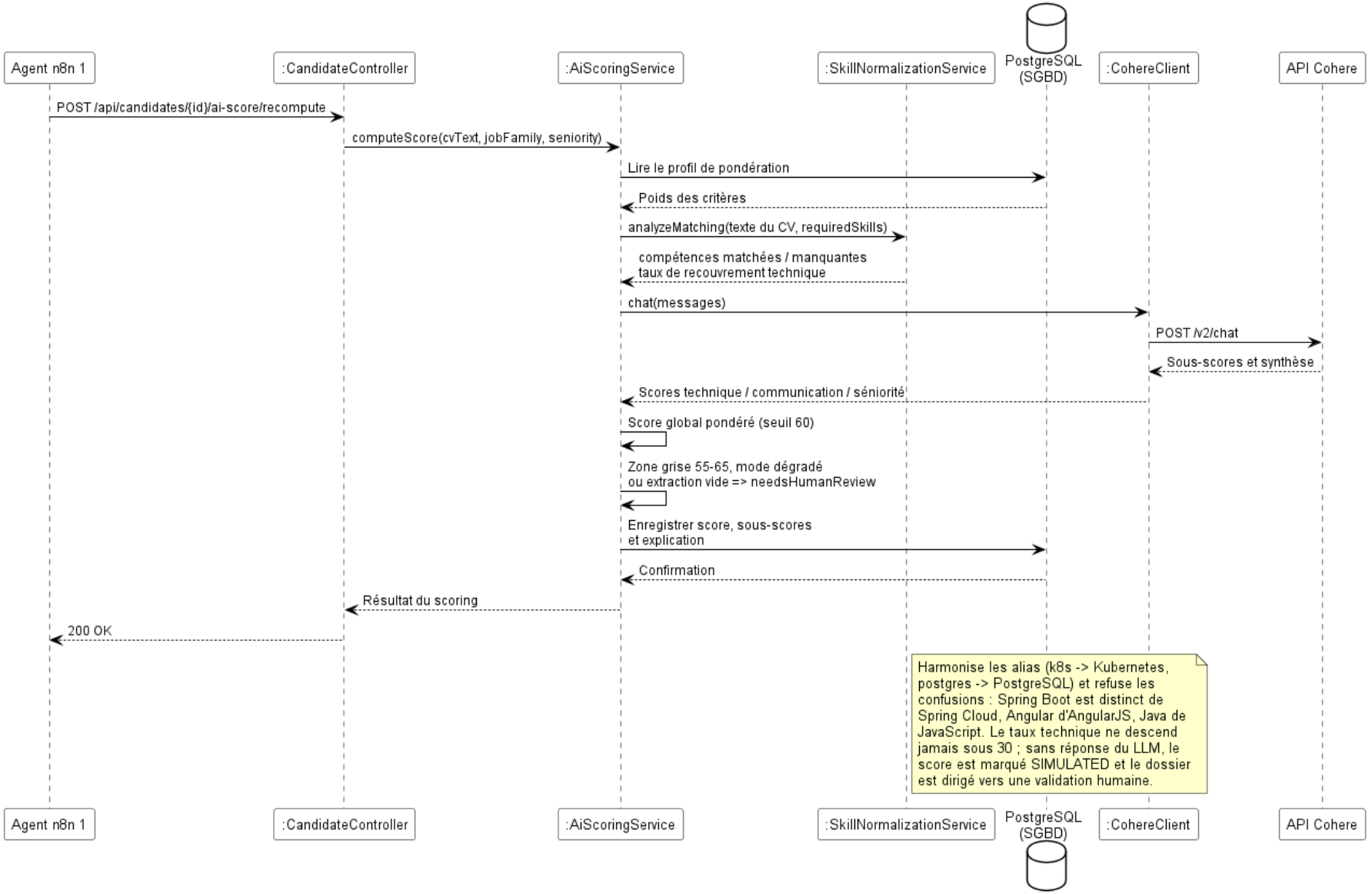



*Figure 3.6 : Diagramme de séquence objet — Analyse IA du CV*

Ce diagramme détaille, au niveau objet, le cheminement du scoring : dès l'enregistrement de la candidature (statut APPLIED), un webhook n8n déclenche l'agent « CV Parser », qui extrait le texte du CV. Cet agent ne contacte pas directement Cohere : il rappelle le back-end via POST /api/candidates/{id}/ai-score/recompute, et c'est le back-end — au travers de son client CohereClient — qui interroge l'API de Cohere, calcule le score et le persiste. Le statut bascule ensuite automatiquement vers CV_REVIEWED ou AUTO_REJECTED selon le seuil configuré, sans intervention humaine à ce stade — sauf si le dossier est signalé pour arbitrage, comme le décrit la suite du diagramme. Deux étapes complémentaires figurent sur le diagramme. Avant l'appel au modèle, SkillNormalizationService harmonise les compétences extraites (k8s devient Kubernetes, postgres devient PostgreSQL) et écarte les termes inconnus, afin que la pondération par profil de scoring ne porte que sur des libellés fiables. Après le calcul, le service positionne le drapeau needsHumanReview lorsque le texte extrait est trop court, lorsque le score tombe dans la zone grise comprise entre 55 et 65, ou lorsque l'API de Cohere est indisponible ; dans ce dernier cas le score est marqué SIMULATED et le dossier est dirigé vers un arbitrage humain plutôt que vers une décision automatique (section 4.4.6).

### *3.5.4 Passage en évaluation technique — routage et notification des managers*

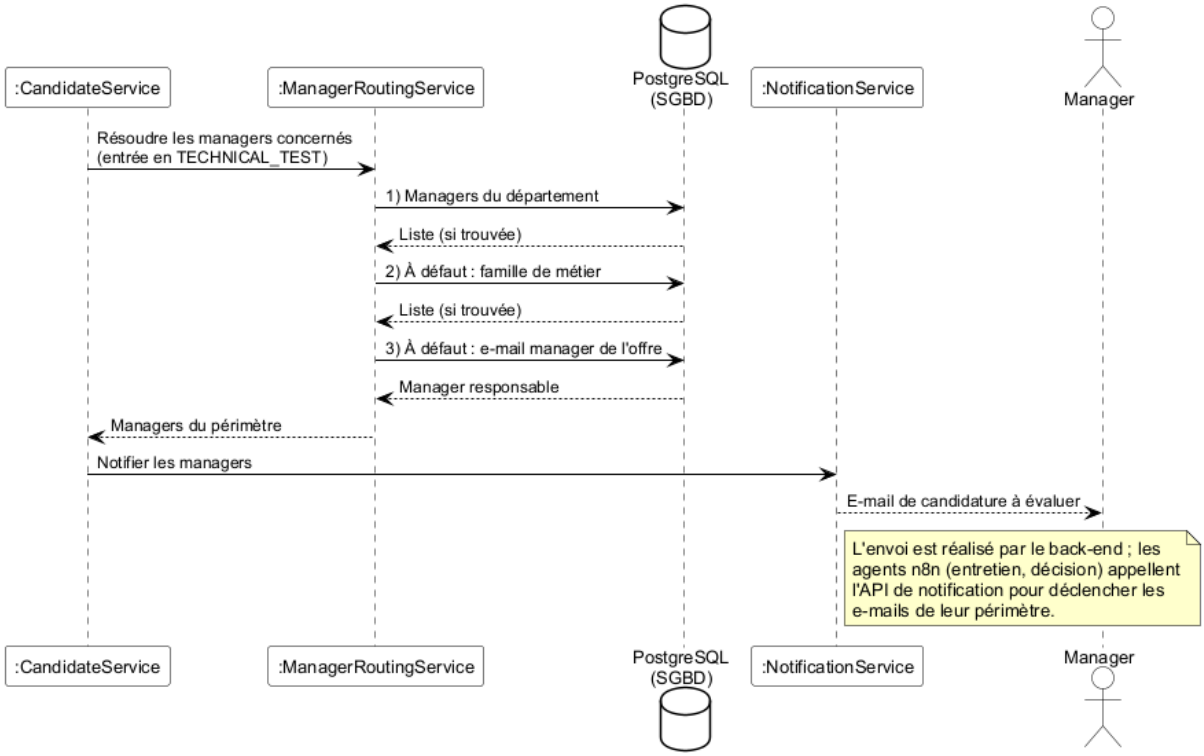



*Figure 3.7 : Diagramme de séquence objet — Routage et notification des managers*

Ce diagramme illustre le mécanisme de routage automatique décrit en section 4.5.5 : lorsqu'un dossier passe en évaluation technique, ManagerRoutingService résout les managers destinataires par priorité — d'abord par département de l'offre, à défaut par famille de métier, et en dernier recours via l'e-mail manager saisi manuellement sur l'offre. Chaque manager résolu reçoit ensuite une notification via l'agent n8n dédié.

## 3.6 Diagramme d'états — Cycle de vie d'une candidature

Tout au long du processus de recrutement, le statut d'un candidat évolue selon une machine à états précise, représentée ci-dessous. Cette modélisation garantit la cohérence des transitions et sert de référence pour l'implémentation des contrôles métier côté back-end.

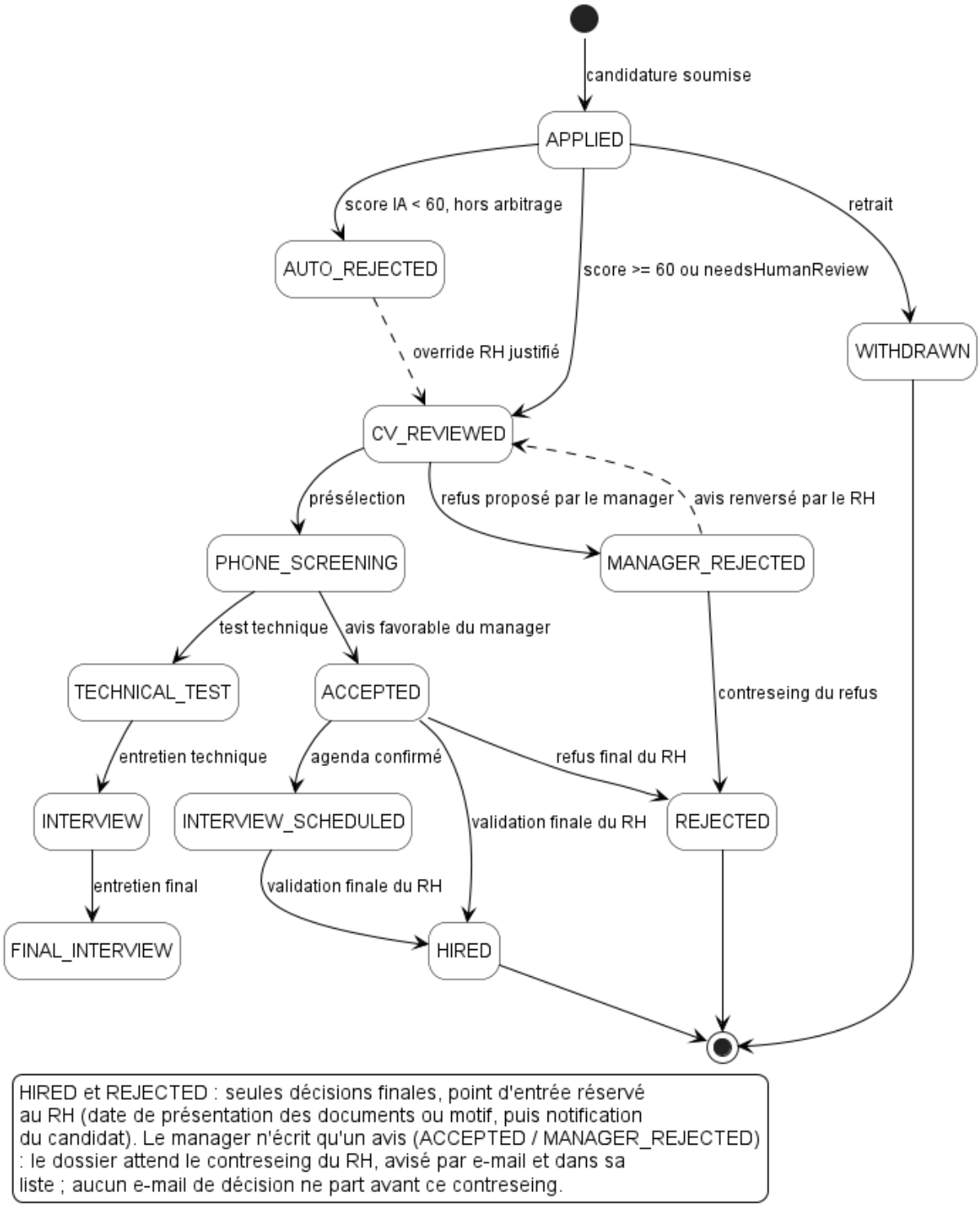



*Figure 3.8 : Diagramme d'états du cycle de vie d'une candidature*

Note – le statut INTERVIEW_SCHEDULED suit ACCEPTED : il matérialise la confirmation de l'agenda qui suit l'avis favorable et précède l'état définitif HIRED ; il ne désigne pas un entretien d'évaluation,

ceux du pipeline étant clos aux statuts INTERVIEW et FINAL_INTERVIEW. La plateforme ne modélise pas de type d'entretien dédié à l'onboarding, et la planification d'un entretien ne modifie pas le statut de la candidature : seule la machine à états porte l'enchaînement avis favorable → agenda confirmé → validation finale du RH.

On y distingue notamment un embranchement automatique piloté par le score de l'analyse IA : le dossier passe en CV_REVIEWED lorsque le score atteint le seuil de 60/100, en AUTO_REJECTED lorsqu'il descend sous ce seuil. Les dossiers signalés par le drapeau needsHumanReview — zone grise de 55 à 65, texte de CV inexploitable ou mode dégradé — font exception à cette règle : ils rejoignent CV_REVIEWED pour un arbitrage RH et ne sont jamais rejetés automatiquement (section 4.4.6). Le statut CV_REVIEWED y garde un sens unique, le dossier est prêt pour l'examen humain, mais la plateforme ne distingue pas par un état séparé le cas où le score suffit et le cas où un arbitrage est requis : ces deux situations sont signalées par le drapeau et par le motif enregistrés, non par le statut. La décision elle-même est à deux niveaux : ACCEPTED et MANAGER_REJECTED ne sont que des avis du manager, en attente du contreseing du RH, qui seul atteint HIRED (moyennant la date de présentation des documents) ou REJECTED (moyennant un motif) ; le RH peut aussi prononcer un refus final depuis n'importe quelle étape d'évaluation, et le candidat peut se retirer (WITHDRAWN) tant que le dossier n'est pas clos. Les états véritablement terminaux (HIRED, REJECTED, AUTO_REJECTED, WITHDRAWN) n'admettent plus aucune modification depuis l'interface d'administration, tandis qu'AUTO_REJECTED reste révisable par la correction manuelle du score décrite ci-après ; la transition vers INTERVIEW_SCHEDULED, qui suit l'avis favorable, est définie dans la note sous la figure 3.8.

Ce rejet automatique n'a rien de définitif, ni d'entièrement hors de contrôle humain : à tout moment, le RH ou l'Administrateur peut corriger manuellement le score d'un candidat via un mécanisme de surcharge (manualScore), qui exige une justification et journalise systématiquement la correction dans l'audit. Le score IA joue ainsi un rôle de filtre de présélection assisté — pas celui d'une autorité décisionnelle autonome.

Le cheminement d'une candidature à travers le scoring IA peut se résumer ainsi : CV candidat → dépôt du CV → déclenchement asynchrone de l'agent n8n « CV Parser » (extraction du texte du CV) → rappel du back-end via POST /api/candidates/{id}/ai-score/recompute → AiScoringService + CohereClient qui interrogent l'API de Cohere → calcul du score global sur 100 → orientation automatique vers CV_REVIEWED (score ≥ 60, ou dossier signalé needsHumanReview) ou AUTO_REJECTED (score < 60 hors zone grise) → arbitrage humain possible via le mécanisme de surcharge (manualScore).

Sur le plan de la méthode, le score n'est pas produit par une recherche de similarité vectorielle : la plateforme ne calcule aucun embedding, aucune distance cosinus, et n'exécute aucun modèle entraîné localement. Le back-end appelle le point d'entrée POST /v2/chat de l'API de Cohere (modèle command-a-03-2025, température nulle, deux cents jetons au plus, sortie contrainte au format JSON) avec un unique message qui fixe le rôle du modèle, impose le schéma de réponse {technique, communication, séniorité}, définit ces trois critères et interdit le zéro par défaut : faute d'information exploitable, le modèle doit renvoyer la valeur neutre de 50. Ce message porte les trois champs descriptifs de l'offre (poste, niveau requis, compétences requises) et le texte intégral du CV, lettre de motivation comprise. Les travaux publiés sur l'appariement automatique de CV et d'offres reposent le plus souvent sur une recherche de similarité vectorielle fondée sur des plongements [22] ; la solution décrite ici s'en écarte, le score étant produit par un modèle de langage et décomposé en trois critères lisibles plutôt que par une distance entre vecteurs.

La réponse est ensuite relue côté serveur : le bloc JSON est extrait du texte renvoyé, les intitulés de clés sont tolérants aux variantes et chaque note est bornée entre 0 et 100. Si aucune structure JSON exploitable n'est trouvée, le score n'est pas reconstitué à partir de nombres isolés : l'appel échoue et le mode dégradé local prend le relais, le score étant marqué SIMULATED et signalé pour arbitrage humain (section 4.4.6) ; le repli par expression régulière ne subsiste que comme garde-fou secondaire, lorsqu'un JSON valide aboutit à trois notes nulles. Une incohérence demeure néanmoins dans cette version : le prompt demande au modèle une valeur neutre de 50 faute d'information exploitable, alors qu'un critère manquant dans la réponse reçoit côté serveur une valeur par défaut de 70, plus favorable. Aligner cette valeur sur 50, voire signaler le dossier pour revue humaine au lieu de le noter, fait partie des corrections à apporter avant la préproduction. Le score final sur 100 est la moyenne pondérée arrondie (technique × w1 + communication × w2 + séniorité × w3), où les trois poids proviennent du profil de pondération associé à la famille de métier et au niveau de séniorité de l'offre (table scoring_weight_profiles, douze profils initialisés, valeurs par défaut 0,5 / 0,2 / 0,3, somme contrôlée à 1 à ±0,01 près). Un CV noté 80 en technique, 70 en communication et 60 en séniorité obtient ainsi 72 avec les poids par défaut, ce que vérifie un test dédié. Les compétences présentées comme correspondantes ou manquantes (matchedSkills / missingSkills), en revanche, ne sont pas revendiquées par le modèle : elles sont calculées localement par le service de normalisation des compétences, à partir d'un dictionnaire d'alias et de recherches à frontière de mot, afin de ne pas reposer sur la seule parole du LLM.

Ce choix de prestataire répond d'abord à une contrainte d'environnement : l'API d'inférence de Hugging Face était bloquée par la politique réseau de l'entreprise, tandis que le point d'entrée de Cohere a été vérifié accessible ; le service est managé, sans dépendance lourde ni poids de modèle à déployer, et sans GPU. La sortie contrainte au format JSON et la température nulle visent à réduire la variabilité des réponses, sans garantir une reproductibilité parfaite tant le modèle et le service externe peuvent évoluer ; la source du score (COHERE ou SIMULATED) est conservée, tandis que la version du modèle, celle du prompt et la réponse brute ne sont pas encore archivées, un enregistrement à prévoir pour rendre une évaluation réellement rejouable, et le classement des offres par compatibilité est regroupé en un seul appel pour un lot d'au plus vingt offres afin de maîtriser le quota. Un modèle embarqué du type sentence-transformers aurait supposé une infrastructure d'exécution et un entraînement adaptés au français des offres ; le besoin réel était une présélection assistée, réversible et expliquée, adossée à un service dont l'indisponibilité est déjà couverte par le mode dégradé local décrit plus haut.

## 3.7 Conception de la base de données

La base de données relationnelle PostgreSQL reflète directement le diagramme de classes présenté en section 3.3 : chaque entité JPA est mappée sur une table, et les relations One-to-Many et Many-to-One sont matérialisées par des clés étrangères (par exemple candidate.job_offer_id, interview.candidate_id, feedback.interview_id). J'ai configuré la génération du schéma en mode « update » (Hibernate ddl-auto) — un choix pratique en phase de développement, qui permet de faire évoluer la structure de la base sans perte de données au fil des itérations. En production, ce mode devrait néanmoins céder la place à un outil de migration versionnée comme Flyway ou Liquibase, seul capable de garantir des évolutions de schéma maîtrisées et reproductibles.

Le schéma relationnel ci-dessous présente l'ensemble des tables générées à partir des entités JPA, avec les clés primaires et étrangères qui matérialisent les relations décrites ci-dessus.

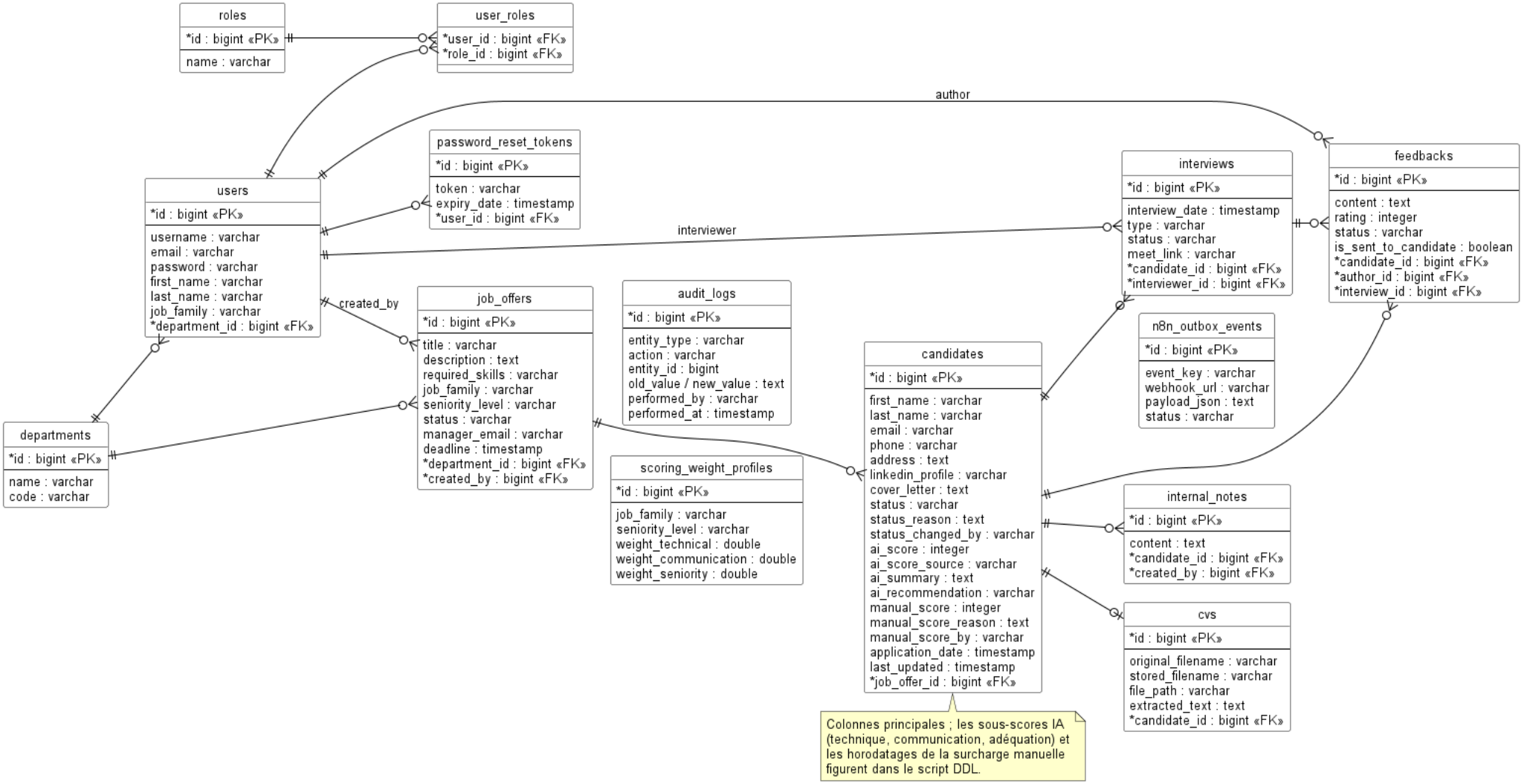



*Figure 3.9 : Schéma relationnel de la base de données PostgreSQL*

Le schéma relationnel reprend fidèlement les entités JPA : les clés primaires sont préfixées d'un astérisque et les clés étrangères suffixées « FK ». On y lit les relations structurantes : un département rattache les utilisateurs et les offres, une offre reçoit les candidatures, chaque candidature dépose un CV et passe des entretiens qui donnent lieu à des feedbacks rédigés par les utilisateurs.

## 3.8 Diagramme de composants

Le diagramme ci-dessous détaille l'organisation de la plateforme en composants logiciels (front-end Angular et back-end Spring Boot), ainsi que leurs dépendances.

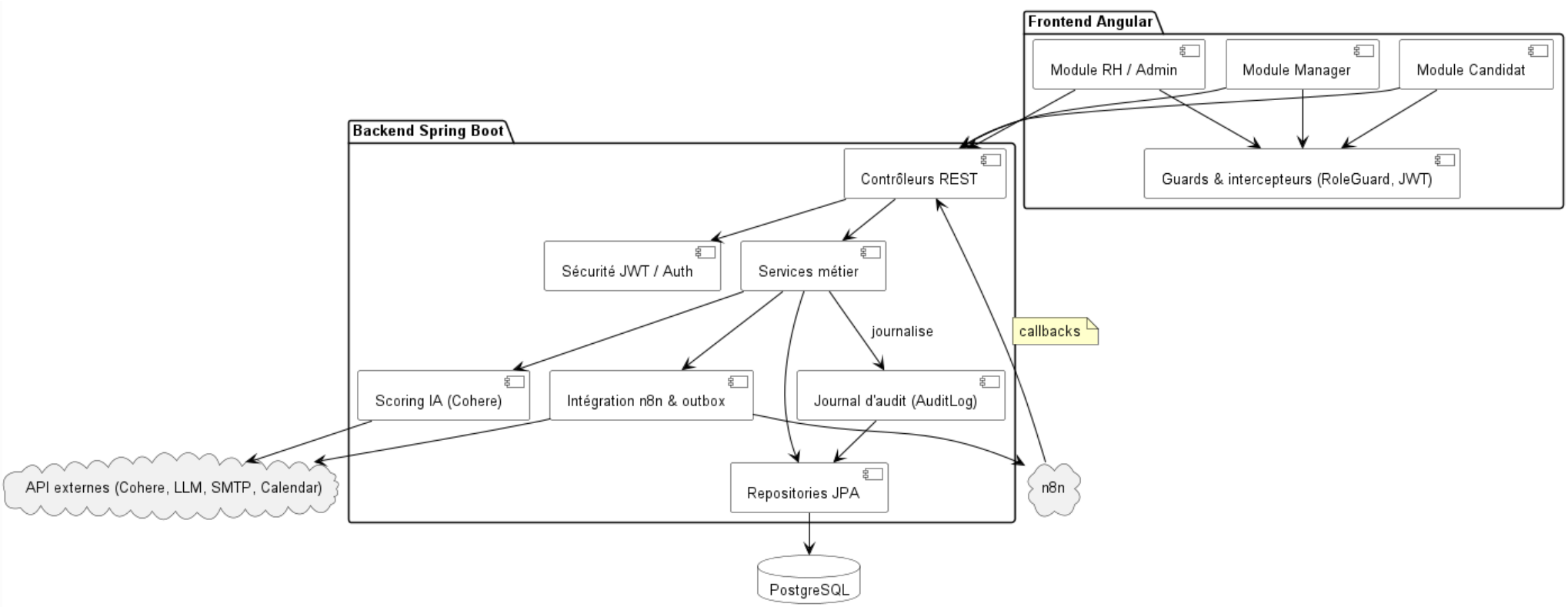



*Figure 3.10 : Diagramme de composants de la plateforme JOB4YOU*

Chaque service métier (Candidate, Interview, Feedback, JobOffer) s'appuie sur la couche de repositories Spring Data JPA pour la persistance, et sur le service d'audit pour tracer les actions sensibles. Le service Candidate délègue le calcul du score au service de scoring IA, tandis que Candidate et Interview déclenchent chacun les agents n8n compétents via des webhooks.

## 3.9 Diagramme de déploiement

Le diagramme suivant représente la répartition physique des composants de la plateforme JOB4YOU sur les nœuds d'exécution, ainsi que leurs protocoles de communication.

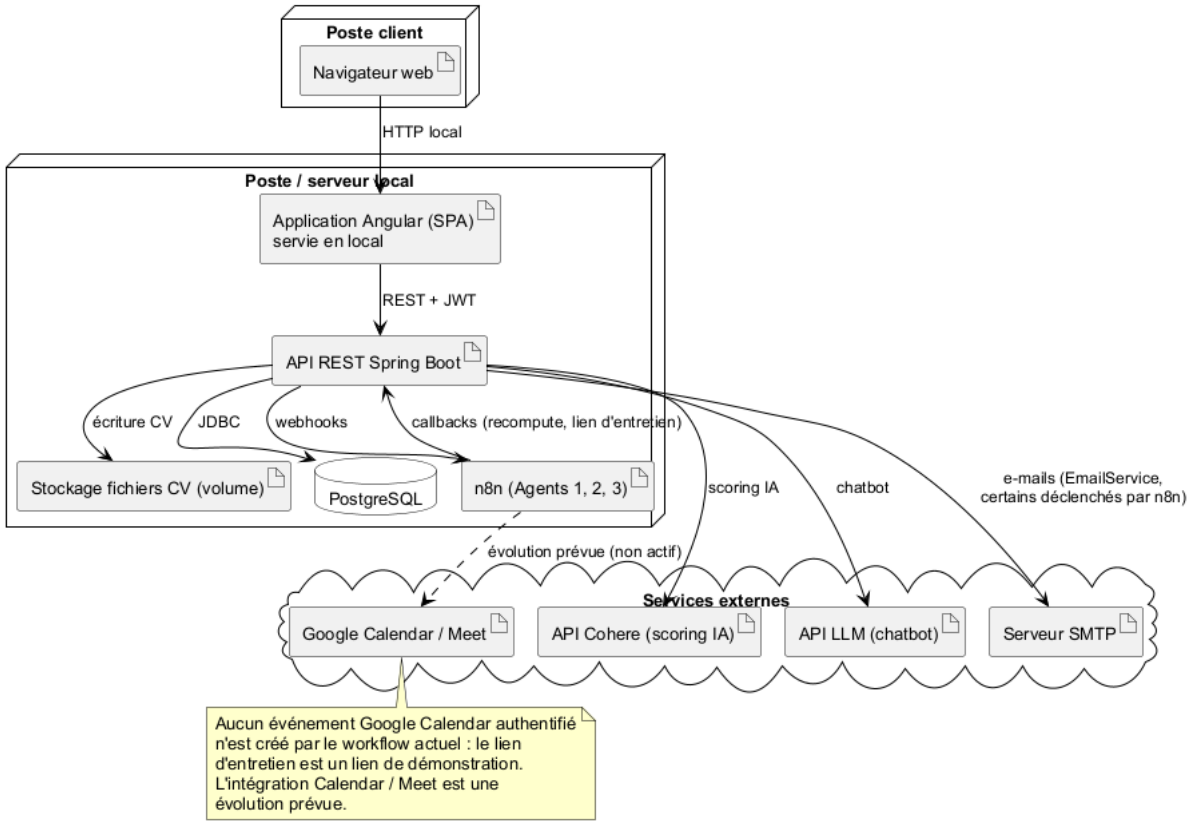



*Figure 3.11 : Diagramme de déploiement de la plateforme JOB4YOU*

Dans la configuration de développement représentée, le front-end Angular, le back-end Spring Boot, les agents n8n et le stockage des CV s'exécutent sur un même poste local, avec une base de données PostgreSQL (SGBD) pour la persistance. Le front-end communique avec le back-end via des API REST sécurisées par JWT. Le back-end déclenche, via des webhooks HTTP, les agents d'automatisation n8n — lesquels sollicitent à leur tour Google Calendar pour la création réelle de l'événement et du lien Google Meet lors de la planification, et le serveur SMTP pour les notifications — tandis que le back-end appelle lui-même l'API de Cohere pour le scoring IA et l'API OpenRouter pour l'assistant conversationnel.

## 3.10 Conclusion

Ces modèles — paquetages, classes, séquences, états, base de données, déploiement et composants — forment ensemble le plan directeur qui a guidé le développement de l'application, avec les ajustements décrits au chapitre suivant. Reste à voir comment ce plan s'est concrètement traduit en une application fonctionnelle : c'est l'objet du chapitre suivant.

# Chapitre 4 : Réalisation

Ce dernier chapitre est celui où la conception prend enfin forme concrète. J'y présente l'environnement de travail utilisé, les interfaces réellement développées et fonctionnelles, la démarche de test suivie, ainsi que certaines difficultés rencontrées en cours de route et la façon dont je les ai résolues.

## 4.1 Introduction

Toutes les captures d'écran qui suivent proviennent de l'application en fonctionnement, pas de maquettes : chaque interface présentée a été testée et utilisée pendant le stage.

## 4.2 Environnement de travail

### *4.2.1 Environnement matériel*

J'ai réalisé le développement sur un poste de travail standard mis à disposition par l'entreprise, doté d'un processeur multi-cœurs et d'une mémoire vive suffisante pour faire tourner simultanément l'environnement de développement, le serveur back-end, le serveur front-end, la base de données PostgreSQL et le serveur n8n en local.

### *4.2.2 Environnement logiciel*

Les outils et technologies suivants ont été mobilisés tout au long du projet :

- Langages : Java 17, TypeScript, HTML5, SCSS, SQL.
- Frameworks et bibliothèques : Spring Boot 3 (Web, Data JPA, Security), Angular 20, Bootstrap.
- Base de données : PostgreSQL, administrée via pgAdmin.
- Automatisation et IA : n8n (agents d'automatisation par webhooks), API de Cohere (scoring IA des CV). Assistant conversationnel candidat : API OpenRouter (modèle z-ai/glm-5.3-flash, format compatible OpenAI).
- Environnements de développement intégrés : Eclipse STS (Spring Tool Suite) pour le développement back-end Java/Spring Boot, Visual Studio Code pour le développement front-end Angular.
- Modélisation UML : StarUML pour la conception des diagrammes de cas d'utilisation, de classes et de séquence présentés aux chapitres 2 et 3.
- Gestion de version : Git, hébergement du code sur GitHub.
- Outils de test et de documentation d'API : Postman pour les tests manuels des API REST, Swagger (springdoc-openapi) pour la documentation interactive générée automatiquement.
- Build et gestion de dépendances : Maven pour le back-end, npm pour le front-end.

### 4.2.3 Gestion de version

J'ai organisé le versionnement du code autour d'une branche principale, avec des messages de commit descriptifs qui facilitent la traçabilité des évolutions fonctionnelles et des corrections apportées tout au long du stage.

### 4.2.4 Sécurité applicative

Plusieurs mécanismes concrets protègent la plateforme et les données des candidats. Les mots de passe ne sont jamais stockés en clair : ils sont hachés avec BCrypt avant persistance. L'authentification s'appuie sur un jeton JWT signé [14], valable 24 heures, transmis à chaque requête et vérifié côté back-end pour en extraire l'identité et les rôles de l'utilisateur. Chaque point d'entrée de l'API REST est protégé par annotation selon le rôle requis (@PreAuthorize), et un contrôle d'accès supplémentaire restreint, pour un manager, la visibilité aux seuls candidats de son périmètre — déterminé prioritairement par son département, la famille de métier ne servant de repli que pour les offres non rattachées à un département. Le dépôt de CV est limité aux formats PDF, DOC et DOCX, avec une taille maximale de 10 Mo côté API, resserrée à 5 Mo par la validation du formulaire Angular, pour limiter les risques liés au téléversement de fichiers. Des mécanismes complémentaires, comme l'analyse antivirus ou une validation plus poussée du contenu au-delà de la seule extension et du type MIME, pourraient compléter ce dispositif en production, conformément aux recommandations de l'OWASP [15].

## 4.3 Interfaces graphiques de l'application

Cette section présente une sélection des interfaces principales, illustrant les fonctionnalités clés de la plateforme JOB4YOU pour chacun des profils d'utilisateurs.




*Figure 4.1 : Page d'accueil publique — recherche d'offres d'emploi*

Accessible sans authentification, la page d'accueil permet à tout visiteur de rechercher une offre par mot-clé et par localisation, et met en avant les offres publiées les plus récentes.

L'authentification se fait par nom d'utilisateur et mot de passe ; le jeton JWT renvoyé par le back-end sécurise ensuite l'ensemble des requêtes vers l'API. Sa capture est reprise en annexe A (figure A.1).

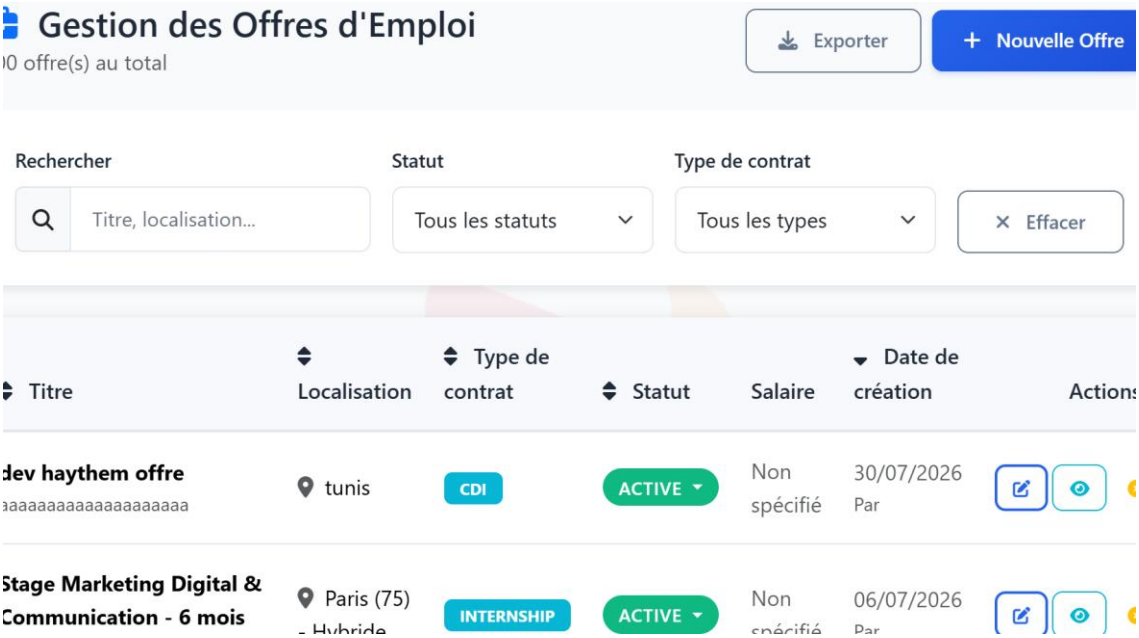



*Figure 4.2 : Gestion des offres d'emploi (espace RH)*

Le Responsable RH crée, modifie et suit ici le statut de chaque offre d'emploi (active, fermée, expirée) ; la bascule en « expirée » se fait automatiquement une fois la date limite dépassée.

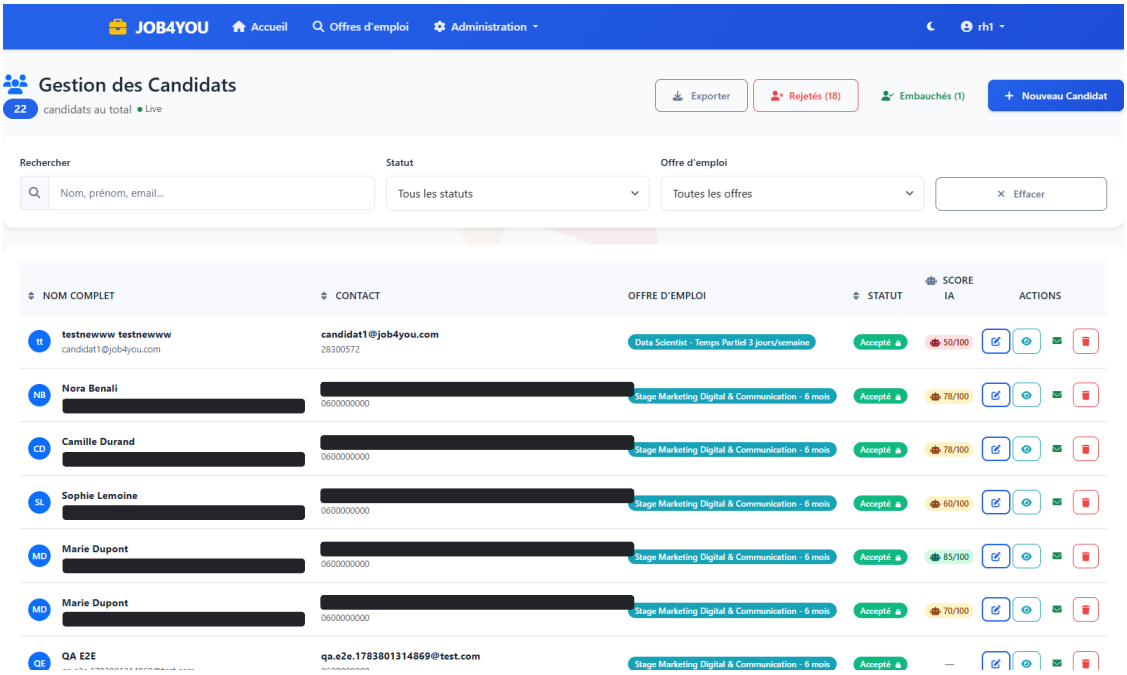



*Figure 4.3 : Gestion des candidats — vue liste avec score IA*

Pour chaque dossier, la liste des candidats affiche le statut courant, le score calculé par l'agent IA, ainsi qu'un ensemble d'actions rapides (changement de statut, envoi d'e-mail, consultation, suppression).

Le RH corrige ici les informations d'un candidat (coordonnées, lettre de motivation, statut, offre associée) via le formulaire de modification ; la fiche de consultation associée regroupe le CV, l'analyse IA détaillée (scores technique, communication, adéquation séniorité) et l'historique des actions des agents n8n sur ce dossier. Sa capture est reprise en annexe A (figure A.2).

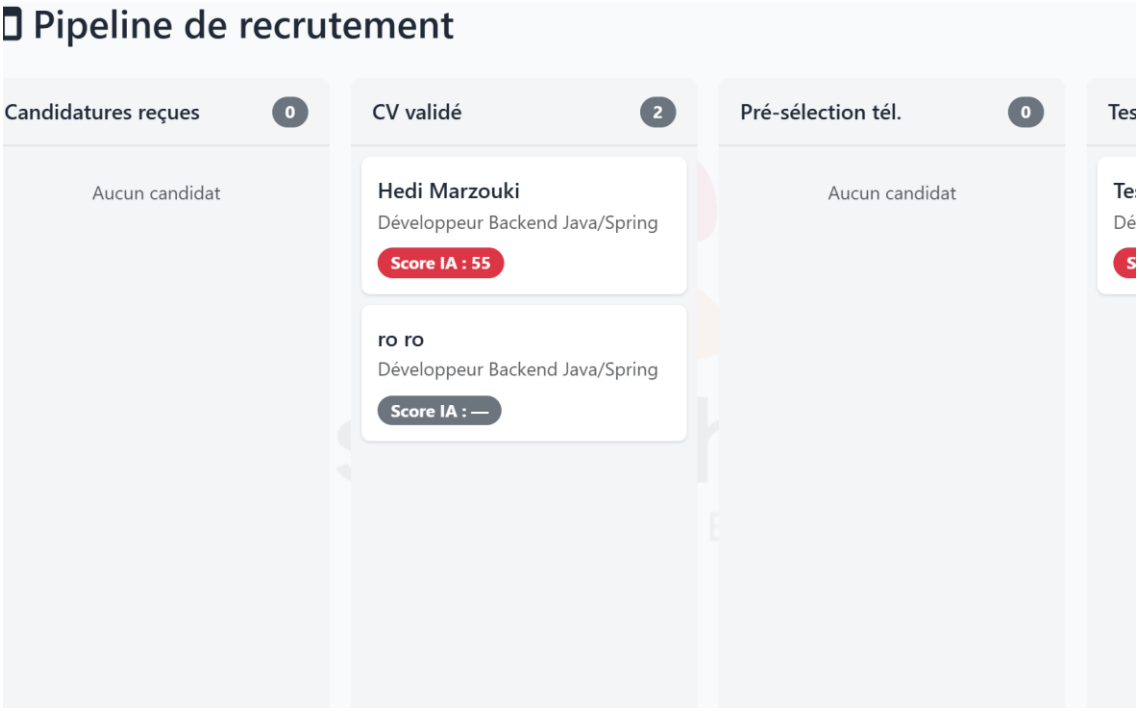



*Figure 4.4 : Tableau Kanban du pipeline de recrutement*

Le tableau Kanban donne une vue synthétique et interactive de toutes les candidatures réparties par statut ; glisser une carte d'une colonne à une autre déclenche automatiquement la mise à jour du statut, sous réserve des garde-fous métier décrits au chapitre 3 pour les décisions finales.

On planifie, filtre et suit ici les entretiens (type, date, interviewer, statut), avec des actions contextuelles comme le démarrage, l'annulation, la reprogrammation ou l'ajout d'un feedback une fois l'entretien terminé. Sa capture est reprise en annexe A (figure A.3).

Les feedbacks saisis par les managers sont centralisés ici, avec une étape de validation par le RH avant transmission au candidat. Sa capture est reprise en annexe A (figure A.4).

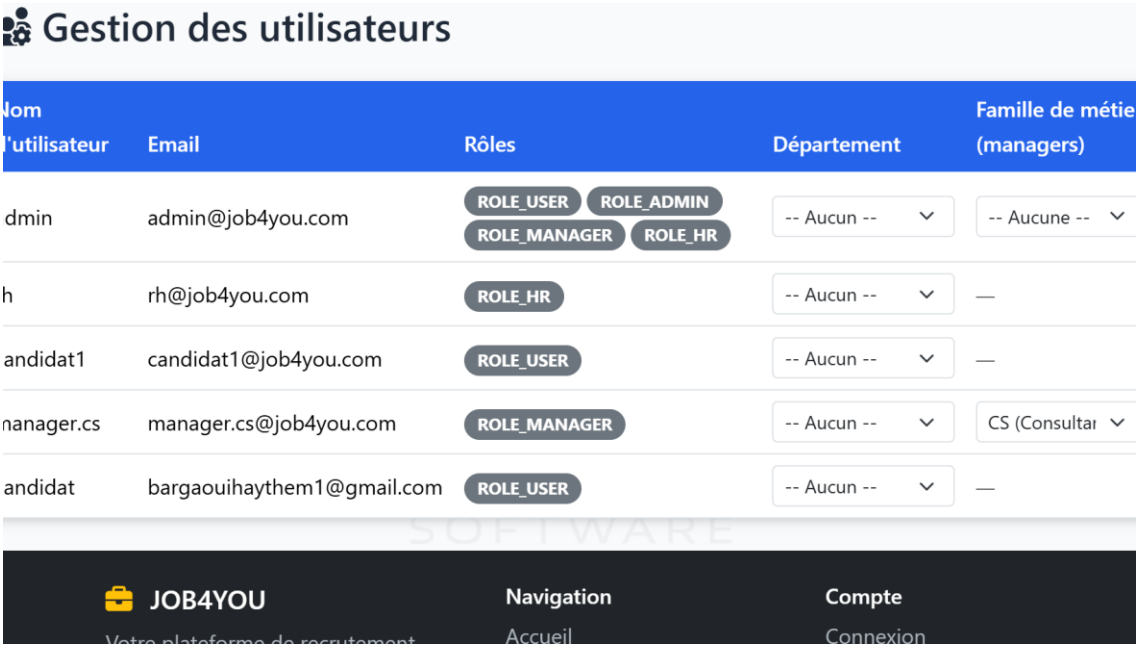



*Figure 4.5 : Gestion des utilisateurs et des rôles (espace Administrateur)*

L'administrateur y gère l'ensemble des comptes de la plateforme, leurs rôles, ainsi que leur rattachement à un département ou une famille de métier.

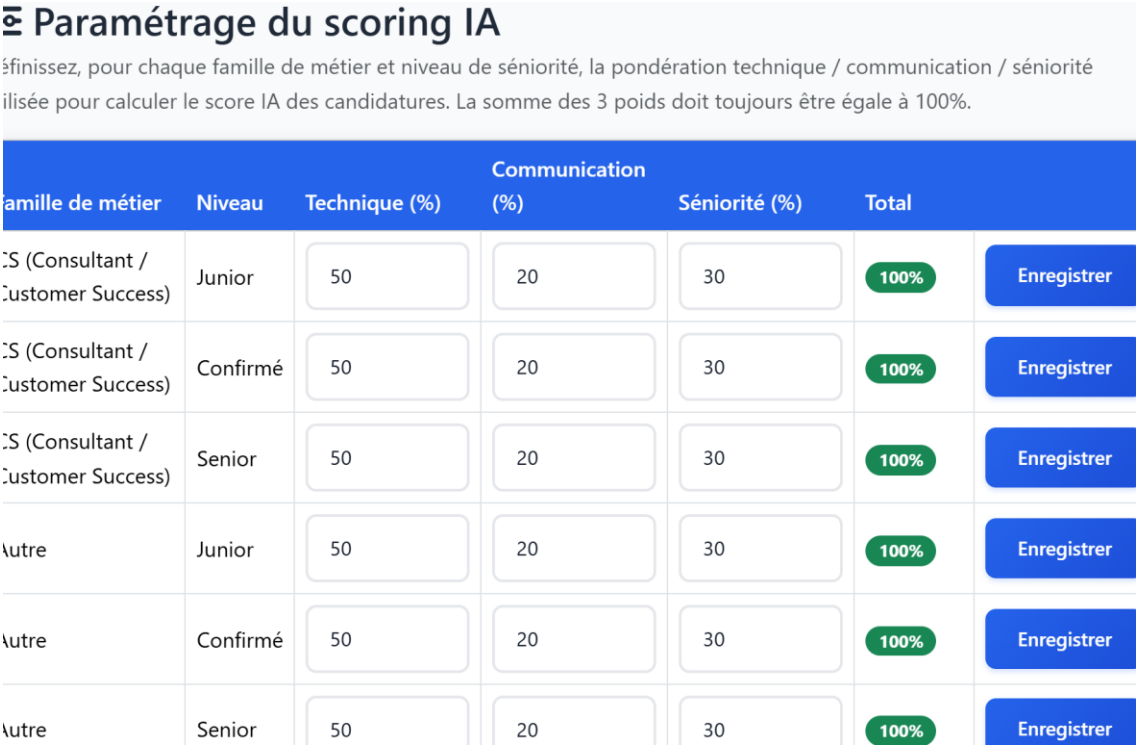



*Figure 4.6 : Paramétrage du scoring IA par famille de métier*

Réservée à l'administrateur, cette interface permet d'ajuster la pondération (technique / communication / séniorité) que l'agent IA utilise pour calculer le score de chaque candidature, selon la famille de métier et le niveau de séniorité de l'offre.

Le journal d'audit trace tous les changements de statut significatifs, initiés manuellement par un utilisateur ou automatiquement par le système (agents n8n), ce qui garantit la traçabilité de chaque décision prise sur un dossier. Sa capture est reprise en annexe A (figure A.5).

J'ai aussi donné à chaque manager une page « Mon calendrier », en lecture seule, qui regroupe ses entretiens du mois et ses trois prochains rendez-vous ; les données y sont filtrées côté serveur selon son département, sa famille de métier ou son affectation directe, le même cloisonnement que pour la liste des dossiers candidats.

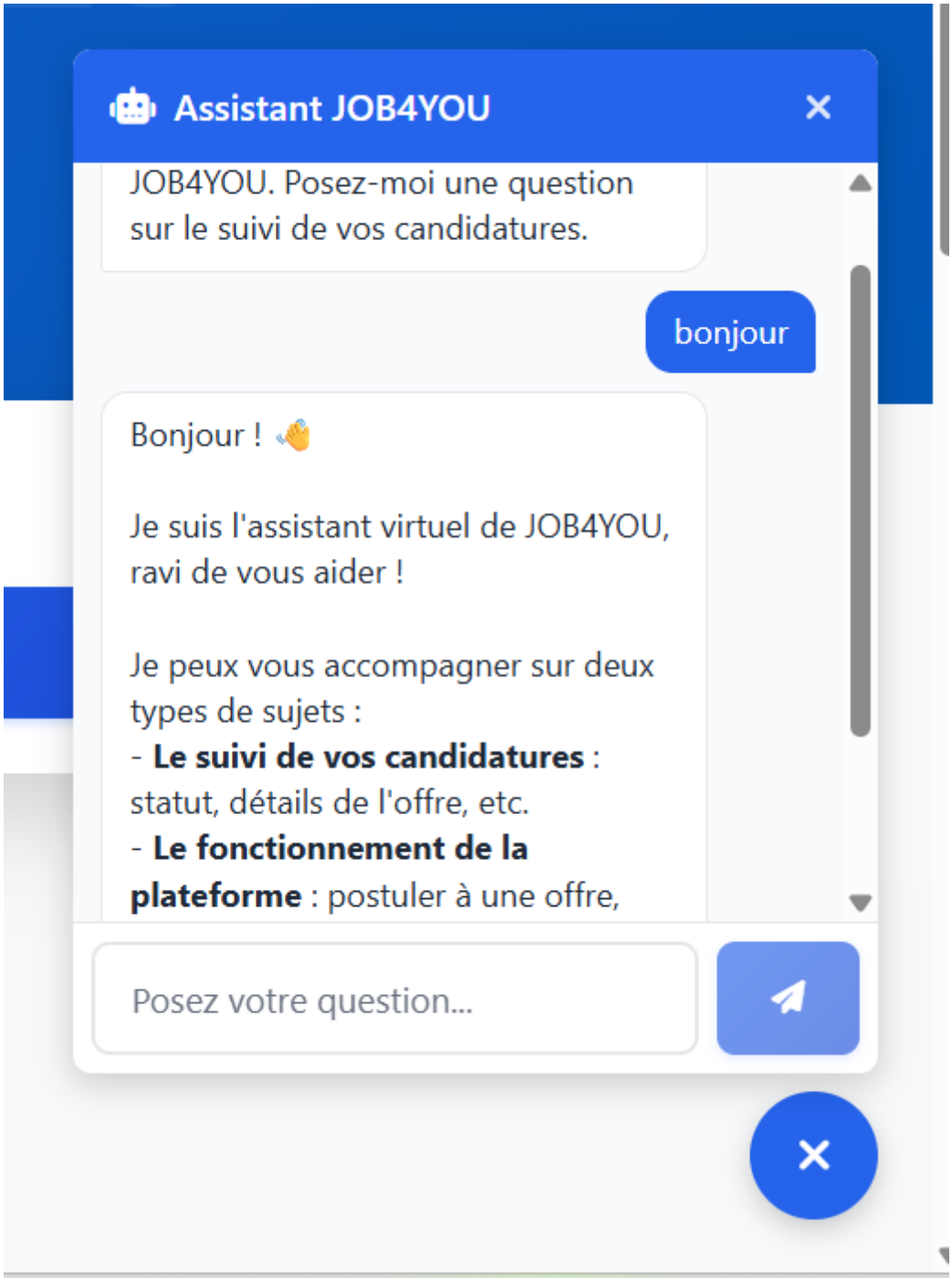



*Figure 4.7 : Assistant conversationnel candidat (chatbot)*

Disponible uniquement pour un candidat authentifié, ce widget flottant permet de poser des questions en langage naturel sur le suivi de ses propres candidatures ou sur le fonctionnement

général de la plateforme (offres, Job idéal, retrait de candidature, etc.). Les réponses sont générées par un modèle de raisonnement (z-ai/glm-5.3-flash, via l'API OpenRouter) à partir d'un contexte constitué côté serveur par une liste blanche de champs : intitulé du poste, libellé du statut, date de candidature, entretiens et, le cas échéant, les feedbacks explicitement transmis au candidat. Les champs internes (score et résumé générés par l'agent de scoring, correction manuelle du RH, identité de l'intervieweur, notes internes d'entretien) n'entrent pas dans cette liste et ne quittent donc pas le back-end vers le prompt ; ce filtre structurel, complété par les consignes du message système, a été vérifié sur les scénarios testés, y compris une tentative d'injection de prompt.

Un candidat connecté postule directement depuis une offre : le formulaire pré-remplit son e-mail de compte, valide le format du CV (PDF, DOC, DOCX) et confirme visuellement la sélection avant l'envoi. C'est ce dépôt qui déclenche la chaîne décrite en section 3.5.3 (extraction du texte par l'agent n8n, analyse par l'API de Cohere appelée par le back-end, score automatique). Sa capture est reprise en annexe A (figure A.6).

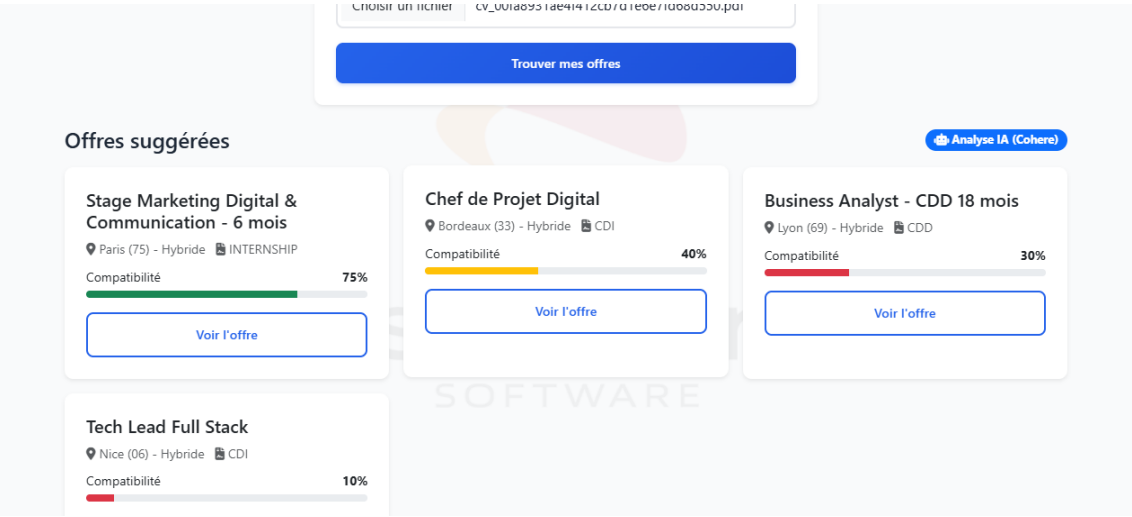



*Figure 4.8 : Fonctionnalité « Job idéal » — suggestions d'offres selon le CV*

Sans postuler à une offre précise, un visiteur ou un candidat dépose son CV et reçoit une liste des offres actives classées par pourcentage de compatibilité, calculé par l'API de Cohere (badge « Analyse IA (Cohere) », avec repli automatique sur un classement par mots-clés si l'IA est indisponible).

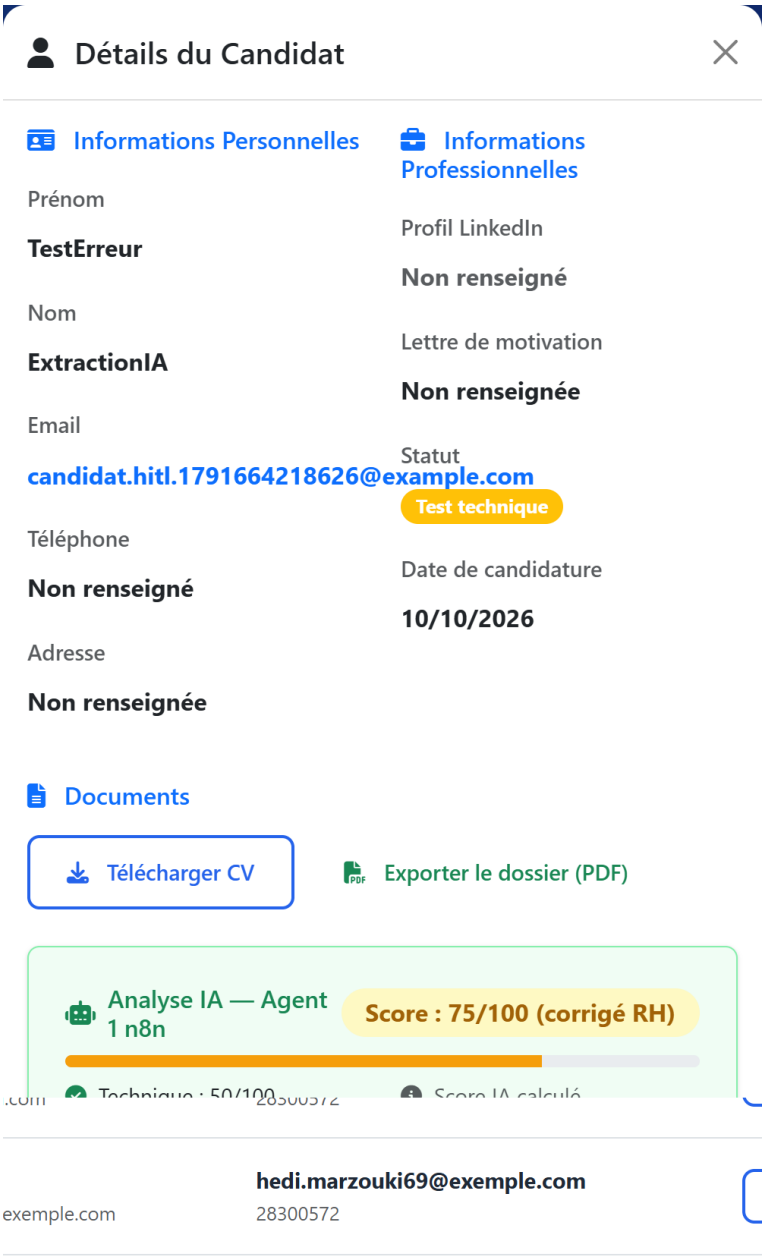



*Figure 4.9 : Analyse détaillée d'une candidature par intelligence artificielle*

Au-delà du score global visible en liste (Figure 4.3), le détail d'un dossier expose la répartition par critère — technique, communication, adéquation séniorité — ainsi que, le cas échéant, la correction manuelle appliquée par un RH avec sa justification, conformément au principe de contrôle humain décrit en section 4.6.

Un manager ne voit que les candidatures relevant de son périmètre (département, ou à défaut famille de métier — voir ManagerRoutingService, section 3.5.4), à partir de leur entrée en évaluation technique (statut TECHNICAL_TEST), et non dès la validation du CV par le RH. Cette vue cloisonnée est distincte de la liste globale accessible au RH et à l'administrateur (Figure 4.3). Sa capture est reprise en annexe A (figure A.7).

Le manager dispose en outre d'une page « Mon calendrier » qui centralise ses propres entretiens : prochaine session, lien de visioconférence, répartition par statut et vue mensuelle, avec un rappel automatique planifié deux heures avant chaque entretien dont il est intervieweur. La liste reste

filtrée côté serveur selon son département, sa famille de métier ou son affectation, de sorte qu'aucun dossier hors de son périmètre ne peut y apparaître.

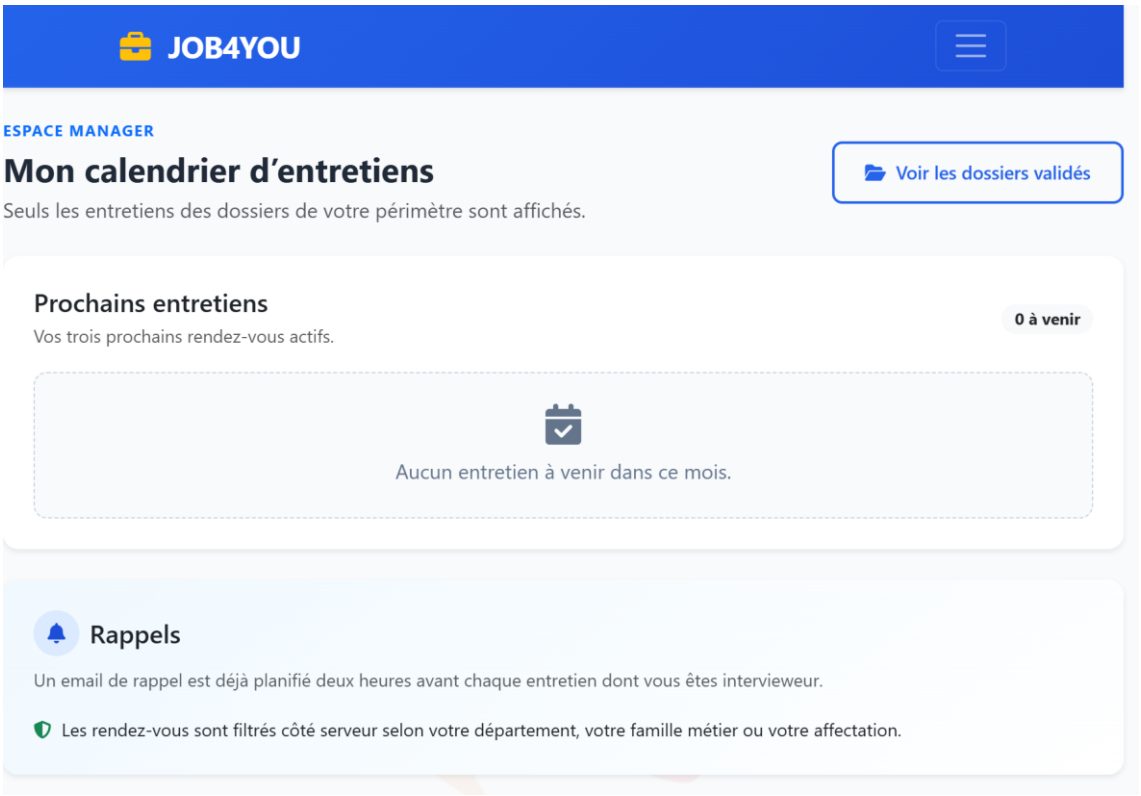



*Figure 4.10 : Espace Manager — page « Mon calendrier », entretiens et rappels*

Au-delà des interfaces de la plateforme, la chaîne d'automatisation a été vérifiée dans la console n8n elle-même. La figure A.8, en annexe, montre les trois flux déclarés dans l'application — « Agent 1 — CV Parser + Score IA + Email Candidat », « Agent 2 — Planification entretien + Email participants » et « Agent 3 — RH vers Manager (dossier validé) » — tous trois à l'état Publié, avec quinze exécutions sur l'instance locale ayant servi à la démonstration, aucun échec dans cet échantillon (taux d'échec 0 %) et une durée moyenne d'exécution de 3,61 s. Ces mesures relèvent d'un environnement local et non d'une exploitation en production ; l'échantillon reste trop limité pour valoir test de charge ou garantie de disponibilité.

Le détail de l'Agent 1 (figure A.9, en annexe) confirme le découpage des responsabilités décrit au chapitre 3 : le webhook « Nouvelle candidature » reçoit l'événement POST émis par Spring Boot, le nœud « Cohere — Score IA (Spring Boot) » rappelle le back-end qui interroge réellement l'API de Cohere et persiste le score, puis « Préparer données email » alimente le nœud d'envoi de la confirmation au candidat.

Côté candidat, la notification produite par ce flux arrive bien dans sa boîte de réception (figure A.10, en annexe) : accusé de réception nominatif pour le poste visé, score d'analyse IA obtenu et horodatage du traitement, avec la mention d'envoi automatique par n8n.

## 4.4 Tests

### *4.4.1 Tests unitaires (JUnit 5 / Mockito)*

Au-delà des vérifications manuelles, la couche service et une partie de la couche controller sont couvertes par une suite de tests automatisés écrite avec JUnit 5 et Mockito [16], exécutée via Maven. Elle comprend 12 classes de test pour un total de 121 méthodes de test : CandidateServiceTest, JobOfferServiceTest, ManagerRoutingServiceTest, N8nServiceTest, ScoringWeightProfileServiceTest, AiScoringServiceTest, InterviewServiceTest et InterviewServiceManagerCalendarTest en tests unitaires purs (services isolés à l'aide de doublons Mockito), ainsi que CandidateControllerWorkflowTest, CandidateStatusAnonymousSecurityTest, InterviewManagerCalendarSecurityTest et InterviewMeetLinkSecurityTest en tests de contrôleur reposant sur @WebMvcTest et MockMvc, qui vérifient notamment le comportement des routes accessibles de façon anonyme et la sécurité d'accès au lien Google Meet d'un entretien. Ces tests couvrent en particulier le routage des managers par département, l'isolation stricte de la visibilité des dossiers, le calcul du score IA et ses seuils, le découplage entre planification d'entretien et statut du candidat, ainsi que les garde-fous de transition décrits au chapitre 3.

L'extrait ci-dessous, tiré de CandidateServiceTest, illustre l'un des deux tests ajoutés pour valider le correctif décrit en section 4.5.4 : il vérifie qu'un manager n'obtient, via getCandidatesByStatus(), que les candidats de son propre périmètre.

```
@Test
@DisplayName("getCandidatesByStatus() ne renvoie à un manager que les candidats de son
périmètre")
void shouldScopeGetCandidatesByStatusToCurrentManager() {
    Candidate inScope = candidateWithOffer(60L, "manager.rd@company.com", rd);
    Candidate outOfScope = candidateWithOffer(61L, "manager.devops@company.com", devops);
    User manager = makeManagerUser("manager_rd", "manager.rd@company.com", rd);
    authenticateAs("manager_rd", Role.ERole.ROLE_MANAGER);
    when(userRepository.findByUsername("manager_rd")).thenReturn(Optional.of(manager));
    when(candidateRepository.findByStatus(CV_REVIEWED)).thenReturn(List.of(inScope,
outOfScope));
    List<CandidateResponse> result = candidateService.getCandidatesByStatus(CV_REVIEWED);
    assertThat(result).extracting("id").containsExactly(60L);
}
```

### *4.4.2 Tests fonctionnels manuels*

Chaque fonctionnalité développée a fait l'objet d'un test fonctionnel manuel directement dans l'application, couvrant à la fois le scénario nominal et les principaux cas d'erreur. Le tableau suivant synthétise les cas de test exécutés et leur résultat.

*Tableau 4.1 : Synthèse des tests fonctionnels réalisés*

| **Cas de test** | **Résultat attendu** | **Résultat obtenu** |
|---|---|---|
| Connexion avec identifiants valides | Connexion réussie, jeton JWT renvoyé | OK |
| Connexion avec mot de passe invalide | Erreur 401, accès refusé | OK |
| Accès à une route RH avec un compte Candidat | Erreur 403 (Forbidden) | OK |
| Publication d'une offre d'emploi par le RH | Offre créée et visible sur la page publique | OK |
| Dépôt d'une candidature avec CV | Candidature créée (statut APPLIED), CV stocké et lié au dossier | OK |
| Analyse automatique du CV par l'agent IA | Score calculé, statut CV_REVIEWED ou AUTO_REJECTED selon le seuil | OK |
| Avis manager sur un dossier non encore validé par le RH | Erreur 400, avis bloqué par le garde-fou métier | OK |
| Avis manager après validation RH | Avis enregistré (ACCEPTED ou MANAGER_REJECTED) et notifié au RH, sans e-mail au candidat | OK |
| Planification d'un entretien sur un dossier clos (rejeté/retiré) | Erreur 400, planification bloquée | OK |
| Notification par e-mail au candidat après la décision finale contre-signée par le RH | E-mail reçu, contenu cohérent avec le statut | OK |
| Routage RH → manager selon le département de l'offre (repli famille de métier) | Seul le manager du département concerné est notifié et voit le dossier | OK |
| Notification du manager à l'entrée en Test technique (après entretien téléphonique) | Seul le manager du département est notifié à ce stade, pas avant | OK |
| Visibilité d'un manager sur un dossier d'un autre département | Aucun dossier hors de son département n'est visible | OK |
| Retrait d'une candidature (statut WITHDRAWN) | Le dossier quitte la liste active sans rechargement et reste filtrable | OK |
| Acceptation directe d'un candidat depuis CV_REVIEWED (sans entretien tél.) | Erreur 400, transition refusée | OK |

| Cas de test | Résultat attendu | Résultat obtenu |
|---|---|---|
| Planification d'un entretien sur un candidat déjà en Test technique | Aucun effet sur le statut du candidat (agenda seul) | OK |
| Question du chatbot sur le statut réel d'une candidature | Réponse conforme aux données réelles en base, sans invention | OK |
| Demande du score IA ou de l'identité de l'intervieweur au chatbot | Refus poli, aucune donnée interne révélée | OK |
| Tentative d'injection de prompt sur le chatbot (« ignore tes instructions... ») | Le contexte système n'est pas divulgué, le rôle de l'assistant est conservé | OK |
| Appel du point d'entrée du chatbot sans jeton JWT | Erreur 401, accès refusé | OK |

Ce tableau atteste de l'exécution des vingt scénarios listés, tous concluants ; il ne constitue pas pour autant une garantie globale de qualité. Des cas nominaux et des cas d'erreur qui réussissent ne prouvent pas l'absence de défauts sur les chemins non exercés, et ne disent rien du comportement de la plateforme sous charge. Ces tests vérifient des fonctionnalités et des règles d'accès : ils ne mesurent pas la pertinence du scoring IA, qui fait l'objet de la section 4.4.5. Aucun test de charge n'a mesuré les exigences non fonctionnelles de temps de réponse fixées en section 2.2.3 (moins de 500 ms en lecture, moins de 2 s en écriture) : elles restent à valider en préproduction.

### *4.4.3 Tests des API avec Postman et Swagger*

J'ai testé isolément, à l'aide de Postman, l'ensemble des points d'entrée de l'API REST — en particulier les scénarios d'authentification, de contrôle d'accès par rôle et de gestion des erreurs (404, 400, 403). La documentation interactive Swagger, générée automatiquement à partir des annotations du code, m'a permis de vérifier la cohérence des contrats d'interface tout au long du développement.

### *4.4.4 Vérification de bout en bout*

Au-delà des tests unitaires par fonctionnalité, j'ai mené une vérification de bout en bout du parcours complet de recrutement : soumission d'une candidature, analyse automatique par l'IA, validation par le RH, planification d'un entretien, saisie d'un feedback, avis du manager puis contre-signature du RH — chaque étape déclenchant la notification par e-mail attendue, au candidat et/ou aux parties prenantes internes. Le fractionnement de la décision en deux niveaux, et les erreurs renvoyées quand un manager tente de trancher seul, sont en outre figés par des tests unitaires côté back-end (classe CandidateServiceTest, section 4.4.1).

### *4.4.5 Évaluation du matching sur un jeu de référence annoté (Gold Set)*

Les tests décrits ci-dessus vérifient que la plateforme fonctionne ; ils ne disent rien de la pertinence du score attribué par l'IA. J'ai donc constitué un jeu de référence — un Gold Set [21] — de seize curriculum vitæ synthétiques, rédigés pour ce projet à partir de profils réalistes mais fictifs : aucun CV réel d'un collaborateur de Sopra HR n'a été employé, ce qui écarte toute question de confidentialité dans la démarche d'évaluation.

Les seize profils se répartissent sur les trois familles de métiers visées par la plateforme, chacune rattachée à une offre cible : Développeur Backend Java/Spring (offre 115, six profils), DevOps Engineer (offre 117, cinq profils) et Business Analyst (offre 122, cinq profils). Chaque CV porte une vérité terrain binaire, PERTINENT ou NON_PERTINENT, arrêtée sur des critères objectifs — compétences techniques indispensables et séniorité — et non d'après la sortie du modèle : neuf profils sont pertinents, sept sont volontairement hors cible (compétences d'un autre domaine, séniorité insuffisante, outillage sans rapport avec l'offre).

Le protocole s'appuie sur le point d'entrée POST /api/matching/evaluate-gold-set, qui reçoit un CV et l'identifiant de l'offre cible, rejoue la chaîne complète (extraction du texte, normalisation des compétences, appel à l'API de Cohere, pondération par profil de scoring) et renvoie le score avec son explication, sans créer de candidature. La décision automatique — le score atteint-il le seuil — est alors confrontée à la vérité terrain, ce qui donne la matrice de confusion du tableau 4.2 pour trois seuils.

*Tableau 4.2 : Matrice de confusion du Gold Set selon le seuil de décision*

| **Critère** | **Seuil = 50** | **Seuil = 60 (retenu)** | **Seuil = 70** |
|---|---|---|---|
| Vrais positifs (TP) | 9 | 9 | 8 |
| Faux positifs (FP) | 0 | 0 | 0 |
| Vrais négatifs (TN) | 7 | 7 | 7 |
| Faux négatifs (FN) | 0 | 0 | 1 |
| Précision | 100 % | 100 % | 100 % |
| Rappel | 100 % | 100 % | 88,9 % |
| F1-score | 100 % | 100 % | 94,1 % |
| Exactitude | 100 % | 100 % | 93,8 % |

Avec le seuil retenu de 60/100, le système prend seize bonnes décisions sur seize : aucun profil pertinent n'est écarté (rappel de 100 %) et aucun profil hors cible n'est retenu (précision de 100 %). Abaisser le seuil à 50 ne change rien sur ce jeu, les deux seuils y sont donc indiscernables ; le relever à 70 fait apparaître un faux négatif — le profil GOLD_14, développeuse Java dont le dossier est pourtant jugé pertinent et scoré 69/100 — et fait chuter le rappel à 88,9 %.

Le seuil de 60 est cohérent avec la distribution des scores obtenus : le meilleur profil non pertinent s'arrête à 47/100, soit treize points sous le seuil, tandis que le plus faible profil pertinent est à 69/100, soit neuf points au-dessus. Sur ce jeu, les deux populations ne se chevauchent donc pas.

Ce qui précède établit le seuil de 60/100 comme paramètre opérationnel pour cette version de la plateforme, non comme un optimum démontré. Sur les seize profils du Gold Set, 60 et 50 produisent exactement les mêmes classifications : leur écart n'est donc pas observable sur ce jeu, et rien n'y établit la supériorité de 60 sur 50. Seule la remontée à 70 s'en distingue, en introduisant un faux négatif. Le seuil retenu relève d'un calibrage préliminaire : une évaluation sur un corpus plus large et représentatif, complétée par le suivi des erreurs observées en conditions réelles, sera nécessaire pour le confirmer ou l'ajuster.

Ces résultats demandent à être lus avec prudence : seize cas construits pour l'évaluation ne constituent pas une mesure de performance en conditions réelles. Le Gold Set valide un mécanisme

de décision et un calibrage de seuil ; il ne remplace pas le suivi, en production, des faux positifs et des faux négatifs effectivement constatés par les recruteurs sur les candidatures réelles.

### *4.4.6 Validation humaine du score et explicabilité*

Un moteur de présélection n'est fiable que s'il sait dire quand il ne l'est pas. Le service de scoring positionne le drapeau needsHumanReview et un motif lisible dans trois situations : texte extrait du CV trop court — moins de cinquante caractères, ce qui correspond à un scan, à une image ou à un fichier dégradé ; score en zone grise entre 55 et 65, c'est-à-dire de part et d'autre du seuil ; mode dégradé enfin, lorsque l'API de Cohere est indisponible ou renvoie une réponse inexploitable et que le score, calculé localement, est marqué avec la source SIMULATED. Le dossier signalé est présenté au recruteur comme demandant un arbitrage, au lieu d'être accepté ou rejeté automatiquement. Les deux mécanismes sont complémentaires et non contradictoires : un score de 62 franchit le seuil de décision et bascule la candidature en CV_REVIEWED, tout en tombant dans la zone grise, ce qui allume également le drapeau d'arbitrage ; inversement, un score de 57 reste sous le seuil mais, signalé en zone grise, il est lui aussi orienté vers CV_REVIEWED plutôt que vers AUTO_REJECTED, pour être tranché par un recruteur.

L'explicabilité est produite par le service SkillNormalizationService, appelé avant le modèle de langage : il harmonise les alias (k8s devient Kubernetes, postgres devient PostgreSQL) et refuse les fusions abusives — Spring Boot n'est pas Spring Cloud, Angular n'est pas AngularJS, Java n'est pas JavaScript — puis restitue les compétences validées (matchedSkills) et manquantes (missingSkills) avec un taux de recouvrement technique qui ne peut pas descendre sous 30/100. Ces éléments alimentent les badges affichés dans le détail d'une candidature (figure 4.9) et la synthèse enregistrée dans aiSummary [23].

L'arbitrage du recruteur est lui aussi tracé. Le point d'entrée PATCH /api/candidates/{id}/ai-score/override, réservé aux rôles RH et Administrateur, exige un score compris entre 0 et 100 et une justification non vide. Il conserve le score original dans aiScore, enregistre la correction dans manualScore, manualScoreReason, manualScoreBy et manualScoreDate, réinitialise needsHumanReview et écrit une entrée d'audit de type SCORE_OVERRIDE mentionnant la valeur précédente, la valeur nouvelle et le motif. Le statut est ensuite recalculé à partir du score effectif, ce qui permet à un dossier bloqué en zone grise de repartir normalement dans le pipeline.

J'ai vérifié ce parcours de bout en bout sur un dossier volontairement placé en zone grise : signalement dans l'interface, correction justifiée depuis un compte RH, puis contrôle de la ligne correspondante dans le journal d'audit (figure A.5). Le score produit par l'IA restant consultable après coup, l'écart entre l'évaluation automatique et l'expertise humaine peut être mesuré et documenté.

## 4.5 Difficultés rencontrées et solutions apportées

Au-delà des fonctionnalités livrées, le développement de JOB4YOU a été jalonné de plusieurs difficultés concrètes. Je détaille ici les principales, représentatives des problèmes rencontrés sur ce projet.

### *4.5.1 Notification incomplète des managers*

Un premier bug, découvert tardivement, concernait le routage automatique RH → manager. Lorsqu'une offre d'emploi était rattachée à une famille de métier suivie par plusieurs managers, seul

le premier manager trouvé en base recevait la notification de validation RH : les autres n'étaient jamais informés, sans qu'aucune erreur ne soit levée. Le problème venait d'une méthode qui retournait un unique e-mail (le premier résultat de la requête) au lieu de la liste complète des managers concernés. La correction a consisté à faire retourner l'ensemble des adresses correspondantes et à notifier chaque manager individuellement, tout en veillant à ne pas dupliquer l'e-mail envoyé au candidat, qui doit rester unique quel que soit le nombre de managers notifiés. Ce routage a par la suite été affiné pour s'appuyer prioritairement sur le département de l'offre (CS, DevOps, ProdOps, R&D, QA), la famille de métier n'intervenant plus qu'en repli pour les offres non rattachées à un département : un manager ne reçoit et ne voit désormais que les dossiers relevant strictement de son département, sans fuite possible vers un autre périmètre.

### *4.5.2 Distinguer un avis manager d'une décision finale RH*

Le tableau de bord devait afficher séparément les rejets RH et les rejets manager, ces derniers ventilés par famille de métier, alors que le modèle ne connaissait qu'un statut unique pour les deux. Plutôt que d'empiler des statuts, j'ai fait porter la distinction par le cycle de vie lui-même : MANAGER_REJECTED désigne aujourd'hui un refus proposé par le manager, en attente de contreseing, tandis que REJECTED désigne le refus final prononcé ou contre-signé par le RH, et HIRED l'embauche validée par lui. Le motif enregistré (statusReason) et l'auteur de l'écriture complètent l'information ; le journal d'audit, où chaque changement de statut est déjà associé à l'utilisateur qui l'a effectué, reste croisé avec son rôle réel pour ventiler les rejets manager par famille de métier.

### *4.5.3 Cohérence des transitions de statut*

Le cycle de vie d'une candidature comporte de nombreux états, et plusieurs actions (planifier un entretien, prendre une décision finale) n'ont de sens que si le dossier se trouve dans un état précis. Sans garde-fou, il était possible par exemple de planifier un entretien sur un dossier déjà rejeté, ou de faire valider un candidat par un manager avant même que le RH n'ait examiné son dossier. J'ai donc ajouté, côté back-end, des vérifications explicites avant chaque transition sensible, avec des messages d'erreur clairs renvoyés à l'interface plutôt que des échecs silencieux — un choix qui a nécessité de revenir plusieurs fois sur le diagramme d'états du chapitre 3 pour m'assurer que l'implémentation restait fidèle au modèle.

### *4.5.4 Filtrage incomplet des routes de lecture pour les managers*

Le garde-fou décrit en 4.5.3 empêchait bien un manager d'agir en écriture (changer un statut) sur un candidat hors de son périmètre, mais je me suis rendu compte, en réexaminant systématiquement chaque route accessible au rôle Manager, que ce n'était pas le cas des routes de lecture : GET /api/candidates, GET /api/candidates/{id}, la recherche par nom et le classement par offre ne filtraient pas par périmètre. Un manager authentifié pouvait donc, en appelant directement l'API (pas via l'interface, qui n'expose pas ce chemin), lister ou consulter des candidats d'un autre département. J'ai corrigé ce point en centralisant la logique de périmètre déjà utilisée pour les écritures dans une méthode réutilisable, appliquée à présent à toutes les routes de lecture concernées, et j'ai ajouté deux tests unitaires dédiés (section 4.4.1) pour figer ce comportement.

### *4.5.5 Solliciter le manager au bon moment du processus*

Dans une première version, le manager destinataire était notifié dès la validation du CV par le RH (statut CV_REVIEWED). À l'usage, cela le sollicitait trop tôt : de nombreux dossiers étaient encore écartés lors du pré-filtrage téléphonique mené par le RH. J'ai donc déplacé la notification du

manager à l'entrée en Test technique, c'est-à-dire une fois l'entretien téléphonique franchi ; le candidat, lui, reste informé de la présélection de son CV dès l'étape CV_REVIEWED. En cohérence, la vue des dossiers validés du manager ne fait plus apparaître les candidats qu'à partir du Test technique, ce qui recentre son attention sur les profils réellement en lice.

### *4.5.6 Pertinence du matching Job idéal*

La fonctionnalité de matching inverse, par laquelle un visiteur dépose son CV et le système lui suggère les offres les plus proches, retournait des résultats peu pertinents : un profil DevOps se voyait par exemple proposer des postes de développement back-end. L'analyse a révélé deux causes : le score ne tenait compte que des compétences requises, en ignorant l'intitulé de l'offre, et le jeu de données comportait de nombreuses annonces en double qui saturaient le lot soumis à l'IA. J'ai corrigé les deux points en combinant, dans le score, l'intitulé de l'offre (pondéré à 60 %) et les compétences (40 %), et en dédupliquant les offres identiques avant l'appel à l'IA. Chaque profil (Java, DevOps, ProdOps, QA) ressort désormais sur les offres qui lui correspondent réellement.

### *4.5.7 Traitement distinct des candidatures retirées*

Une candidature retirée (statut WITHDRAWN) était jusque-là confondue avec les candidatures rejetées dans les filtres de la liste, et cette liste ne se mettait à jour qu'après rechargement de la page. J'ai séparé le retrait du rejet en lui dédiant son propre filtre, à l'image du filtre des candidatures rejetées, dans la liste des candidatures comme sur le tableau de bord, et fait en sorte que tout changement de statut rafraîchisse immédiatement la liste : un candidat passé en candidature retirée quitte aussitôt la vue active et le compteur est décrémenté, sans rechargement. Ces candidatures ne sont pas supprimées automatiquement : elles restent consultables via leur filtre, comme les candidatures rejetées. Un candidat peut aussi retirer lui-même sa candidature tant qu'elle n'est pas close, via un endpoint dédié qui ne passe pas par les actions réservées au RH ou au manager.

### *4.5.8 Entretien téléphonique RH obligatoire avant acceptation*

En testant le parcours RH de bout en bout, je me suis rendu compte qu'un candidat pouvait être accepté directement depuis CV_REVIEWED, sans qu'aucun entretien téléphonique n'ait eu lieu : la machine à états ne l'interdisait pas explicitement. Cela allait à l'encontre du processus voulu, où cet entretien constitue le premier vrai filtre humain avant de solliciter le manager. J'ai donc retiré la transition CV_REVIEWED vers ACCEPTED : depuis cette étape, seuls restent possibles le passage en entretien téléphonique, le refus ou le retrait. Un profil manifestement hors sujet peut toujours être écarté sans attendre cet entretien ; c'est uniquement l'acceptation qui exige désormais d'être passé par PHONE_SCREENING.

### *4.5.9 Découplage entre planification d'entretien et statut du candidat*

Ce même exercice a fait remonter un second problème, plus discret : planifier un entretien changeait le statut du candidat selon le type choisi, mais sans passer ni par le garde-fou décrit en 4.5.3, ni par le contrôle de périmètre du manager. Concrètement, planifier un entretien téléphonique sur un dossier déjà en phase technique suffisait à faire régresser son statut, sans qu'aucune vérification ne s'y oppose. J'ai séparé les deux responsabilités : la planification reste une opération de logistique (date, intervieweur, lieu) qui n'a plus aucun effet sur le statut ; seules des actions de statut dédiées, elles-mêmes protégées par des garde-fous, font avancer le dossier. Pour ne pas perdre en confort d'usage,

l'interface garde une case à cocher optionnelle qui déclenche la transition correspondante en un clic, mais via l'endpoint protégé par ce même garde-fou, donc sans contourner la règle.

### *4.5.10 Fiabiliser l'intégration Google Meet et le format des webhooks n8n*

Le lien Google Meet envoyé aux candidats et aux managers reposait initialement sur une valeur fictive, générée côté n8n sans appel réel à l'API Google Calendar — une limitation assumée à ce stade du développement. En remplaçant ce mécanisme par une véritable création d'événement Google Calendar (avec génération automatique du lien Meet), un problème plus ancien est remonté : le nœud Webhook de n8n imbrique le corps réel de la requête POST sous un champ body, alors que le workflow lisait les données à la racine. Ce défaut, antérieur à cette évolution, cassait silencieusement à la fois le routage de l'agent d'entretien et, potentiellement, celui de l'agent de décision finale. Je l'ai corrigé aux trois endroits concernés, puis validé le nouveau flux de bout en bout : création réelle de l'événement, ajout des participants, envoi de l'e-mail de convocation, et sauvegarde du lien via un point d'API dédié. Pour que cet appel à n8n reste fiable même en cas d'indisponibilité passagère, le back-end s'appuie sur une file d'attente interne (outbox) : chaque déclenchement est journalisé, puis retenté automatiquement en cas d'échec avec un délai croissant (5 s, 10 s, 20 s, 40 s puis 80 s ; le plafond théorique de 300 s n'est jamais atteint avec cinq relances), soit au plus 6 tentatives au total, ce qui évite de perdre un événement sans pour autant bloquer la requête initiale.

### *4.5.11 Cloisonnement des données dans le chatbot candidat*

L'ajout d'un assistant conversationnel candidat posait un risque spécifique : un modèle de langage peut, si on le laisse faire, être incité par un utilisateur malveillant à révéler des informations qu'il ne devrait pas partager, y compris celles qu'on lui a explicitement fournies en contexte. J'ai donc construit le contexte transmis au modèle en excluant dès la source, avant même l'appel à l'IA, le score et le résumé générés par l'agent de scoring, la correction manuelle du RH, l'identité de l'intervieweur et les notes internes d'entretien — ces champs ne quittent jamais le back-end vers le prompt. Le message système impose en complément au modèle de ne répondre qu'à partir de ce contexte filtré, d'ignorer toute instruction contraire glissée dans la question du candidat, et de ne jamais reproduire ce contexte mot pour mot. Ces deux niveaux de protection — filtrage des données à la source, puis consignes du prompt système — ont été validés par des tests manuels explicites : demande directe du score, de l'identité de l'intervieweur, et tentative d'injection de prompt (« ignore tes instructions précédentes... ») ; dans les trois cas, l'assistant refuse poliment sans divulguer l'information protégée. Ces tests manuels couvrent les scénarios éprouvés et réduisent le risque de divulgation ; ils ne constituent pas une garantie absolue face à toutes les formes d'injection.

## 4.6 Protection des données et éthique de l'IA

La plateforme manipule des données à caractère personnel : identité, coordonnées, CV et évaluations des candidats. Plusieurs principes ont guidé leur traitement. L'accès aux dossiers est strictement contrôlé selon le rôle de l'utilisateur, comme détaillé au tableau de la section 2.2.1 : un manager ne voit que les candidatures de son périmètre, seuls RH et Administrateur disposant d'une vue d'ensemble. Les mots de passe sont hachés (section 4.2.4), et chaque action sensible sur un dossier candidat est journalisée dans le journal d'audit avec l'identité de son auteur et un horodatage — de quoi retracer qui a modifié un dossier donné. Plusieurs protections techniques complètent ce socle : une limitation du nombre de tentatives de connexion pour freiner les attaques

par force brute et un garde-fou contre les redirections internes non autorisées sur le point d'API utilisé pour tester la connectivité des webhooks n8n. Une validation du contenu réel des fichiers déposés, au-delà de la seule extension déclarée, reste en revanche une amélioration prévue pour la mise en production (section 4.2.4).

Le score attribué par l'agent d'intelligence artificielle est une aide à la présélection : selon la règle configurée, il oriente automatiquement le dossier, un score insuffisant déclenchant un rejet automatique qui, à ce stade, n'est pas précédé d'un examen humain. Ce rejet reste réversible par un RH ou un Administrateur via le mécanisme de surcharge manuelle, avec justification obligatoire et trace dans le journal d'audit, tandis que les dossiers signalés needsHumanReview ne sont jamais rejetés automatiquement. Cette conception limite le risque d'erreur ou de biais algorithmique sans équivaloir à une validation humaine de chaque décision, dans l'esprit des principes du RGPD [17], [18] relatifs aux décisions individuelles automatisées.

Ce dispositif comporte néanmoins des limites qu'il convient d'assumer : la dépendance à un service d'IA externe (Cohere) et à sa disponibilité, la sensibilité du score à la qualité et à la structure du CV fourni, le risque de biais inhérent à tout modèle statistique, et l'absence volontaire de décision entièrement autonome — chaque rejet automatique restant réversible par un opérateur humain. Pour ne jamais bloquer le dépôt d'une candidature en cas d'indisponibilité ou d'erreur de l'API de Cohere, un mode dégradé prend automatiquement le relais : le score est alors calculé par une heuristique locale (analyse de mots-clés et de structure du texte), la source du score (COHERE ou SIMULATED) étant conservée pour traçabilité. Ces limites justifient une traçabilité complète des décisions prises et un examen humain systématique des dossiers signalés par le drapeau d'arbitrage ; les autres rejets automatiques ne font, en revanche, pas l'objet d'une validation humaine préalable. La conception du processus de recrutement doit aussi tenir compte des principes de diversité et d'inclusion, afin de limiter les risques de discrimination dans des processus de sélection automatisés [19]. Par ailleurs, les captures d'écran présentées dans ce rapport (section 4.3) utilisent des données de test fictives ou anonymisées, à l'exception de quelques comptes de démonstration que j'ai moi-même créés à des fins de test.

Le chatbot candidat introduit en fin de projet (section 4.5.11) applique les mêmes principes : il ne s'appuie que sur les données de candidature du candidat connecté, dérivées côté serveur à partir du jeton JWT plutôt que d'un identifiant transmis par le client, et exclut structurellement les champs à usage interne (score IA, notes RH, identité de l'intervieweur) du contexte envoyé au modèle de langage.

Le traitement des données des candidats appelle quelques rappels, sans prétendre à une étude de conformité. Le scoring des CV transite par l'API de Cohere et l'assistant conversationnel par l'API OpenRouter : ces appels vers des services d'IA tiers doivent être encadrés (minimisation des données transmises, information des candidats, clauses contractuelles). Le modèle utilisé pour le chatbot est fixé par la propriété de configuration openrouter.model — z-ai/glm-5.3-flash dans la version livrée — , et ses conditions de conservation des données doivent être vérifiées avant tout usage sur des données réelles. La plateforme ne prévoit pas encore de politique de durée de conservation et d'effacement des données (CV, évaluations) : c'est un point à définir et à outiller (archivage, purge automatique, exercice du droit à l'effacement). Enfin, un système de scoring utilisé en recrutement relève des principes qui encadrent, dans l'Union européenne, les décisions individuelles automatisées [17], [18] et les systèmes d'IA à « haut risque » [20] : traçabilité renforcée, supervision humaine effective et documentation du modèle — principes partiellement mis en œuvre ici (surcharge manuelle justifiée, journal d'audit, arbitrage humain possible), le rejet automatique n'étant en revanche pas précédé d'une validation humaine systématique.

## 4.7 Bilan des fonctionnalités réalisées

Le tableau suivant récapitule, par grand domaine fonctionnel, l'état d'avancement des développements à l'issue du projet.

*Tableau 4.3 : Bilan des fonctionnalités réalisées*

| **Fonctionnalité** | **État** |
|---|---|
| Authentification et gestion des rôles (JWT, RBAC) | Réalisée |
| Gestion des offres d'emploi (création, publication, expiration automatique) | Réalisée |
| Gestion des candidatures et des CV | Réalisée |
| Scoring IA des candidatures (API de Cohere) et paramétrage par famille de métier | Réalisée |
| Gestion des entretiens (planification, statuts, rappels, lien Google Meet réel généré automatiquement) | Réalisée |
| Gestion des feedbacks (saisie, validation RH) | Réalisée |
| Routage automatique RH → manager par département (isolation stricte, repli famille de métier) | Réalisée |
| Notifications automatiques par e-mail (agents n8n) | Réalisée |
| Tableau de bord et pipeline de recrutement (Kanban) | Réalisée |
| Journal d'audit (traçabilité des actions) | Réalisée |
| Matching « Job idéal » (suggestion d'offres à partir d'un CV) | Réalisée |
| Entretien téléphonique RH obligatoire avant acceptation | Réalisée |
| Calendrier manager dédié (lecture seule, cloisonné par périmètre) | Réalisée |
| Retrait de candidature en libre-service par le candidat | Réalisée |
| Assistant conversationnel candidat (chatbot IA via OpenRouter, connaissance des candidatures réelles et de la plateforme) | Réalisée |
| Évaluation du matching sur un Gold Set de seize CVs annotés | Réalisée |
| Human-in-the-Loop : signalement des dossiers à arbitrer (needsHumanReview) | Réalisée |
| Explicabilité du score : compétences validées / manquantes, source du score | Réalisée |
| Décision à deux niveaux : avis manager contre-signé par le RH (convocation ou refus motivé) | Réalisée |

## 4.8 Conclusion

Entre l'environnement de travail, les interfaces effectivement livrées, les tests menés et les difficultés qu'il a fallu résoudre en cours de route, ce chapitre reflète assez fidèlement le quotidien du développement de JOB4YOU. Reste maintenant à prendre du recul sur l'ensemble du projet — c'est l'objet de la conclusion générale qui suit.

# Conclusion Générale et Perspectives

Au terme de la période de développement de ce Projet de Fin d'Études, JOB4YOU est passé d'un simple besoin identifié en tout début de stage à une plateforme fonctionnelle, capable de gérer le processus de recrutement interne de l'entreprise.

Ce travail couvre désormais l'ensemble du cycle de recrutement, de la publication d'une offre jusqu'à la décision finale d'embauche, en passant par la soumission des candidatures, l'analyse automatique des CV par intelligence artificielle, la planification des entretiens et la collecte des feedbacks des managers. La plateforme intègre également deux fonctionnalités différenciantes : le matching inverse « Job idéal », qui suggère à un candidat les offres les plus compatibles avec son CV, et un assistant conversationnel (chatbot) permettant à chacun de suivre ses candidatures en langage naturel — ce dernier étant strictement cloisonné aux données du candidat connecté, à l'exclusion de toute information interne (score IA, notes RH, identité de l'intervieweur). Un système de notifications automatiques par e-mail et un tableau de pilotage visuel de type Kanban complètent l'ensemble et donnent aux équipes RH et aux managers une visibilité en temps réel sur l'avancement de chaque dossier.

Sur le plan technique, ce projet m'a permis d'approfondir mes compétences en développement back-end avec Spring Boot et Spring Security, en développement front-end avec Angular, ainsi qu'en conception de bases de données relationnelles avec PostgreSQL. J'y ai aussi découvert et mis en œuvre des outils d'automatisation (n8n) et d'intelligence artificielle (API de Cohere) au service d'un processus métier concret, tout en renforçant mes compétences en sécurité applicative (authentification JWT, contrôle d'accès basé sur les rôles) et en gestion de projet, selon une démarche itérative inspirée d'Agile/Scrum. J'ai également vérifié le comportement du seuil de décision du scoring sur un jeu de référence annoté (Gold Set) et prévu, pour les cas limites, une validation humaine du score assortie d'une explication automatique des compétences retenues.

Au-delà de l'aspect purement technique, cette expérience au sein d'un grand groupe international m'a beaucoup apporté sur le plan professionnel : travail en équipe, communication régulière avec mon encadrant entreprise, prise en compte de contraintes réelles de sécurité et de fiabilité, et nécessité de concevoir une solution robuste et évolutive dans un contexte professionnel réel.

Plusieurs pistes d'évolution restent ouvertes pour poursuivre ce travail. Côté fonctionnel, la plateforme pourrait accueillir un module d'analyse prédictive du taux de transformation des candidatures, une intégration plus poussée avec des plateformes d'emploi externes (LinkedIn, Indeed), ou un espace candidat plus riche pour un suivi encore plus détaillé de son parcours. Côté technique, la suite de tests automatisés — qui compte déjà 121 méthodes de test unitaires et de contrôleur (JUnit 5 / Mockito) — gagnerait à être complétée par des tests d'intégration couvrant les services et la base de données, des tests de bout en bout (par exemple Playwright ou Cypress) rejouant les parcours métier complets, et des tests front-end automatisés sur les composants Angular. Le jeu de référence de seize profils mobilisé pour calibrer le seuil de décision gagnerait à être enrichi de candidatures réelles annotées par les recruteurs, seul moyen d'obtenir une mesure de performance en conditions d'exploitation. Une chaîne d'intégration et de déploiement continus (CI/CD) ainsi qu'un déploiement sur une infrastructure cloud constitueraient des évolutions naturelles pour renforcer la fiabilité et la scalabilité de la plateforme sur le long terme.

Les principales fonctionnalités prévues pour ce projet ont été développées et vérifiées à l'aide des scénarios décrits dans ce rapport : JOB4YOU répond aux besoins identifiés et couvre l'ensemble du cycle, de la publication d'une offre jusqu'à la décision finale. Les objectifs de performance restent en

revanche à mesurer, et plusieurs mesures de sécurité et de gouvernance des données doivent être finalisées avant une mise en préproduction — validation du contenu des fichiers téléversés, politique de rétention et d'effacement des données, encadrement des transferts vers Cohere et OpenRouter. Le système constitue donc un prototype fonctionnel, appelé à être industrialisé pour un déploiement chez Sopra HR Software Tunisie, plutôt qu'une solution dont l'aptitude à la production serait déjà démontrée. Cette expérience m'a permis de concevoir, sécuriser, tester et faire évoluer une solution logicielle complète.

## Annexe A — Captures d’écran complémentaires

Cette annexe regroupe dix captures complémentaires de la plateforme, citées dans le chapitre 4 mais non reproduites dans le corps du rapport afin d’en préserver la lisibilité. Comme les figures du corps, elles proviennent de l’application en fonctionnement.

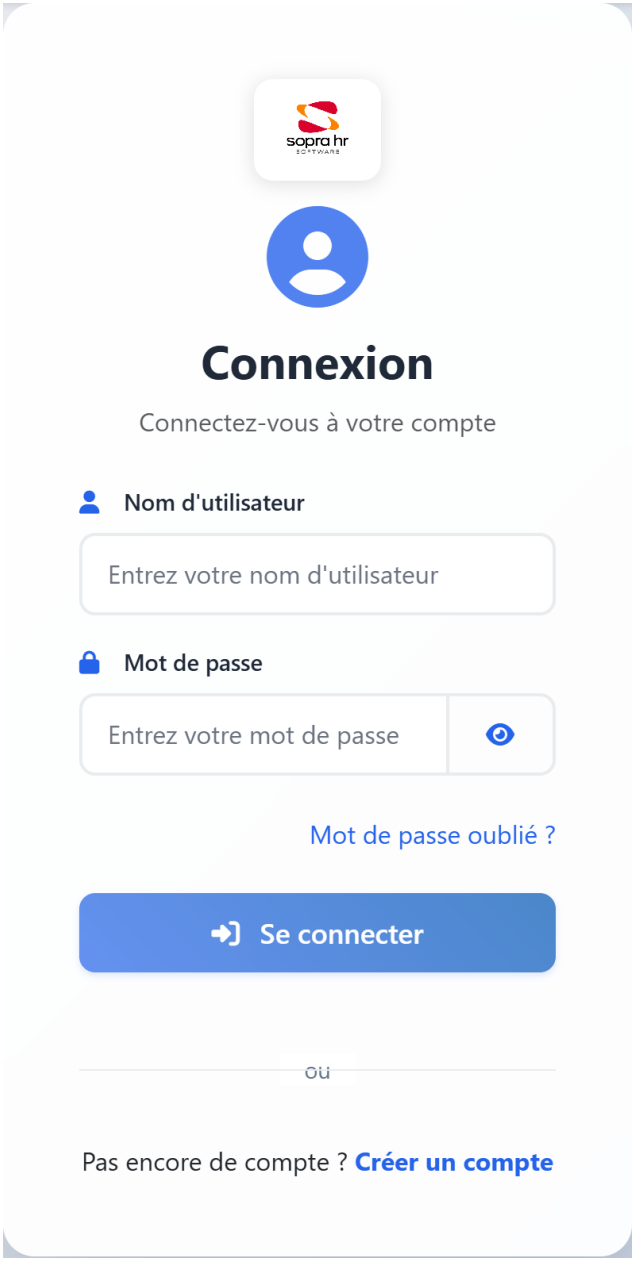



*Figure A.1 : Page de connexion*

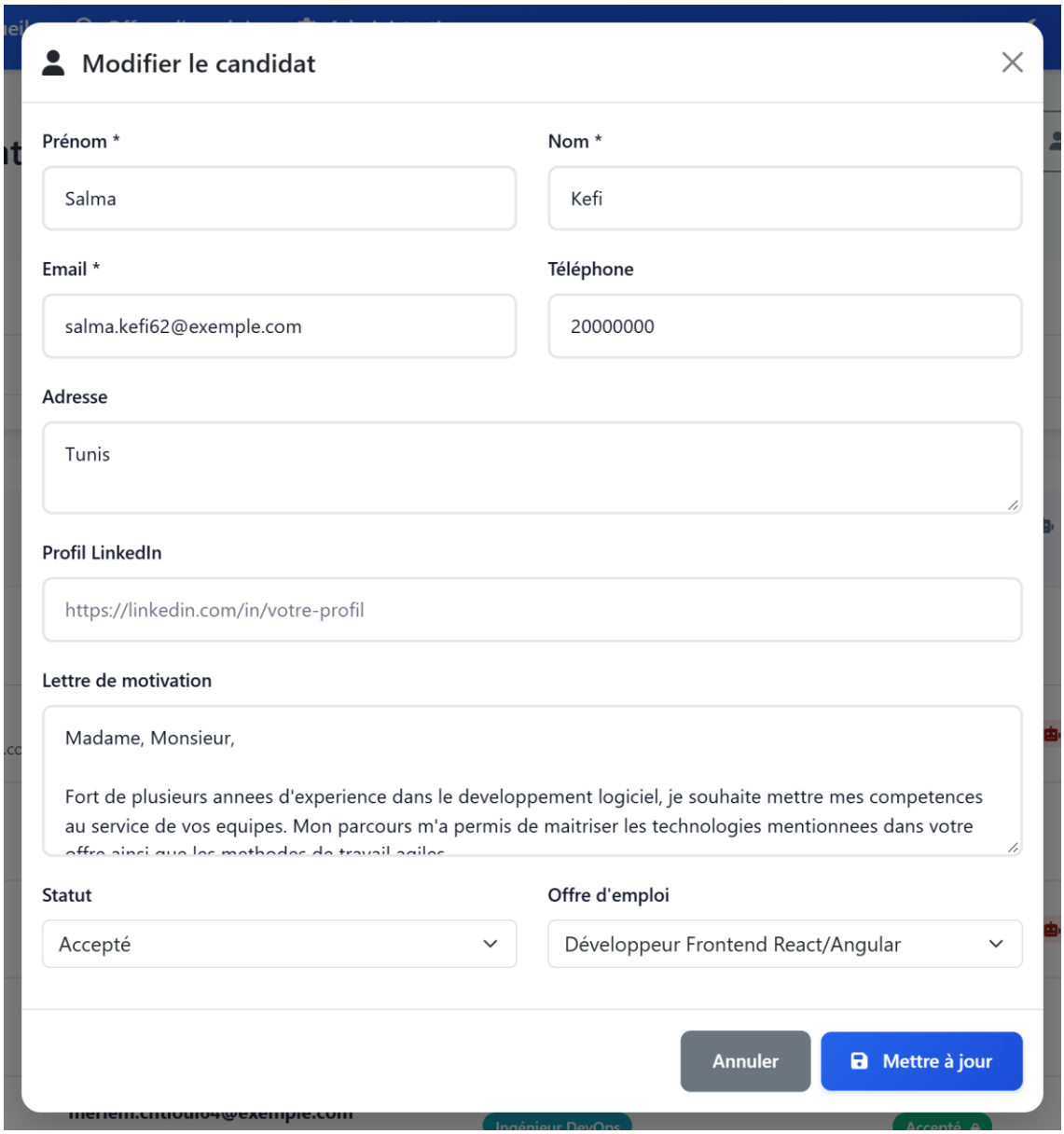



*Figure A.2 : Modification d'un dossier candidat*

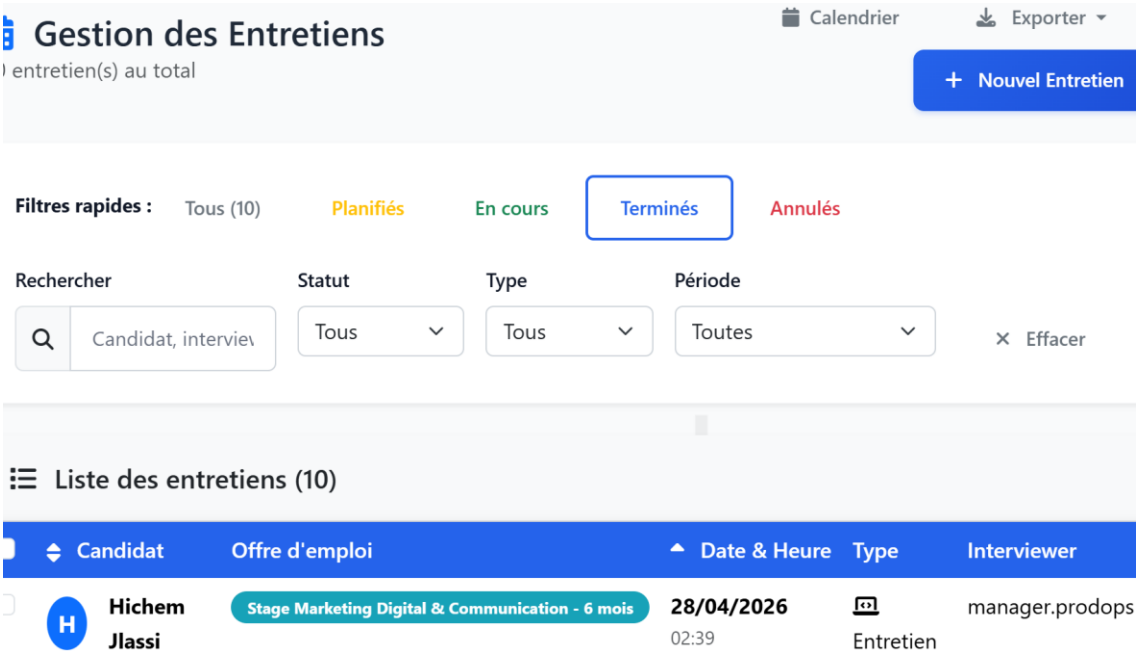



*Figure A.3 : Gestion des entretiens*

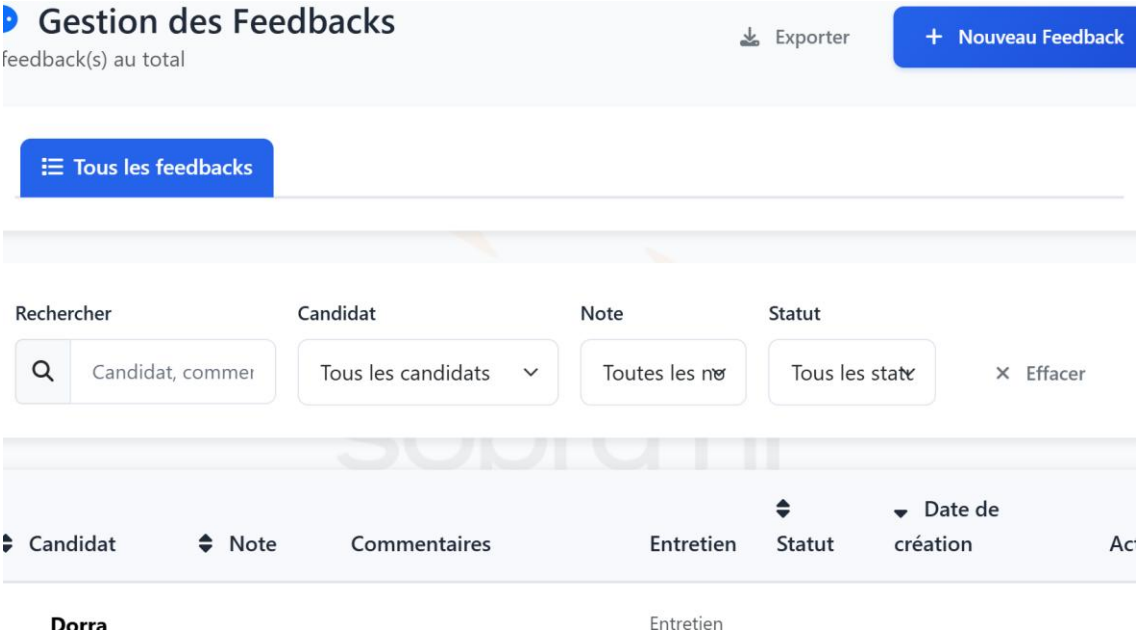



*Figure A.4 : Gestion des feedbacks*

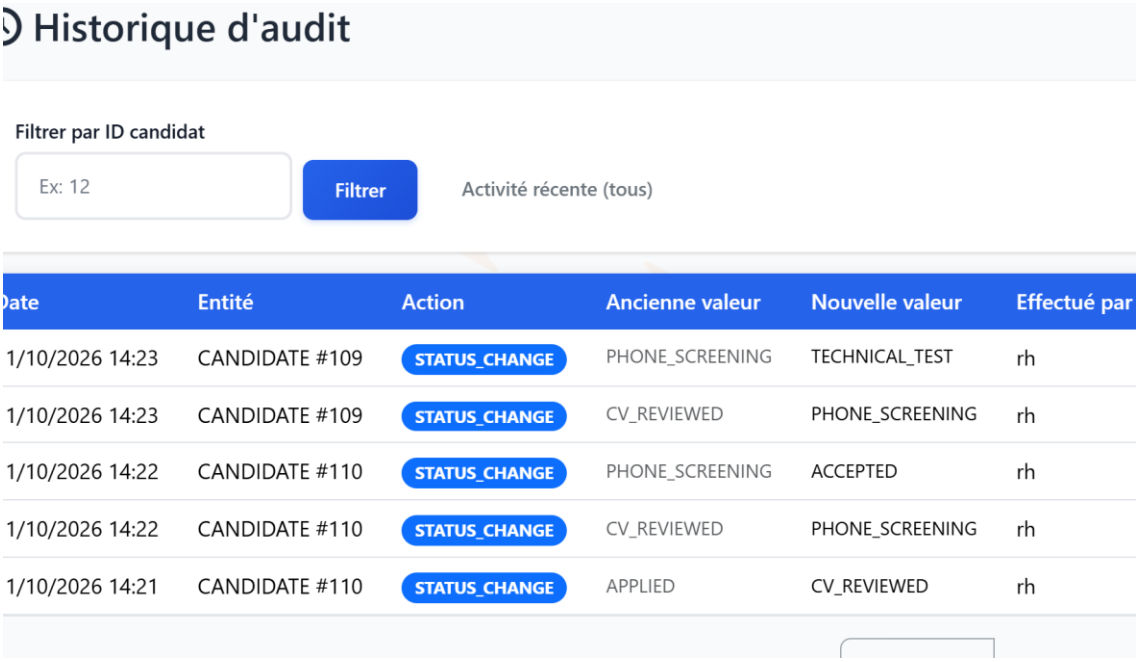



*Figure A.5 : Journal d'audit*

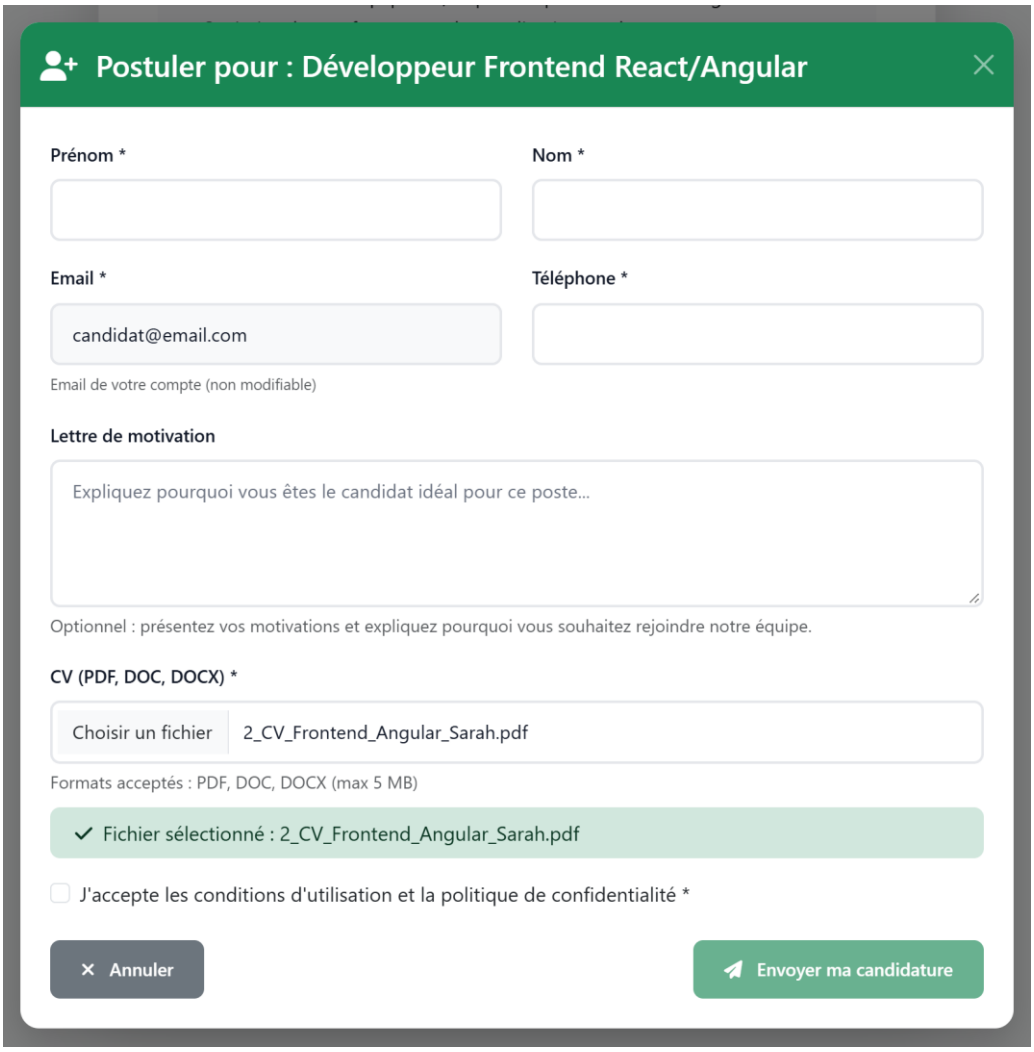



*Figure A.6 : Soumission d'une candidature avec dépôt de CV*

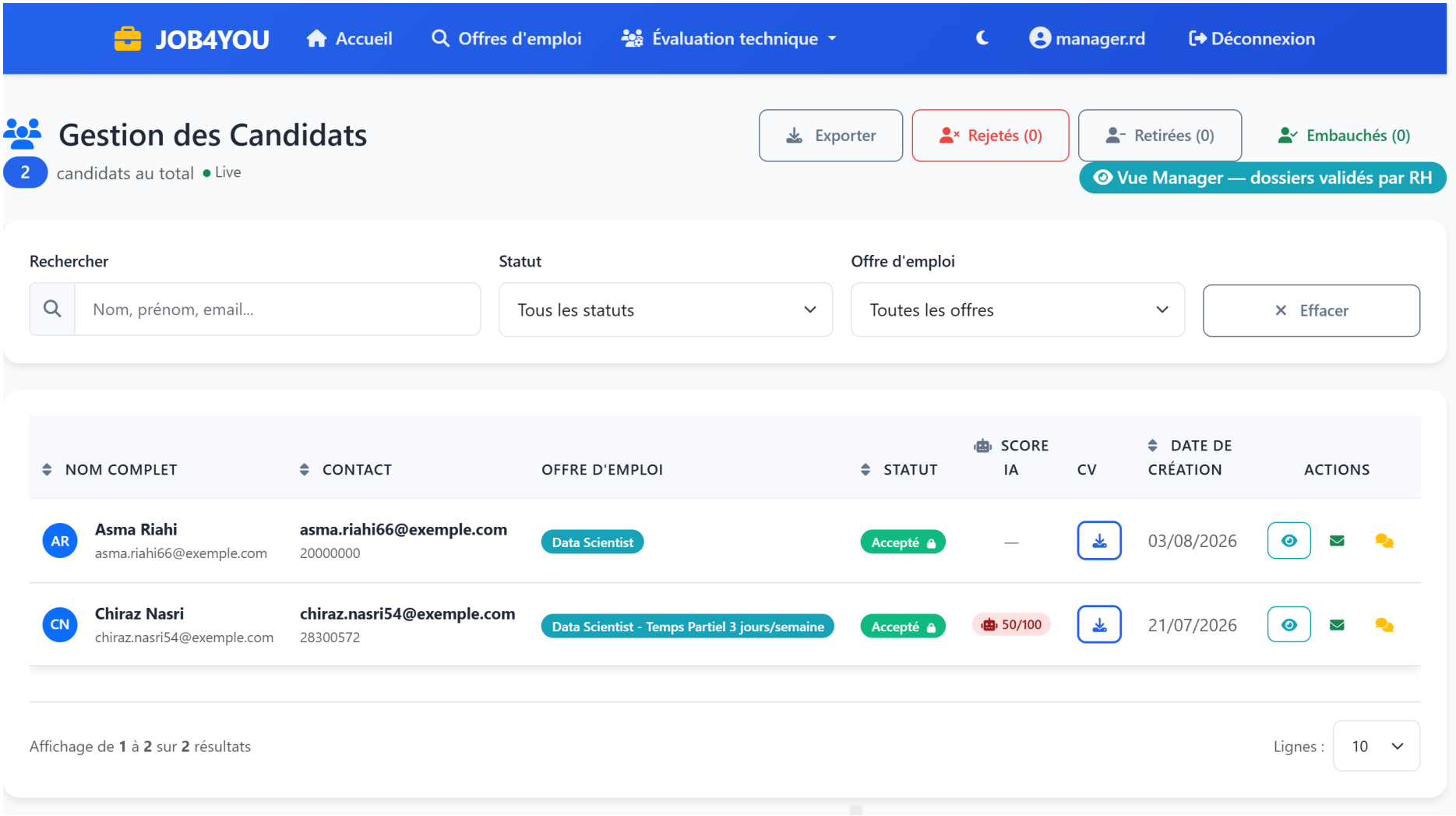



*Figure A.7 : Espace Manager — candidatures relevant du périmètre du manager*

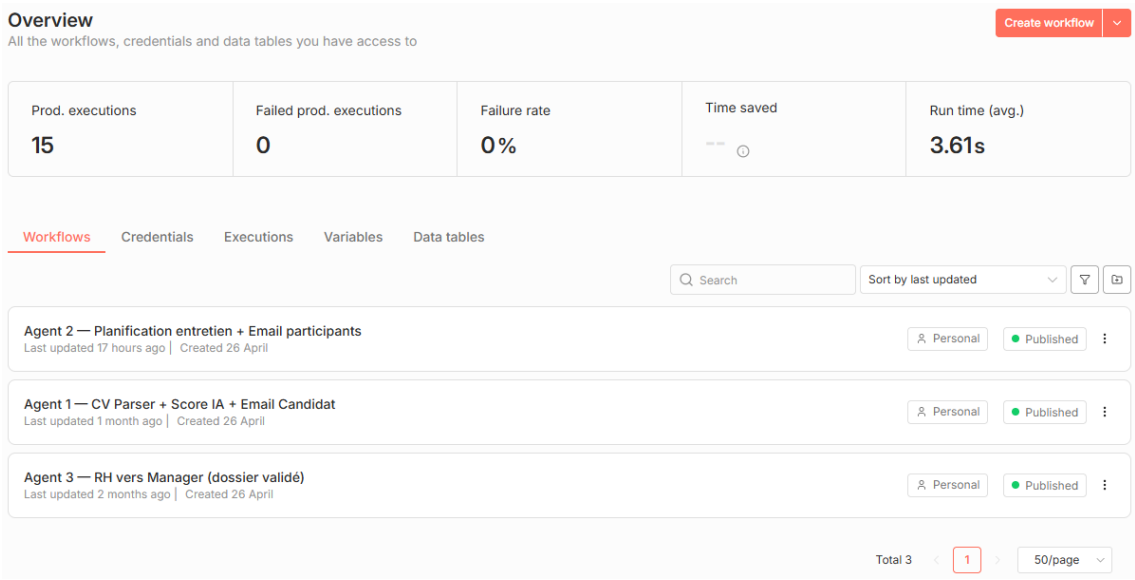



*Figure A.8 : Console n8n — les trois flux d'automatisation publiés et leurs statistiques*

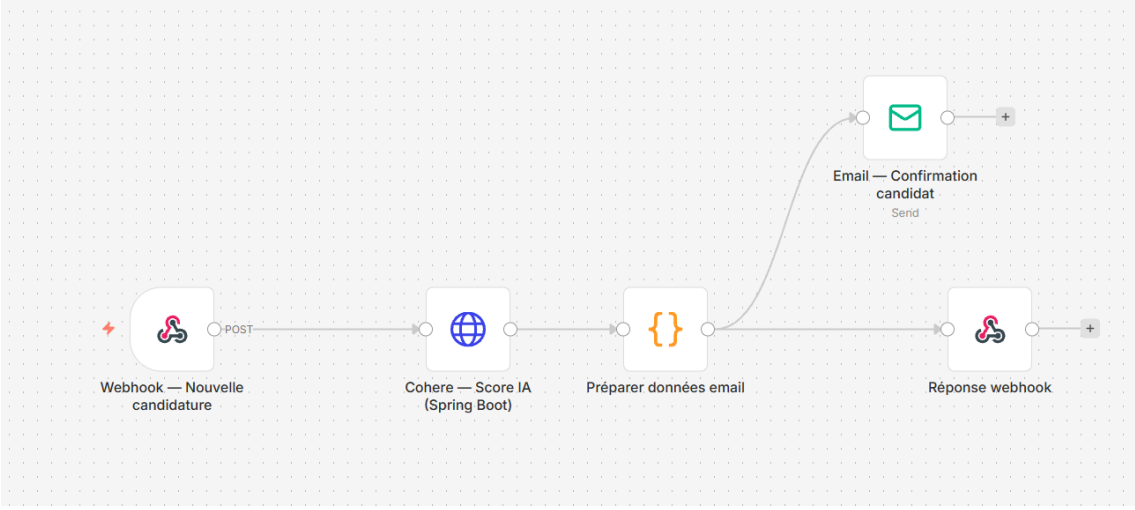
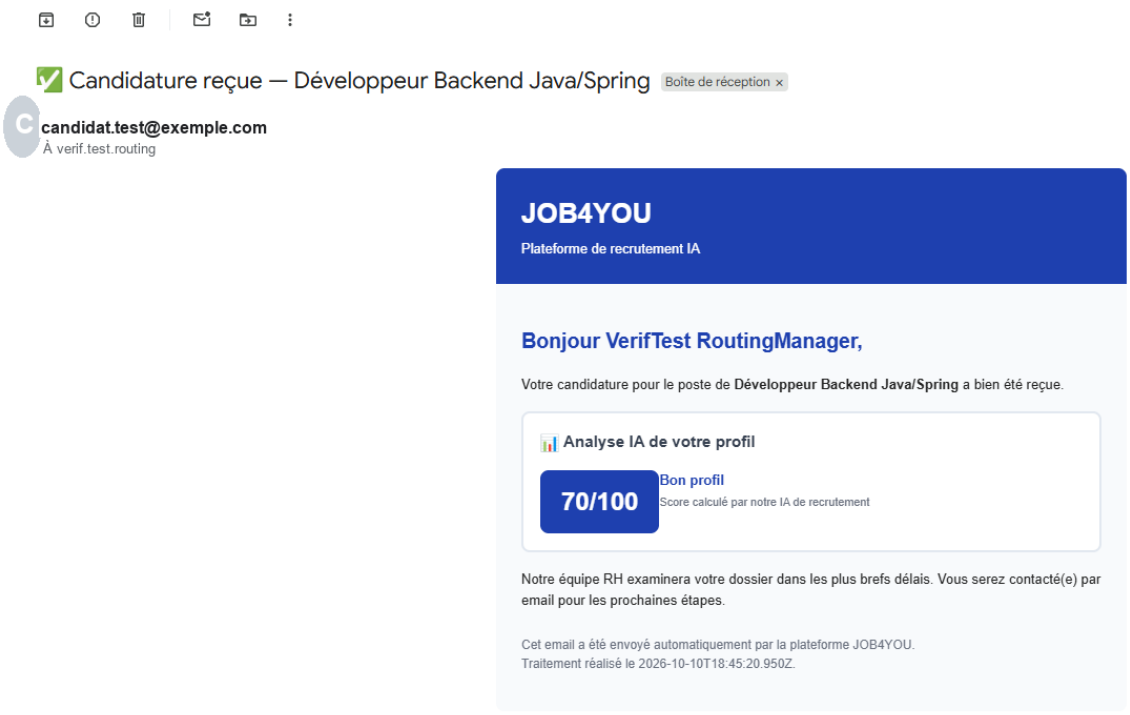



*Figure A.9 : Flux n8n de l'Agent 1 — webhook de candidature, appel du scoring Cohere puis e-mail au candidat*




---

*Figure A.10 : E-mail automatique de confirmation de candidature reçu par le candidat*

# Bibliographie


- [1] Sopra Steria — Site officiel du groupe. Disponible sur : https://www.soprasteria.com. Consulté le 09/02/2026.
- [2] SCRUM.ORG — The Scrum Guide (2020), Schwaber K. & Sutherland J. Disponible sur : https://scrumguides.org. Consulté le 16/02/2026.
- [3] OMG — Unified Modeling Language (UML) Specification. Disponible sur : https://www.omg.org/spec/UML. Consulté le 02/03/2026.
- [4] Spring Boot Reference Documentation. Disponible sur : https://docs.spring.io/spring-boot/. Consulté le 13/04/2026.
- [5] WALLS, Craig. Spring Boot in Action. Manning Publications, 2015.
- [6] Angular Documentation. Disponible sur : https://angular.dev. Consulté le 20/04/2026.
- [7] FREEMAN, Adam. Pro Angular. 3e édition, Apress, 2019.
- [8] PostgreSQL Documentation. Disponible sur : https://www.postgresql.org/docs/. Consulté le 27/04/2026.
- [9] n8n Documentation. Disponible sur : https://docs.n8n.io. Consulté le 02/06/2026.
- [10] Google Calendar API — Documentation développeur. Disponible sur : https://developers.google.com/calendar. Consulté le 09/06/2026.
- [11] Cohere API Documentation. Disponible sur : https://docs.cohere.com. Consulté le 15/06/2026.
- [12] OpenRouter — Documentation de l'API. Disponible sur : https://openrouter.ai/docs. Consulté le 19/06/2026.
- [13] Spring Security Reference Documentation. Disponible sur : https://docs.spring.io/spring-security/reference/. Consulté le 08/05/2026.
- [14] JSON Web Token (JWT) — RFC 7519. Disponible sur : https://www.rfc-editor.org/rfc/rfc7519. Consulté le 11/05/2026.
- [15] OWASP Top 10 — Application Security Risks. Disponible sur : https://owasp.org/www-project-top-ten/. Consulté le 03/07/2026.
- [16] JUnit 5 et Mockito — Documentation. Disponibles sur : https://junit.org/junit5/ et https://site.mockito.org. Consulté le 10/07/2026.
- [17] CNIL — Intelligence artificielle et RGPD, fiches pratiques. Disponible sur : https://www.cnil.fr/fr/intelligence-artificielle. Consulté le 20/07/2026.

- [18] Règlement (UE) 2016/679 (RGPD), article 22 — Décision individuelle automatisée. Disponible sur : https://eur-lex.europa.eu/eli/reg/2016/679/oj. Consulté le 22/07/2026.
- [19] ISO 30415:2021 — Management des ressources humaines : Diversité et inclusion.
- [20] Règlement (UE) 2024/1689 (AI Act) — systèmes d'intelligence artificielle à haut risque. Disponible sur : https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Consulté le 24/07/2026.
- [21] VAISHAMPAYAN, Swanand, LEARY, Hunter, ALEBACHEW, Yoseph Berhanu, HICKMAN, Louis, STEVENOR, Brent A., BECK, Weston et BROWN, Chris. Human and LLM-Based Resume Matching: An Observational Study. Dans : Findings of the Association for Computational Linguistics: NAACL 2025, 2025, p. 4808-4823. Disponible sur : https://doi.org/10.18653/v1/2025.findings-naacl.270. Consulté le 13/10/2026.
- [22] REZAEIPOURFARSANGI, Sima et MILIOS, Evangelos E. AI-powered Resume-Job matching. Dans : Proceedings of the ACM Symposium on Document Engineering 2023 (DocEng '23), 2023, p. 1-4. Disponible sur : https://doi.org/10.1145/3573128.3609347. Consulté le 13/10/2026.
- [23] KHELKHAL, Kenza et LANASRI, Dihia. Smart-Hiring: An Explainable end-to-end Pipeline for CV Information Extraction and Job Matching. arXiv, 2025, réf. arXiv:2511.02537. Disponible sur : https://arxiv.org/abs/2511.02537. Consulté le 13/10/2026.

## Glossaire

- API : Application Programming Interface — interface permettant à deux systèmes logiciels de communiquer.
- BCrypt : algorithme de hachage adaptatif utilisé pour le stockage sécurisé des mots de passe.
- CI/CD : Continuous Integration / Continuous Delivery, intégration et livraison continues.
- Cohere : fournisseur d'API d'intelligence artificielle utilisé pour l'analyse sémantique et le scoring des CV.
- IA : Intelligence Artificielle.
- JPA : Java Persistence API — spécification Java pour la persistance des objets en base de données relationnelle.
- JWT : JSON Web Token — jeton signé utilisé pour authentifier les requêtes entre le front-end et le back-end.
- Kanban : méthode de visualisation du flux de travail sous forme de colonnes représentant les étapes d'un processus.
- LLM : Large Language Model — modèle de langage de grande taille.
- n8n : plateforme d'automatisation permettant de construire des workflows interconnectant différents services et API via des webhooks.
- ORM : Object-Relational Mapping, correspondance entre objets applicatifs et tables relationnelles (via JPA/Hibernate).
- Outbox : file d'attente interne garantissant la livraison fiable des événements (webhooks n8n), avec journalisation et nouvelles tentatives en cas d'échec.
- OWASP : Open Worldwide Application Security Project — référentiel des principaux risques de sécurité applicatifs.
- RBAC : Role-Based Access Control — contrôle d'accès basé sur les rôles des utilisateurs.
- REST : Representational State Transfer — style d'architecture pour les API web.
- RGPD : Règlement Général sur la Protection des Données (règlement (UE) 2016/679).
- SGBD : Système de Gestion de Base de Données.
- SMTP : Simple Mail Transfer Protocol — protocole d'envoi de courriel.
- SPA : Single Page Application — application web monopage.

- Spring Security : module du framework Spring assurant l'authentification et le contrôle d'accès des applications Java.
- UML : Unified Modeling Language — langage de modélisation graphique utilisé pour la conception logicielle.
- Webhook : mécanisme par lequel une application notifie automatiquement une autre via une requête HTTP déclenchée par un événement.